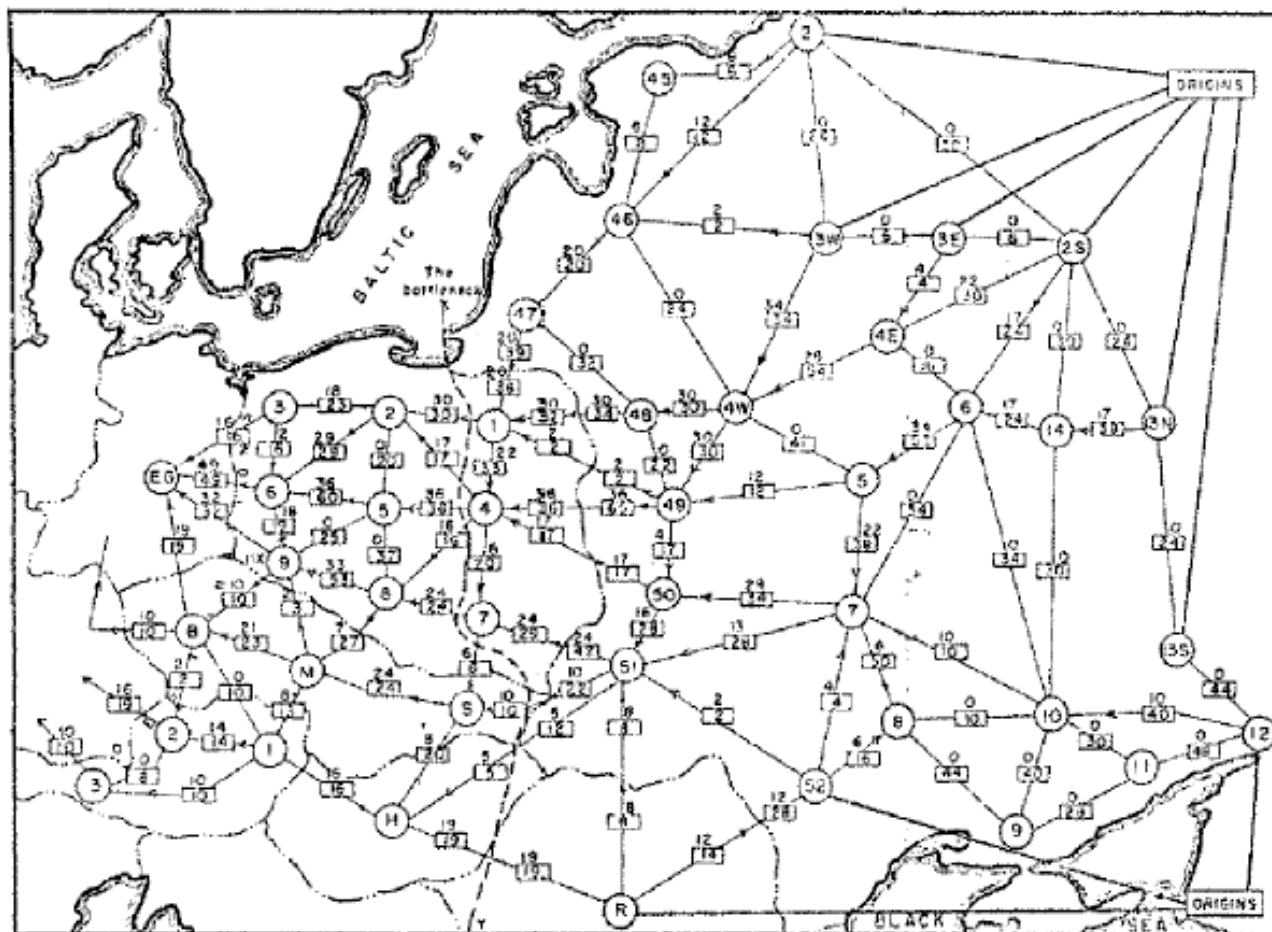


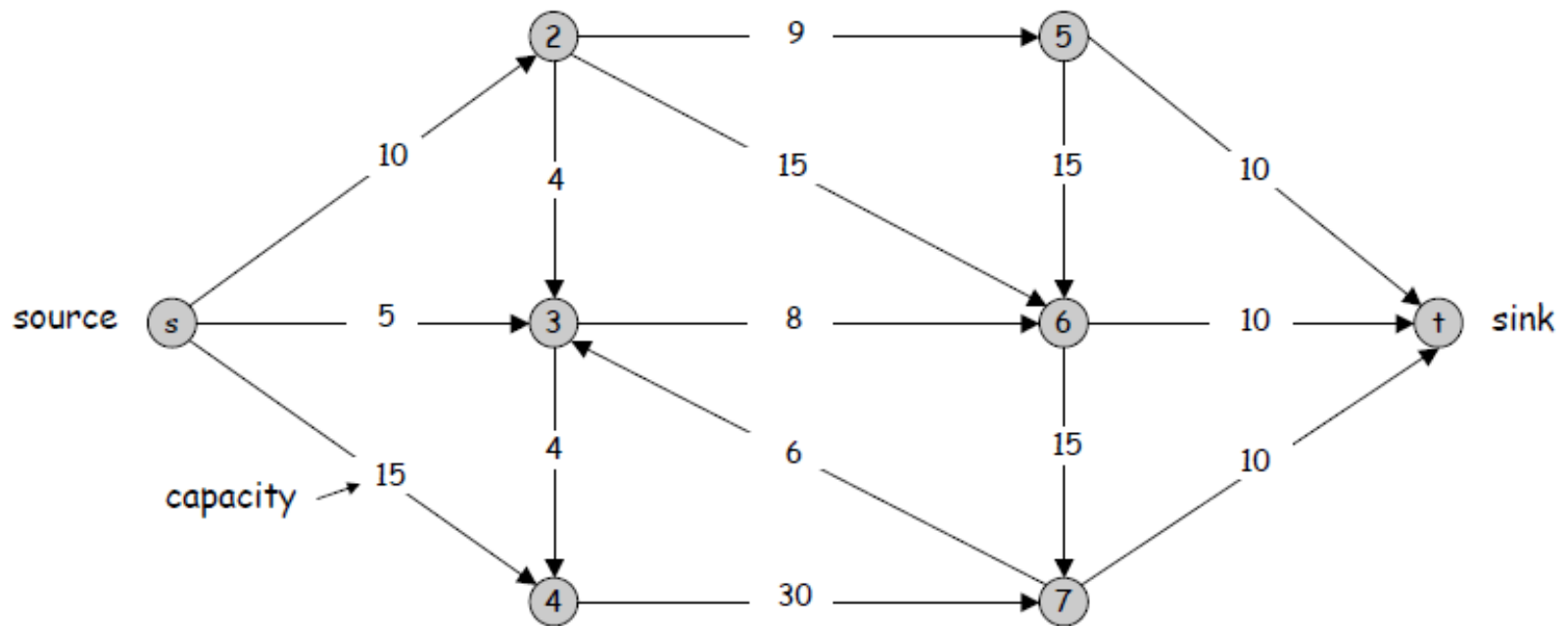
第7章 网络流算法

Soviet Rail Network, 1955



Reference: *On the history of the transportation and maximum flow problems.*
Alexander Schrijver in *Math Programming*, 91: 3, 2002.

最大流问题例子



7.1 最大流算法

7.1.1 最大流问题

定义7.1 容量网络 $N=\langle V, E, c, s, t \rangle$, 其中 $N=\langle V, E \rangle$ 是有向连通图, 记 $n=|V|$, $m=|E|$, $c:E \rightarrow R^*$ 是边的**容量**, s 和 t 是两个特殊的顶点, s 称作**发点(源)**, t 称作**收点(汇)**, 其余的顶点称作**中间点**. 设 $f:E \rightarrow R^*$, 满足下述条件:

(1) 容量限制 $\forall \langle i, j \rangle \in E, f(i, j) \leq c(i, j)$,

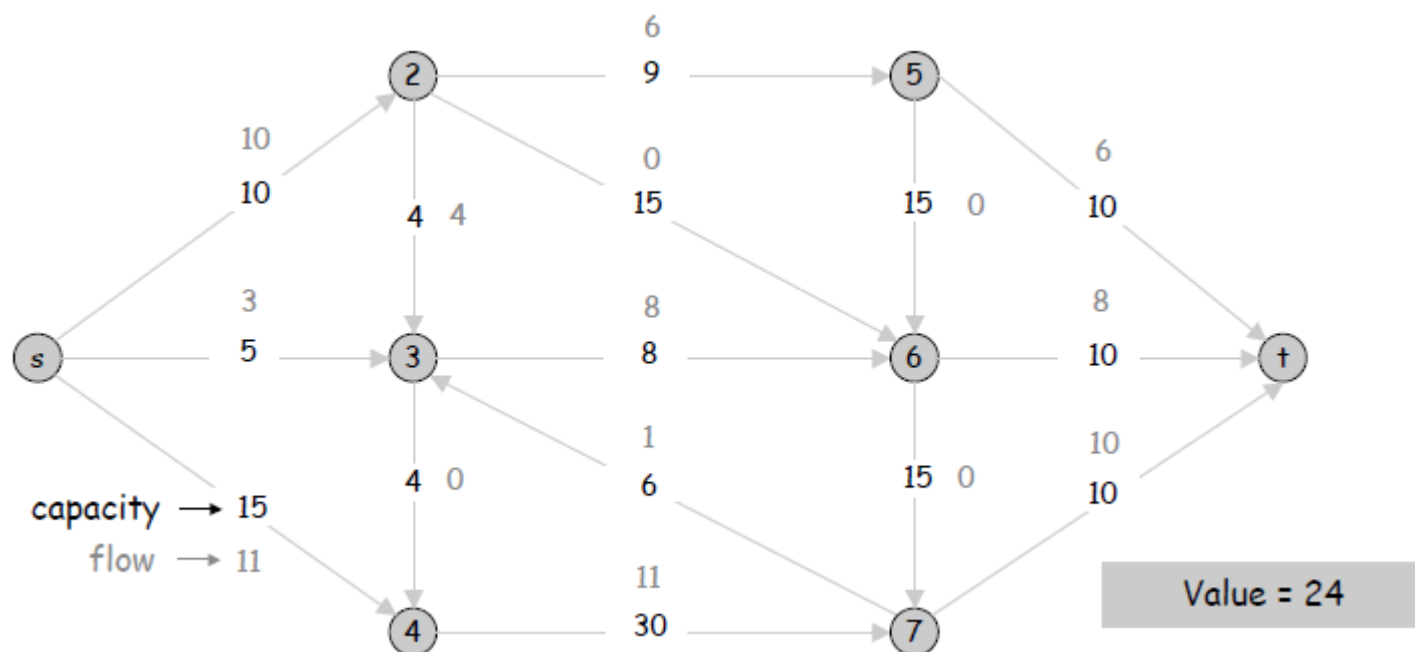
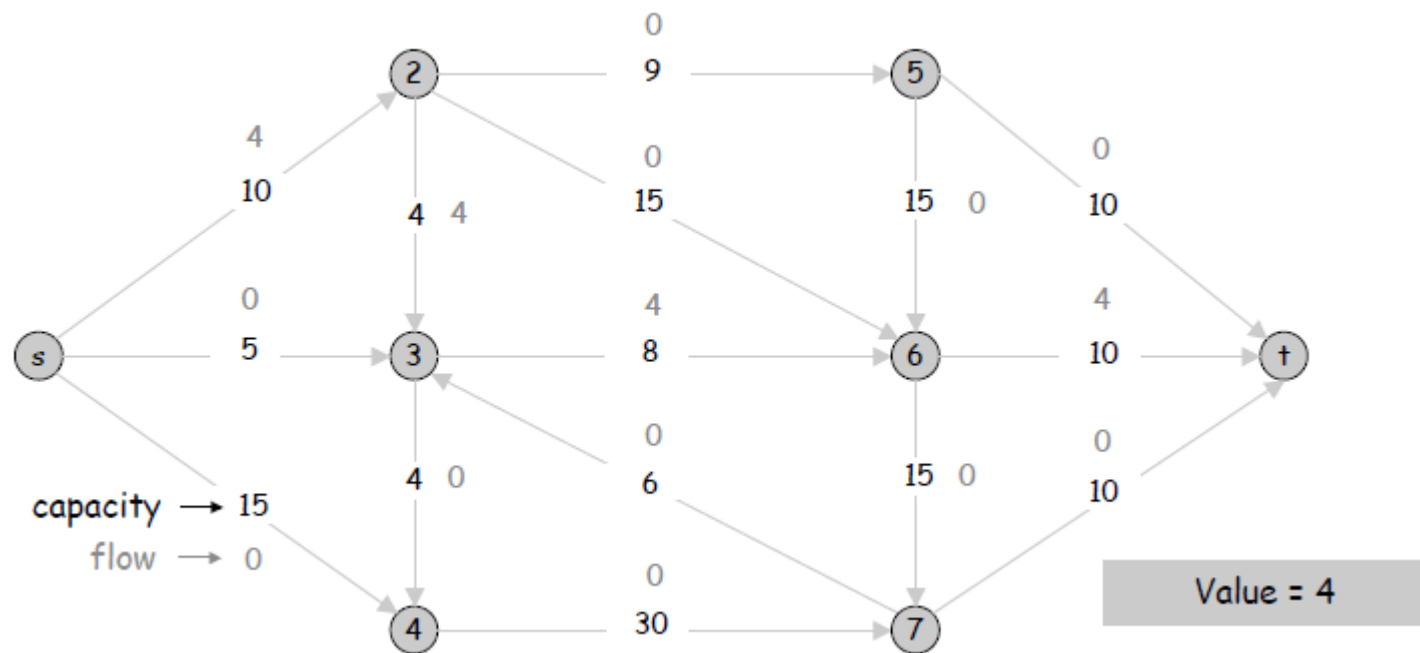
(2) 平衡条件 $\forall i \in V - \{s, t\}, \sum_{\langle j, i \rangle \in E} f(j, i) = \sum_{\langle i, j \rangle \in E} f(i, j)$

称 f 是 N 上的一个**可行流**, 称发点 s 的净流出量为 f 的**流量**, 记作 $v(f)$, 即

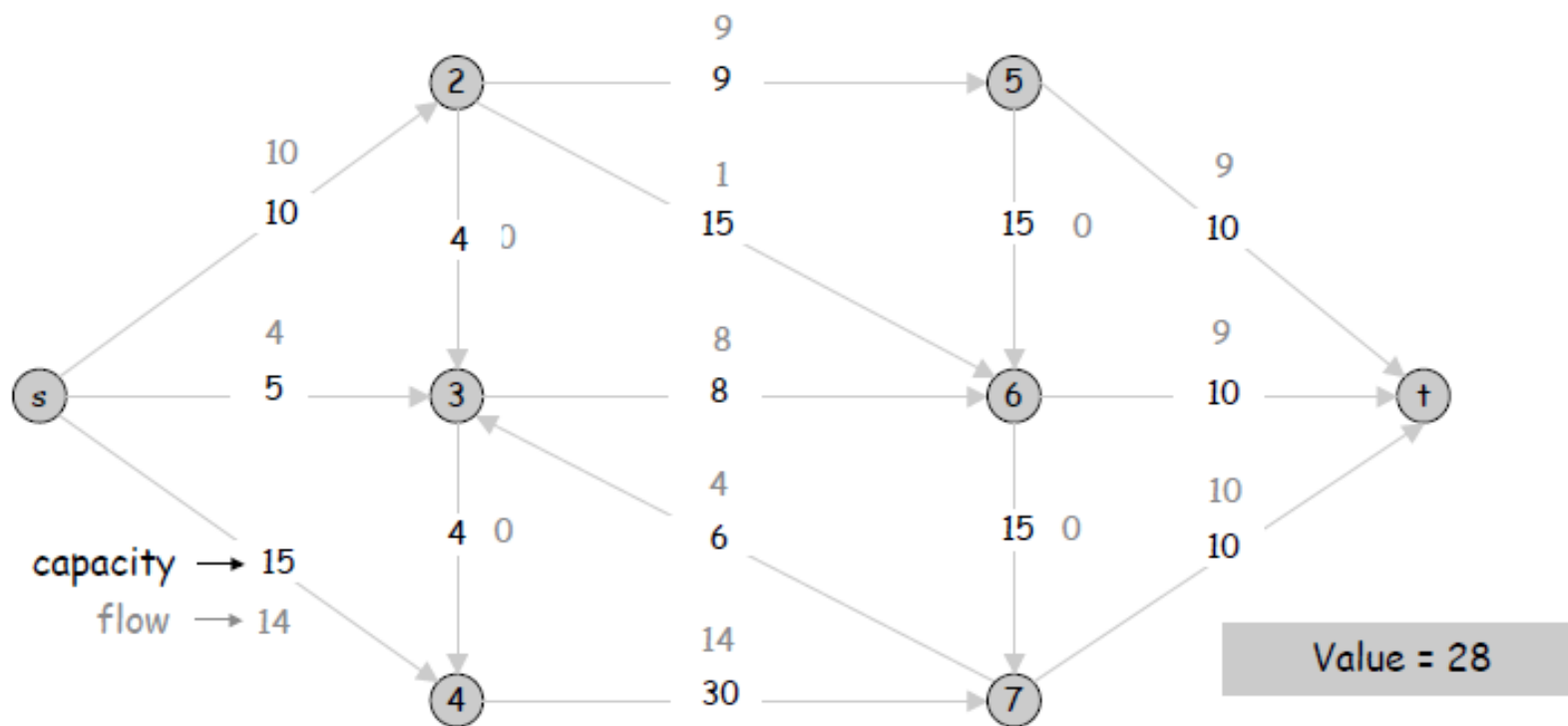
$$v(f) = \sum_{\langle s, j \rangle \in E} f(s, j) - \sum_{\langle j, s \rangle \in E} f(j, s)$$

流量最大的流称作**最大流**.

显然有 $v(f) = \sum_{\langle j, t \rangle \in E} f(j, t) - \sum_{\langle t, j \rangle \in E} f(t, j)$



最大流



最大流问题的线性规划表述

$$\max v(f)$$

s.t.

$$f(i, j) \leq c(i, j), \quad \langle i, j \rangle \in E$$

$$\sum_{\langle i, j \rangle \in E} f(i, j) - \sum_{\langle j, i \rangle \in E} f(j, i) = 0, \quad i \in V - \{s, t\}$$

$$v(f) - \sum_{\langle s, j \rangle \in E} f(s, j) + \sum_{\langle j, s \rangle \in E} f(j, s) = 0,$$

$$f(i, j) \geq 0, \quad \langle i, j \rangle \in E$$

$$v(f) \geq 0$$

能求解线性规划的算法都能求解最大流问题

最大流问题应用广泛，有更有效的求解算法

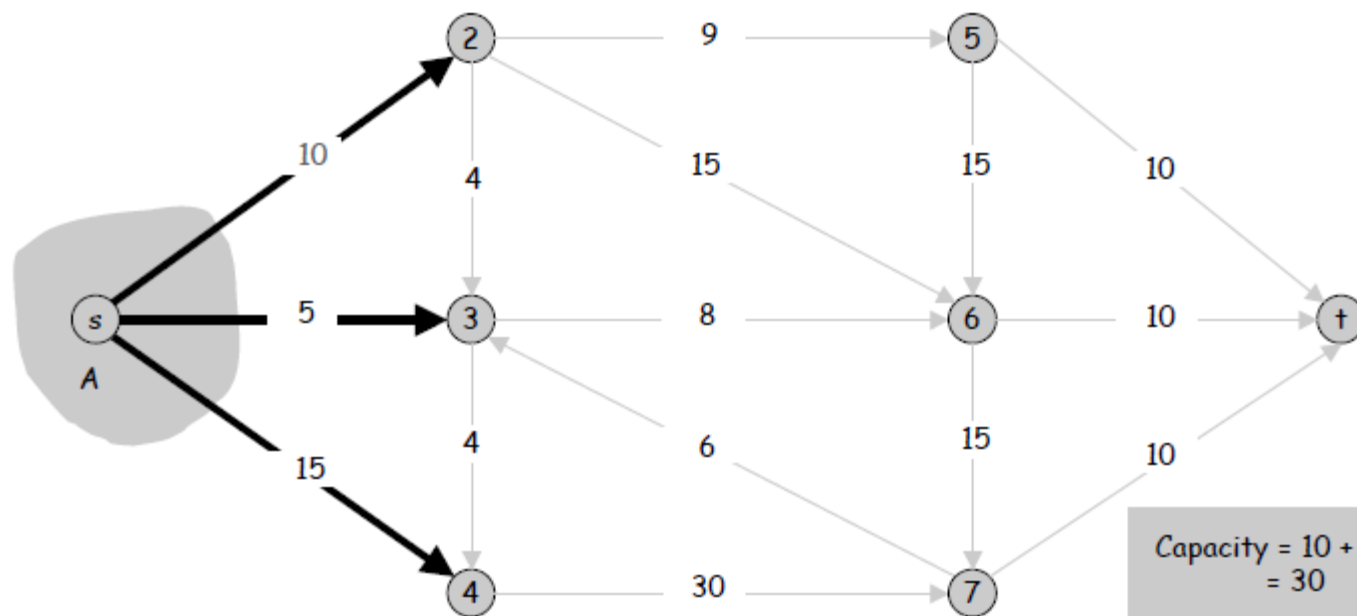
最小割集

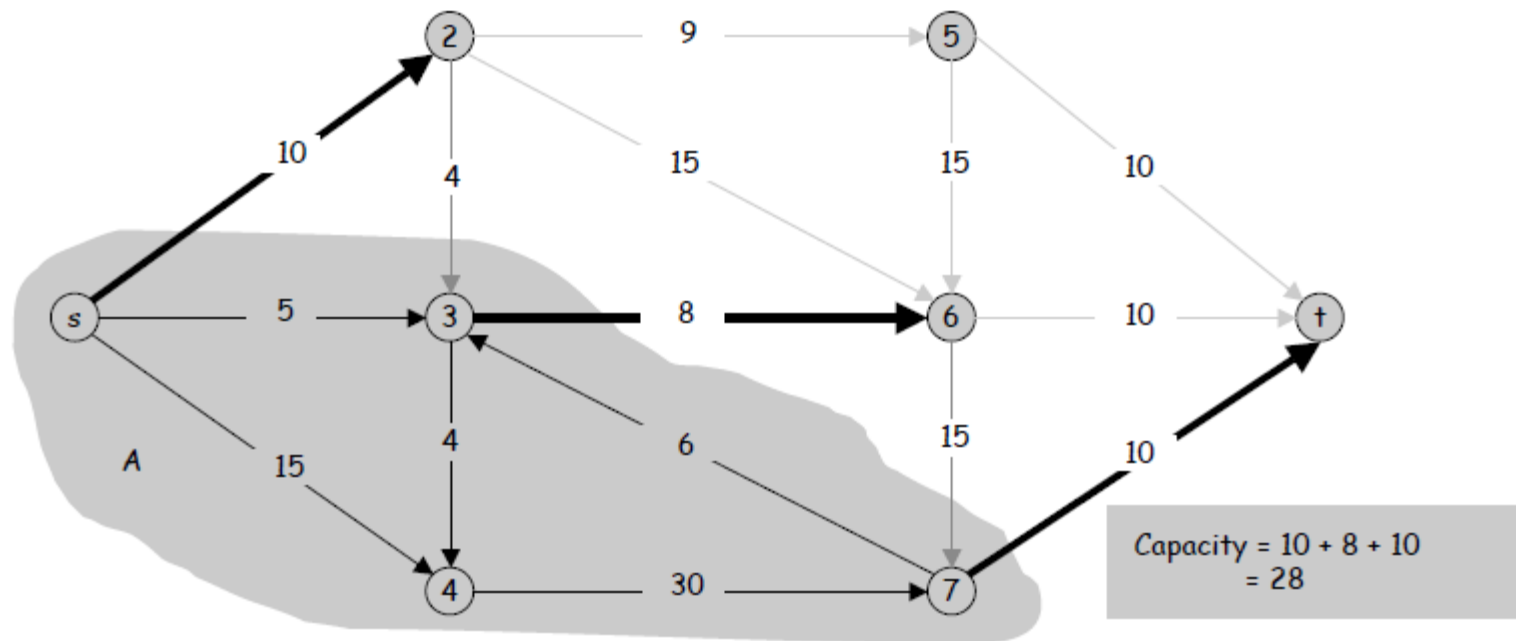
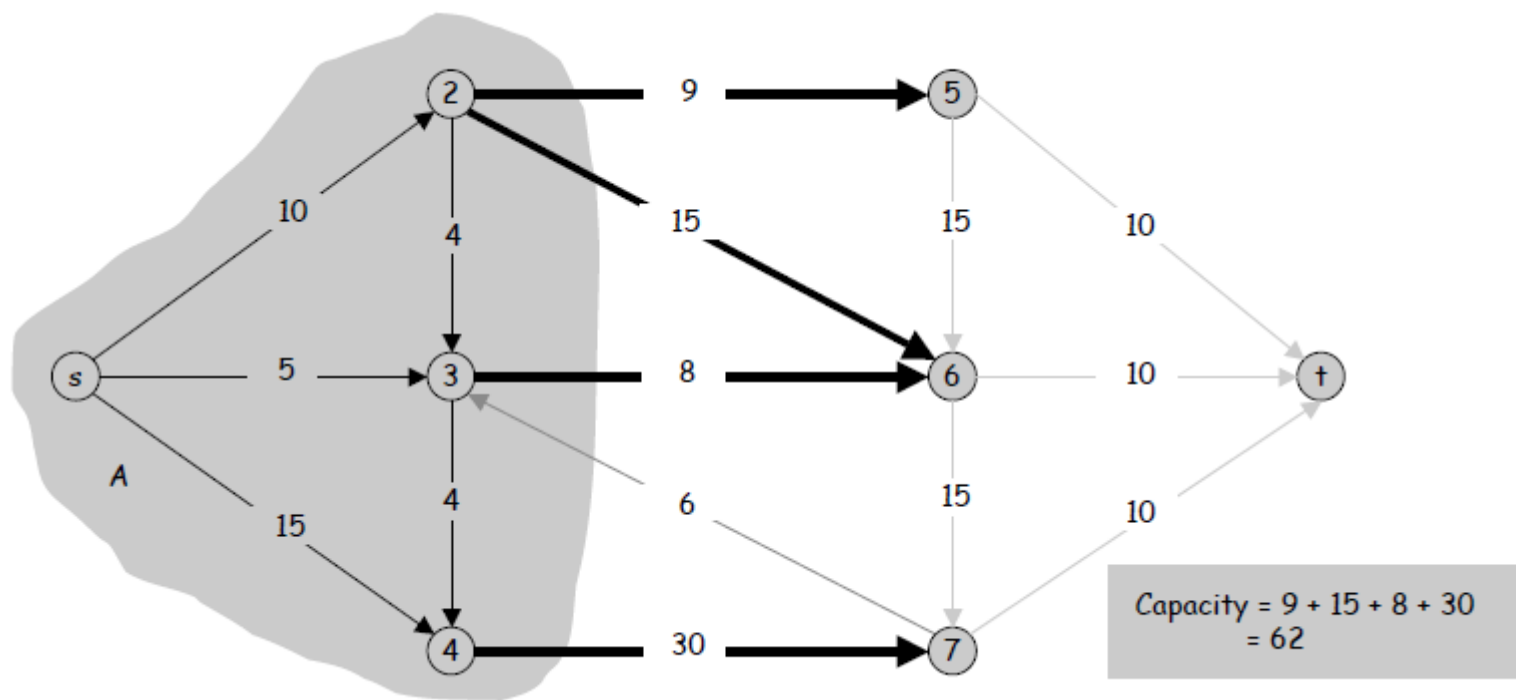
定义7.2 设容量网络 $N = \langle V, E, c, s, t \rangle$, $A \subset V$ 且 $s \in A, t \in \bar{A}$, 称

$$(A, \bar{A}) = \{ \langle i, j \rangle \mid \langle i, j \rangle \in E \text{ 且 } i \in A, j \in \bar{A} \}$$

为 N 的**割集**, $c(A, \bar{A}) = \sum_{\langle i, j \rangle \in (A, \bar{A})} c(i, j)$ 为割集 $(A, \bar{A})$ 的**容量**.

容量最小的割集称作**最小割集**.





引理7.1 设容量网络 $N=\langle V, E, c, s, t \rangle$, f 是 N 上的一个可行流, $A \subset V$ 且 $s \in A, t \in \bar{A}$, 则

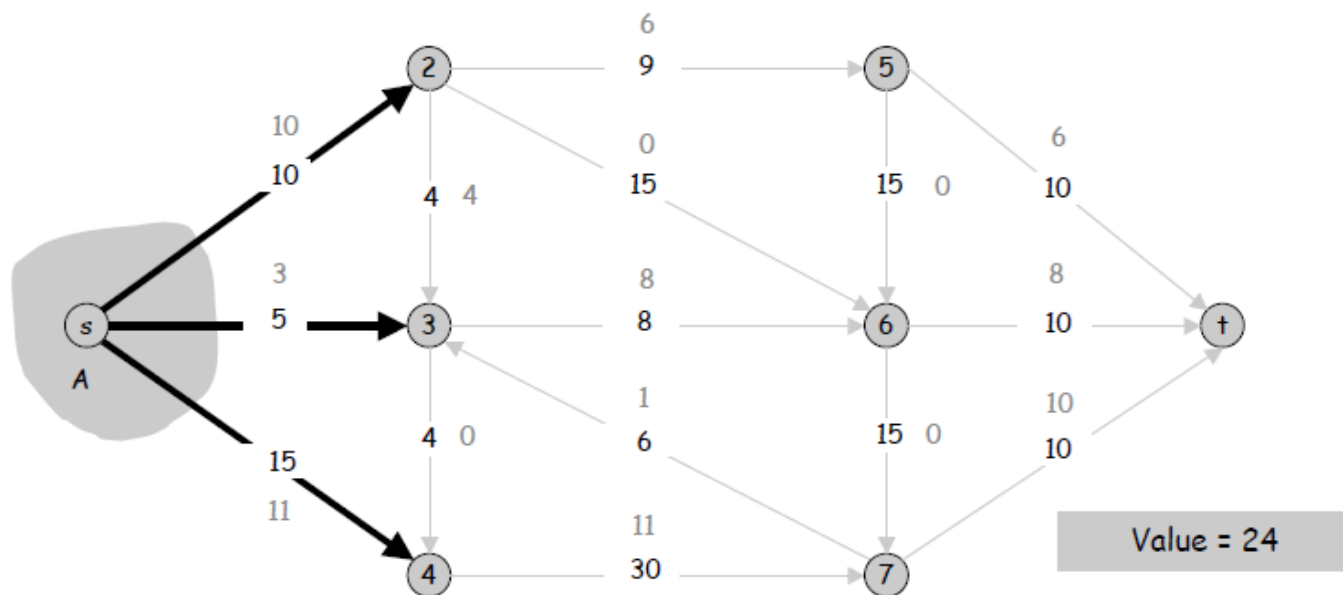
$$v(f) = \sum_{\langle i, j \rangle \in (A, \bar{A})} f(i, j) - \sum_{\langle j, i \rangle \in (\bar{A}, A)} f(j, i)$$

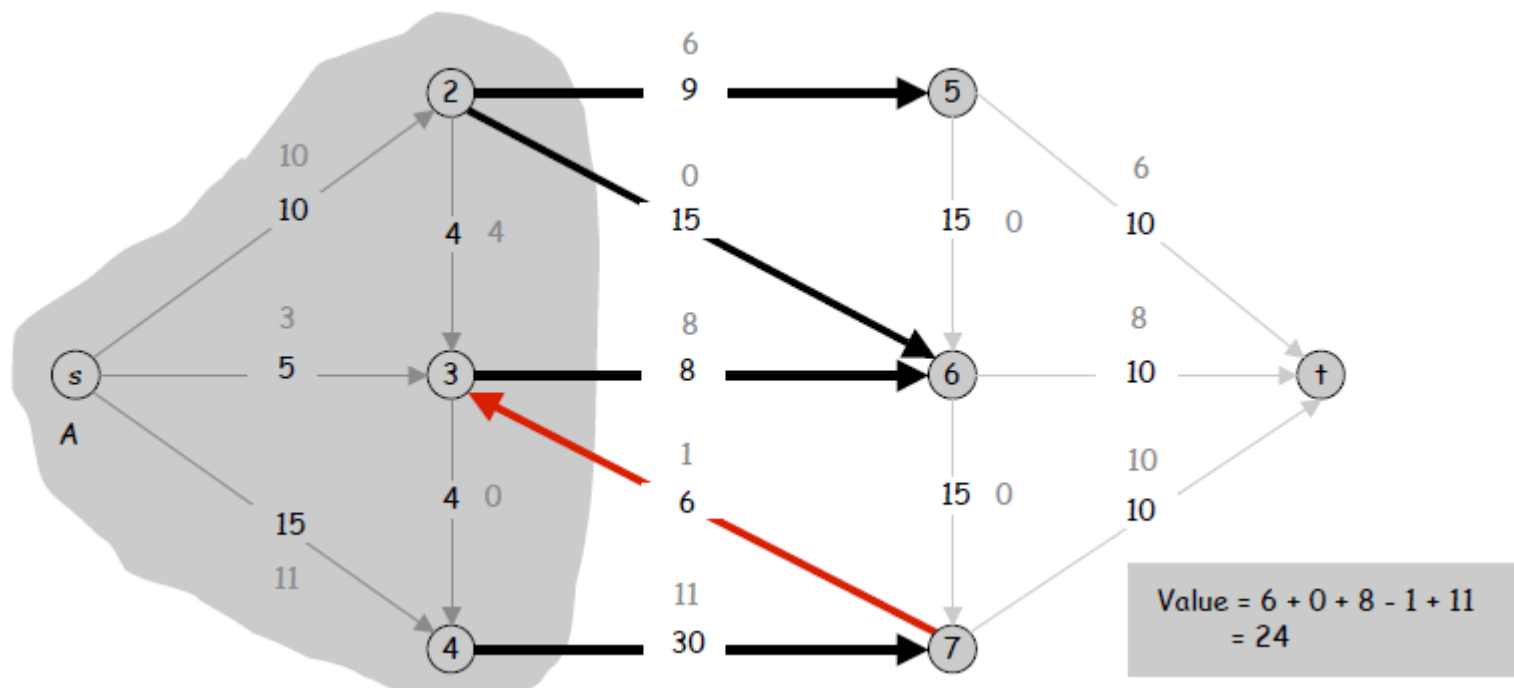
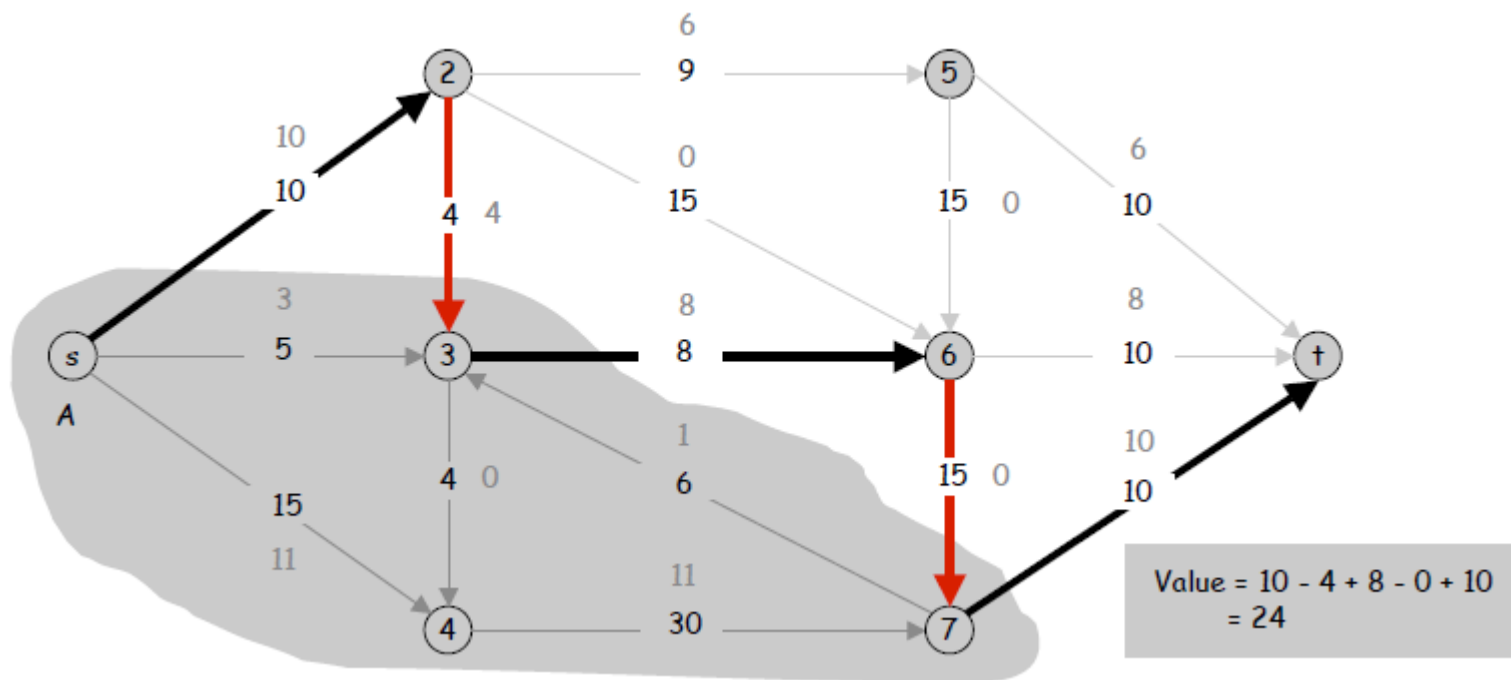
证

$$\begin{aligned} v(f) &= \sum_{\langle s, j \rangle \in E} f(s, j) - \sum_{\langle j, s \rangle \in E} f(j, s) \\ &= \sum_{\langle s, j \rangle \in E} f(s, j) - \sum_{\langle j, s \rangle \in E} f(j, s) \\ &\quad + \sum_{i \in A - \{s\}} \left\{ \sum_{\langle i, j \rangle \in E} f(i, j) - \sum_{\langle j, i \rangle \in E} f(j, i) \right\} \\ &= \sum_{i \in A} \left\{ \sum_{\langle i, j \rangle \in E} f(i, j) - \sum_{\langle j, i \rangle \in E} f(j, i) \right\} \\ &= \sum_{i \in A} \sum_{\langle i, j \rangle \in E} f(i, j) - \sum_{i \in A} \sum_{\langle j, i \rangle \in E} f(j, i) \end{aligned}$$

引理7.1证明（续）

$$\begin{aligned}
 &= \sum_{\substack{i \in A \\ j \in A}} \sum_{\langle i, j \rangle \in E} f(i, j) + \sum_{\substack{i \in A \\ j \in \bar{A}}} \sum_{\langle i, j \rangle \in E} f(i, j) \\
 &\quad - \sum_{\substack{i \in A \\ j \in A}} \sum_{\langle j, i \rangle \in E} f(j, i) - \sum_{\substack{i \in A \\ j \in \bar{A}}} \sum_{\langle j, i \rangle \in E} f(j, i) \\
 &= \sum_{\langle i, j \rangle \in (A, \bar{A})} f(i, j) - \sum_{\langle j, i \rangle \in (\bar{A}, A)} f(j, i)
 \end{aligned}$$



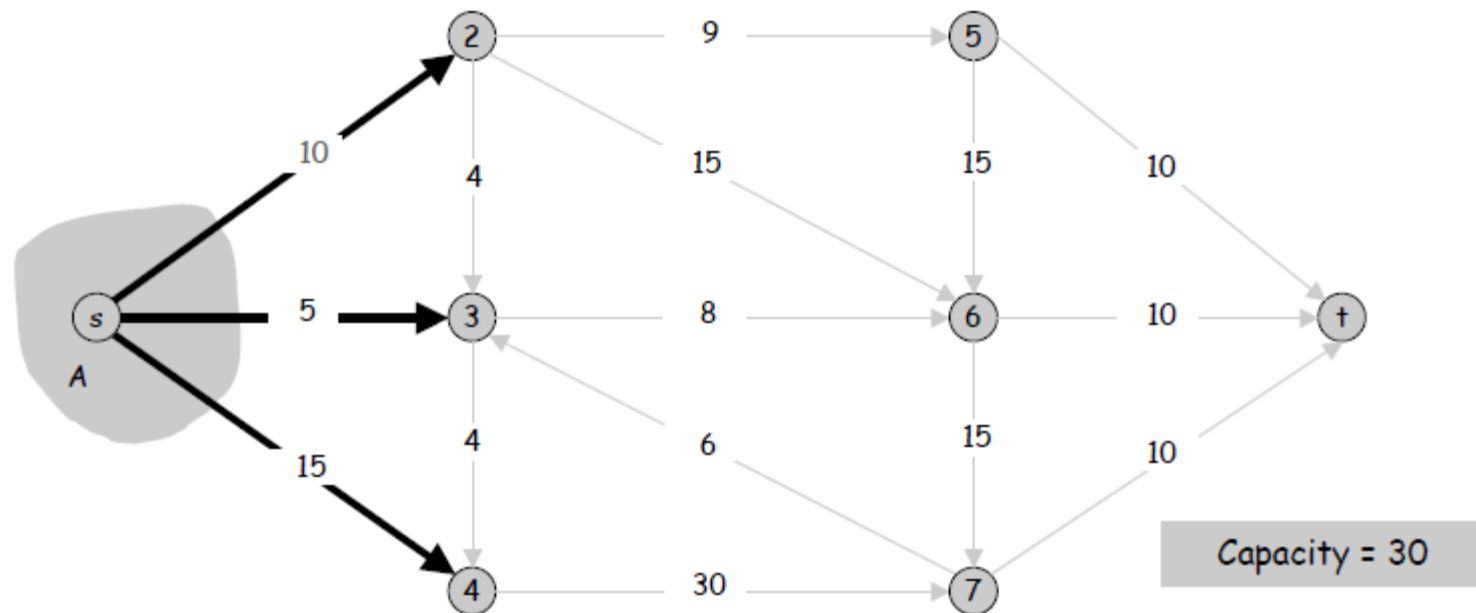


引理7.2 设 f 是任一可行流, $(A, \bar{A})$ 是任一割集, 则

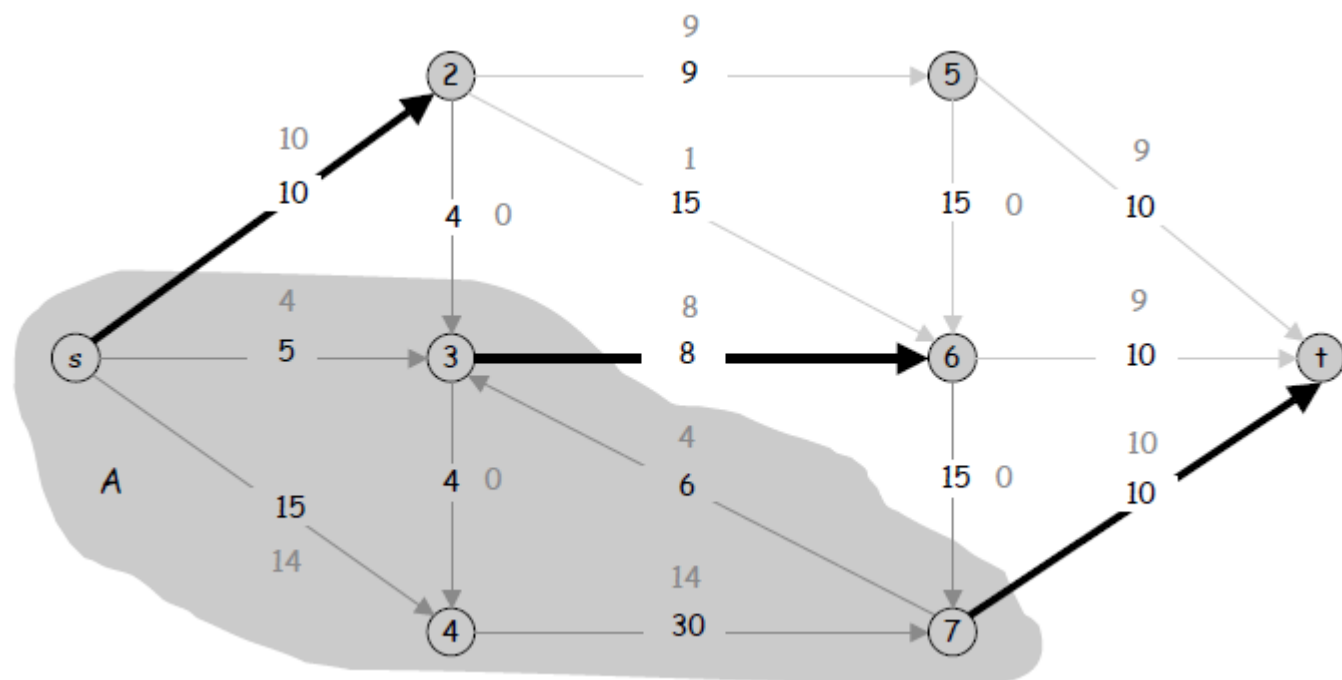
$$v(f) \leq c(A, \bar{A})$$

Pf.

$$\begin{aligned}
 v(f) &= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) \\
 &\leq \sum_{e \text{ out of } A} f(e) \\
 &\leq \sum_{e \text{ out of } A} c(e) \\
 &= \text{cap}(A, \bar{A}) \quad \blacksquare
 \end{aligned}$$



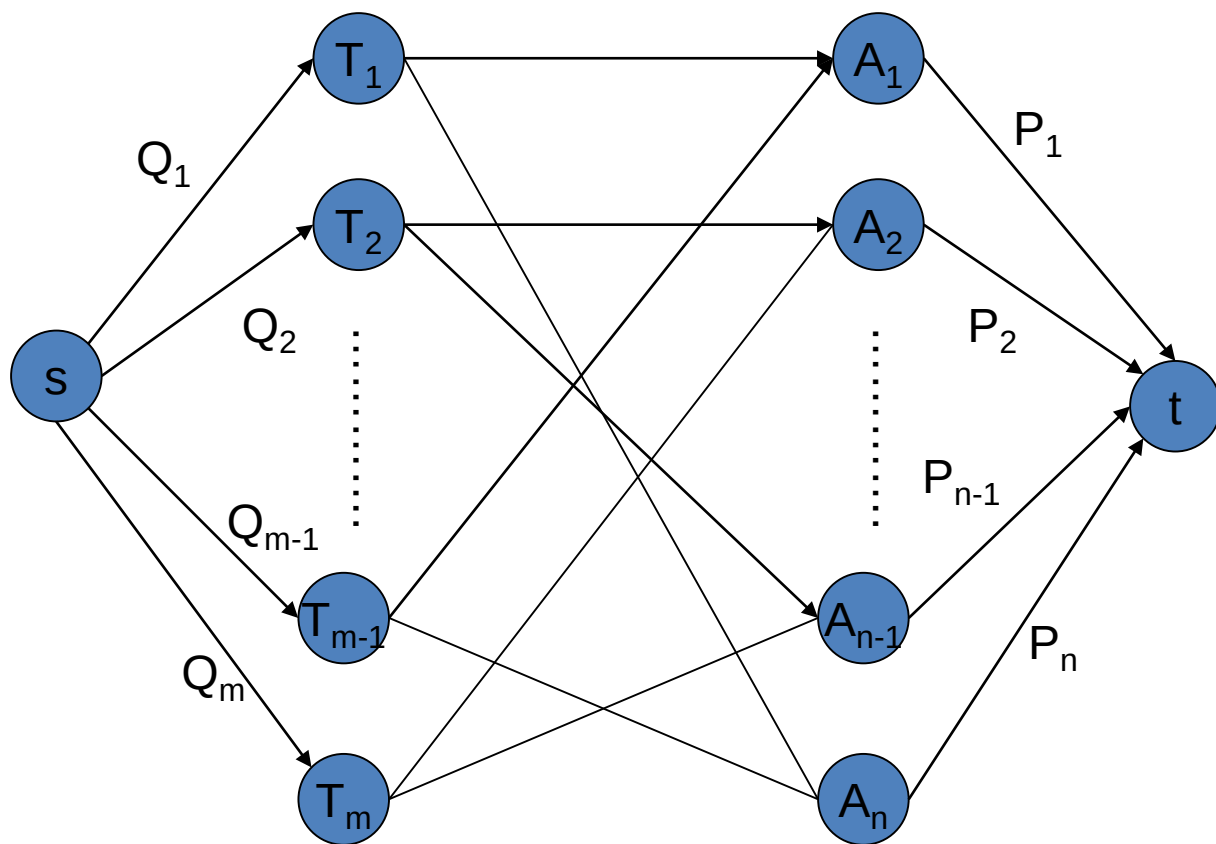
引理7.3 设 f 是任一可行流, $(A, \bar{A})$ 是任一割集, 如果 $v(f) = c(A, \bar{A})$, 则 f 是最大流, $(A, \bar{A})$ 是最小割集.



最小割集的应用

- 可用于决定经营投资策略。如开发产品 A_1 需要先购入工具 T_1, T_2 ，而产品 A_2 需要先购入工具 T_2, T_3 ，则同时开发仅需要负担 T_1, T_2, T_3 的成本。
- 可以将此问题一般化，假定产品 A_i 需要先购入 k 个工具 $T_{i1} T_{i2} \dots T_{ik}$ 。而产品 A_i 开发完成可获利 P_i ，购入工具 T_j 需要 Q_j 的金钱，则该选择哪些产品开发？

利用最小割集



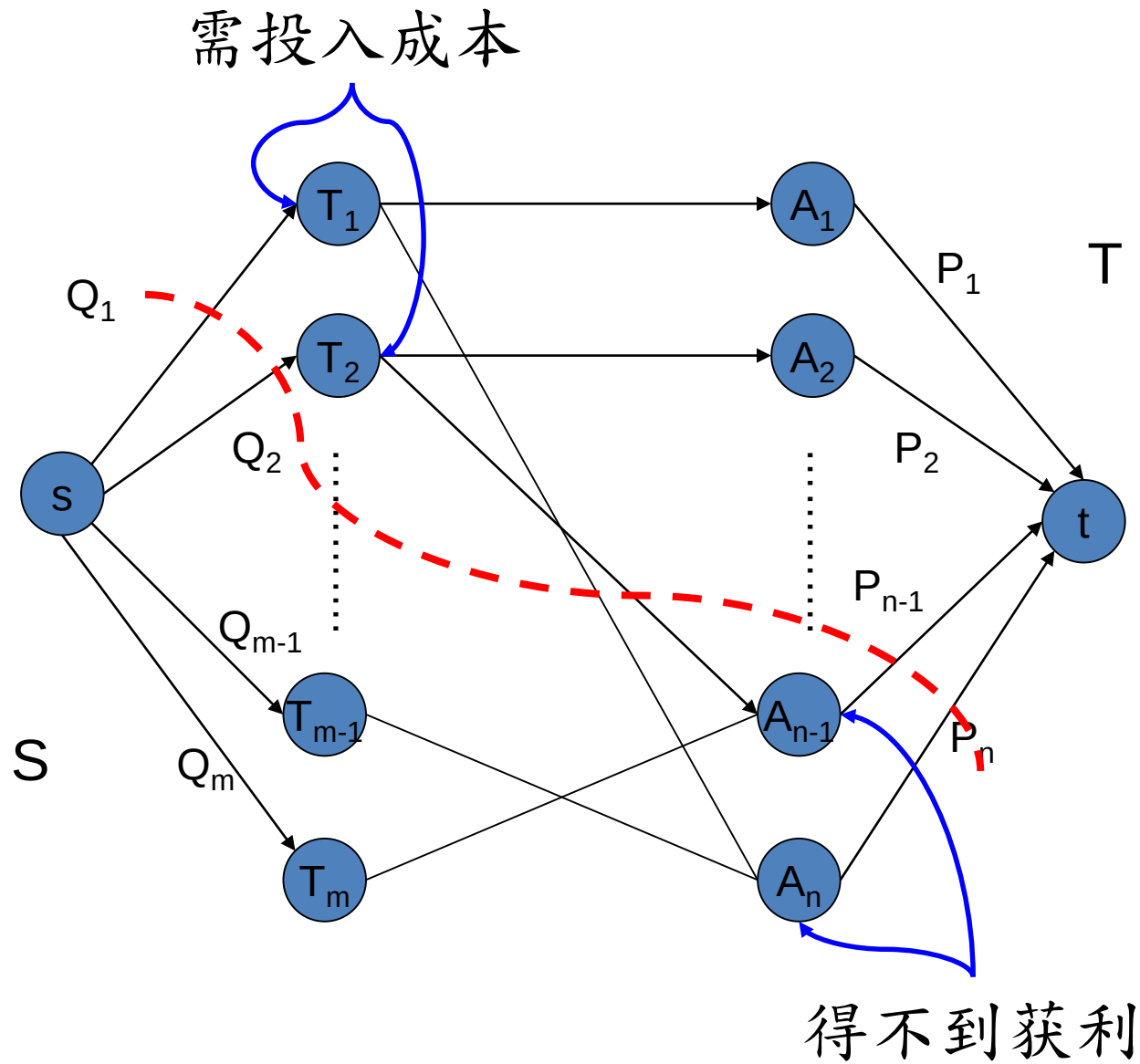
如 A_i 需要 T_j ，则自 T_1 拉一条容量无限大的边到 A_i 。

流量网络的建构方式

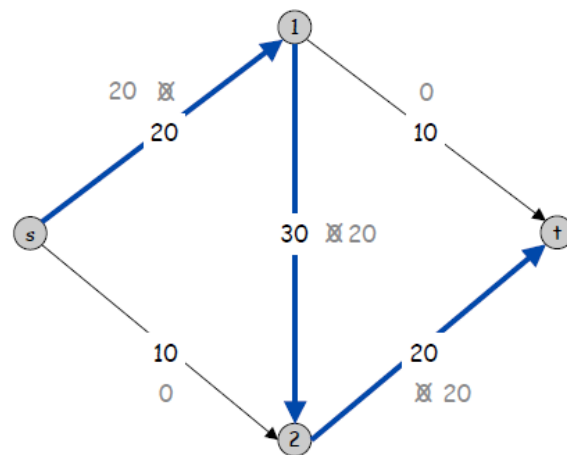
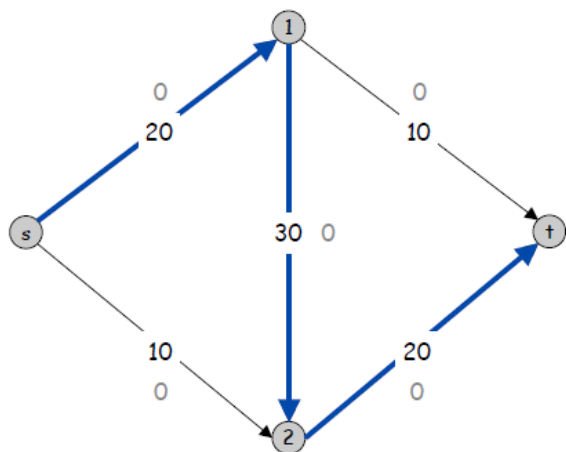
- 将图如上页一般的建构出来，有source s , sink t ，以及每一个产品与工具。
- 对每个工具 T_j 自 s 拉一条容量为 Q_j 的边。
- 自每一个产品 A_i 拉一条容量为 P_i 的边到 t 。
- 如 A_i 需要 T_j ，则自 T_j 拉一条容量无限大的边到 A_i 。

与最大获利的对应

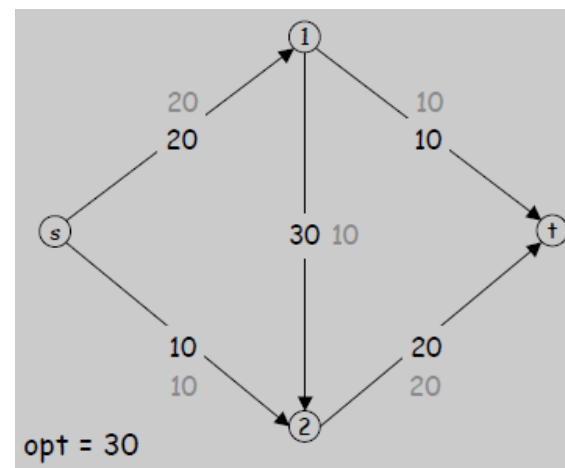
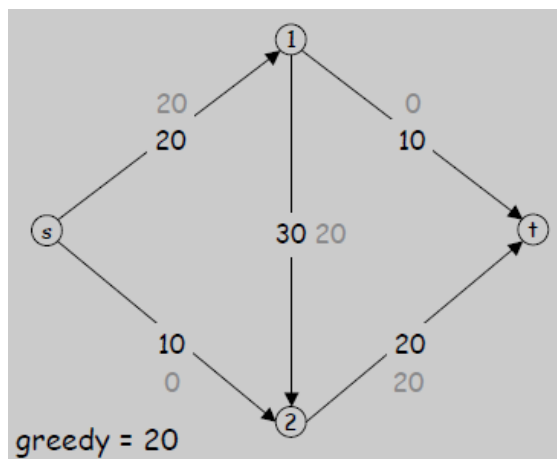
- 所有产品的利润总和扣掉该图的**minimum cut**即是最大获利。
- 观察：能够获利的产品，获利必然比投入的工具成本高，故将此类的产品与工具划入**T**，其他的划入**S**。
- 理想的状态是所有的产品利润全得，没有投入的产品部份系扣除产品接到**sink**的容量，而有投入的产品需扣除投入的工具成本，故扣除**source**接到工具的容量。所扣除部分即为**cut**的容量。



贪心法



陷进局部最优



增广链的相关概念

容量网络 $N = \langle V, E, c, s, t \rangle$, f 是 N 上任一可行流

- **饱和边**: 流量等于容量的边
- **非饱和边**: 流量小于容量的边
- **零流边**: 流量等于0的边
- **非零流边**: 流量大于0的边

增广链的相关概念（续）

- i - j 链： N 中从顶点 i 到 j 的一条边不重复的路径
（不考虑边的方向情况下的路径）
 - i - j 链的方向是从 i 到 j
 - 前向边：链中与链的方向一致的边
 - 后向边：链中与链的方向相反的边
- i - j 增广链：
 - 一条 i - j 链
 - 前向边都是非饱和边
 - 后向边都是非零流边

通过 s - t 增广链增加流量

- 设 P 是一条关于可行流 f 的 s - t 增广链

$$\delta = \min \left\{ \begin{array}{l} \min\{P \text{ 上前向边容量与流量之差}\} \\ \min\{P \text{ 上后向边流量}\} \end{array} \right\} > 0$$

- 基于可行流 f 构造新的可行流 f'

$$f'(i, j) = \begin{cases} f(i, j) + \delta & \langle i, j \rangle \text{ 是 } P \text{ 的前向边} \\ f(i, j) - \delta & \langle i, j \rangle \text{ 是 } P \text{ 的后向边} \\ f(i, j) & \text{否则} \end{cases}$$

- f' 满足容量限制, 平衡条件, 非负, 且

$$v(f') = v(f) + \delta$$

最大流最小割集定理证明

- 最大流上一定不存在 $s-t$ 增广链
- 是否不存在 $s-t$ 增广链的可行流就是最大流?

证明:

- 假设不存在关于可行流 f 的 $s-t$ 增广链, 设
 $A = \{j \in V \mid \text{存在关于可行流 } f \text{ 的 } s-j \text{ 增广链}\}$
 $s \in A; \quad t \notin A \Rightarrow t \in \bar{A}$
- $(A, \bar{A})$ 是割集, 且
 $f(i, j) = c(i, j), \langle i, j \rangle \in (A, \bar{A})$
 $f(j, i) = 0, \langle j, i \rangle \in (\bar{A}, A)$

最大流最小割集定理证明（续）

- 证明 $f(i, j) = c(i, j), \langle i, j \rangle \in (A, \bar{A})$

否则存在 $\langle i, j \rangle \in (A, \bar{A})$ 使得 $f(i, j) < c(i, j)$

则 $s-i$ 增广链延 $\langle i, j \rangle$ 延伸得到 $s-j$ 链也是增广链

则根据 A 的定义有 $j \in A$, 与 $j \in \bar{A}$ 矛盾

- 证明 $f(j, i) = 0, \langle j, i \rangle \in (\bar{A}, A)$

否则存在 $\langle j, i \rangle \in (\bar{A}, A)$ 使得 $f(j, i) > 0$

则 $s-i$ 增广链延 $\langle j, i \rangle$ 延伸得到 $s-j$ 链也是增广链

则根据 A 的定义有 $j \in A$, 与 $j \in \bar{A}$ 矛盾

最大流最小割集定理证明（续）

引理7.1

$$\begin{aligned} v(f) &= \sum_{\langle i,j \rangle \in (A, \bar{A})} f(i,j) - \sum_{\langle j,i \rangle \in (\bar{A}, A)} f(j,i) \\ &= \sum_{\langle i,j \rangle \in (A, \bar{A})} f(i,j) - 0 = \sum_{\langle i,j \rangle \in (A, \bar{A})} c(i,j) = c(A, \bar{A}) \end{aligned}$$

引理7.3

$$v(f) = c(A, \bar{A}) \Rightarrow$$

f 是最大流, $(A, \bar{A})$ 是最小割集

FF算法

FF算法基本步骤:

从给定的初始可行流 (通常取零流) 开始, 寻找关于当前可行流的 s - t 增广链 P , 修改链上的流量, 得到一个新的可行流. 重复进行, 直到不存在 s - t 增广链为止.

用标号法寻找 s - t 增广链. 从 s 开始, 逐个给顶点作标号, 直到 t 得到标号为止. 顶点 j 得到标号表示已找到从 s 到 j 的增广链, 标号为 (l_j, δ_j) . 其中 δ_j 等于这条到 j 的链上所有前向边的容量与流量之差以及所有后向边的流量的最小值.

$l_j = +i$ 或 $l_j = -i$. $l_j = +i$ 表示链是从 i 到 j 且 $\langle i, j \rangle$ 是前向边, $l_j = -i$ 表示链是从 i 到 j 的且 $\langle j, i \rangle$ 是后向边.

FF算法设计思想

顶点分3类: 已标号已检查的, 已标号未检查的, 未标号的.

开始时, 给 s 标号 $(\Delta, +\infty)$, s 是已标号未检查的, 其余顶点都是未标号的.

每一步取一个已标号未检查的顶点 i , 对所有的顶点 j , 若 $\langle i, j \rangle \in E$ 且 $f(i, j) < c(i, j)$, 则给 j 标号 $(+i, \delta_j)$, 其中

$$\delta_j = \min\{ \delta_i, c(i, j) - f(i, j) \};$$

若 $\langle j, i \rangle \in E$ 且 $f(j, i) > 0$, 则给 j 标号 $(-i, \delta_j)$, 其中

$$\delta_j = \min\{ \delta_i, f(j, i) \}.$$

直到不存在已标号未检查的顶点为止.

FF算法

算法7.1 Ford-Fulkerson算法

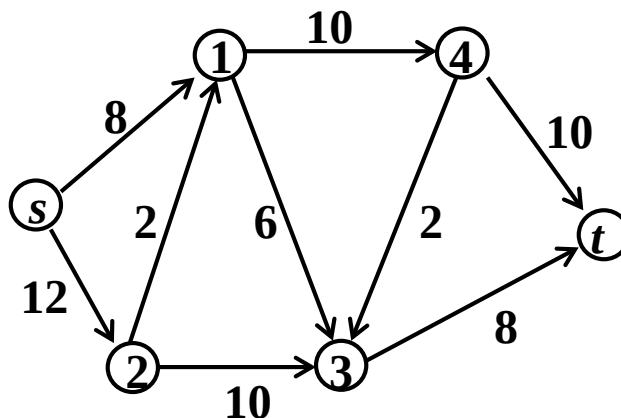
1. $f \leftarrow 0$ //取零流作为初始可行流
2. $T \leftarrow \{s\}, R \leftarrow V - \{s\}, l_s \leftarrow \Delta, \delta_s \leftarrow +\infty$
// T 是已标号未检查的顶点集, R 是未标号的顶点集
3. while $T \neq \emptyset$ do
4. 取 $i \in T, T \leftarrow T - \{i\}$ //检查顶点 i
5. for 所有 R 中与 i 邻接的 j do
6. if $\langle i, j \rangle \in E$ 且 $f(i, j) < c(i, j)$ then
7. $l_j \leftarrow +i, \delta_j \leftarrow \min\{\delta_i, c(i, j) - f(i, j)\}, R \leftarrow R - \{j\}, T \leftarrow T \cup \{j\}$
8. if $j = t$ then goto 13 //已找到一条 s - t 增广链
9. if $\langle j, i \rangle \in E$ 且 $f(j, i) > 0$ then

FF算法（续）

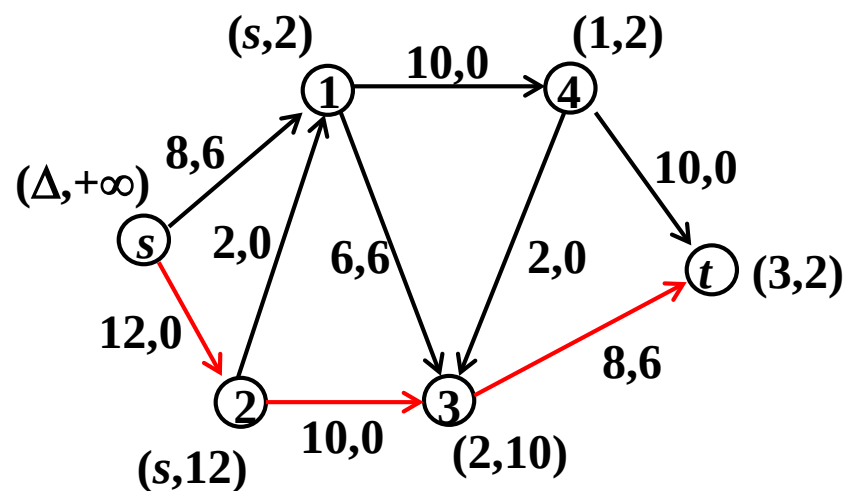
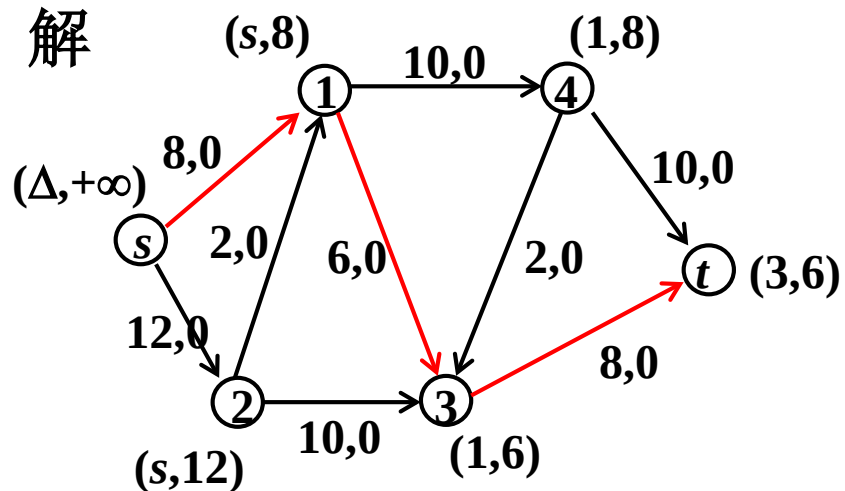
10. $l_j \leftarrow -i, \delta_j \leftarrow \min\{\delta_i, f(j, i)\}, R \leftarrow R - \{j\}, T \leftarrow T \cup \{j\}$
11. if $j = t$ then goto 13 //已找到一条 s - t 增广链
12. return f //不存在 s - t 增广链, 计算结束
13. $\delta \leftarrow \delta_t$ //沿增广链回溯, 修改流量
14. if $l_j = \Delta$ then goto 2 //重新开始下一阶段标号
15. if $l_j = +i$ then
16. $f(i, j) \leftarrow f(i, j) + \delta, j \leftarrow i$
17. if $l_j = -i$ then
18. $f(j, i) \leftarrow f(j, i) - \delta, j \leftarrow i$
19. goto 14

例

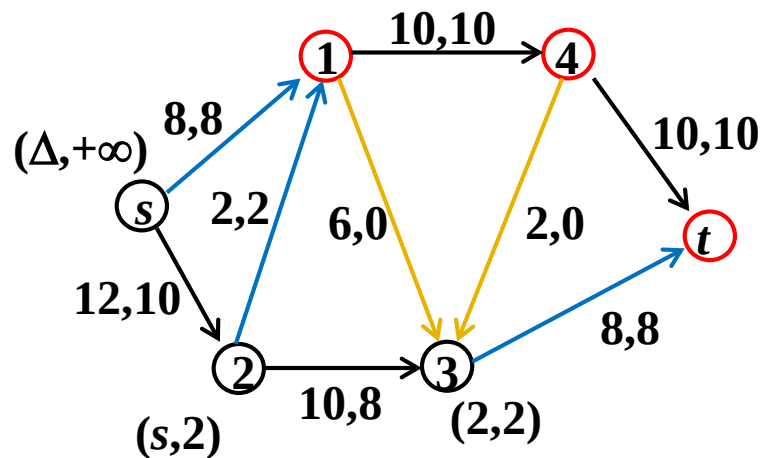
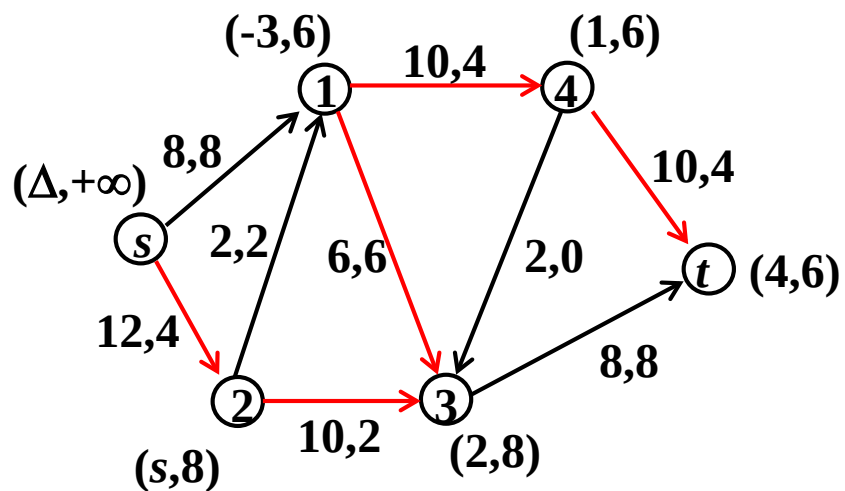
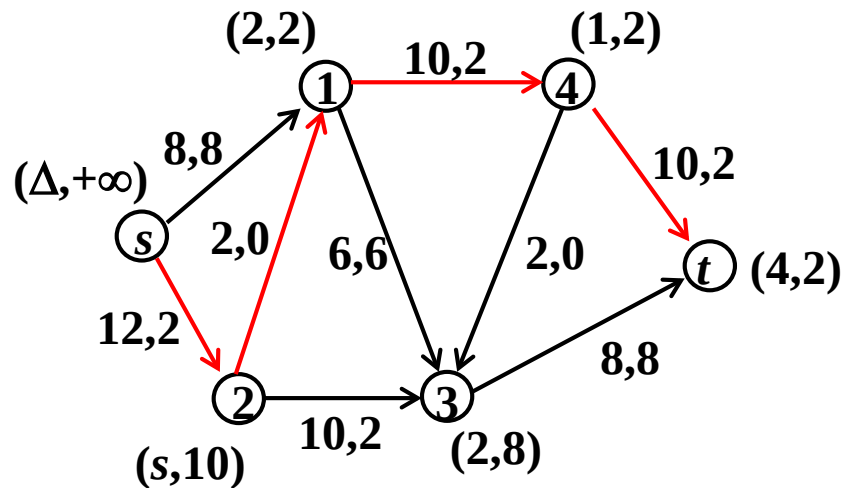
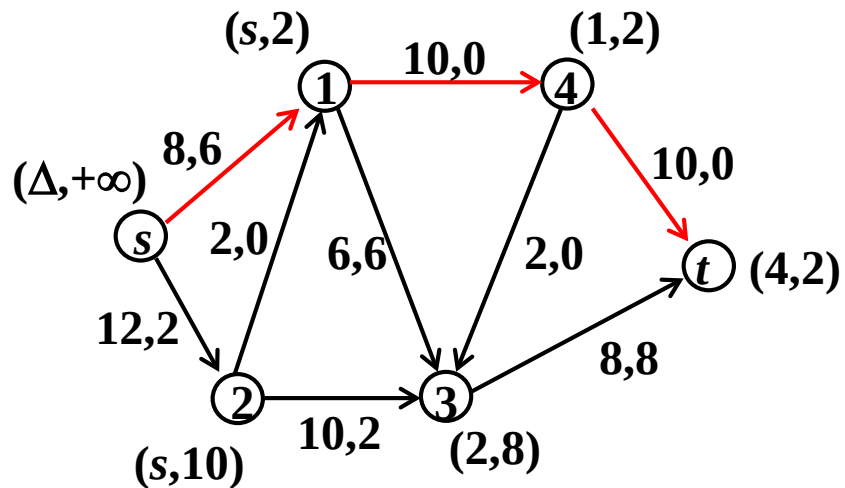
例7.1 应用FF算法



解



例7.1 (续)



FF算法的运行时间和有限终止性

假设所有容量都是正整数, 则算法的时间复杂度为 $O(mC)$.

令
$$C = \sum_{\langle s, j \rangle \in E} c(s, j)$$

最大流量 $v^* \leq C$. 计算中每次的修改量 δ 也是正整数, 每个阶段流量至少增加 1, 至多有 C 个阶段. 而每个阶段标号和修改增广链上的流量可在 $O(m)$ 步内完成.

用计算机计算, 数的表示都有一定的精度. 以其精度为单位, 每个数都可看作整数. 因此, 上面的分析仍然有效, 算法总会在有限步内终止.

但在理论上, 当容量为无理数时, δ 可能越来越小, 趋向于 0. 实际上已给出这样的例子, 算法不能在有限步终止.

FF最大流算法的改进途径

- FF算法没有规定如何求 $s-t$ 增广链
- 改进的策略
 - 保证所求增广链具有一定性质（如最短）
 - 并且能一次能求得多个增广链
- 辅助网络
 - 帮助我们实现上述改进

7.1.3 Dinic有效算法

定义7.4 关于 f 的**辅助网络** $N(f) = \langle V, E(f), ac, s, t \rangle$, 其中

$$E^+(f) = \{ \langle i, j \rangle \mid \langle i, j \rangle \in E \text{ 且 } f(i, j) < c(i, j) \}$$

$$E^-(f) = \{ \langle j, i \rangle \mid \langle i, j \rangle \in E \text{ 且 } f(i, j) > 0 \}$$

$$E(f) = E^+(f) \cup E^-(f)$$

$$ac(i, j) = \begin{cases} c(i, j) - f(i, j), & \langle i, j \rangle \in E^+(f) \\ f(j, i), & \langle i, j \rangle \in E^-(f) \end{cases}$$

ac 称作**辅助容量**. $N(f)$ 也是容量网络.

引理

引理7.4 设 N 的最大流量为 v^* , f 是可行流, 则 $N(f)$ 的最大流量为 $v^* - v(f)$.

证 N 中割集 $(A, \bar{A})$ 也是 $N(f)$ 中割集, 把它记作 $(A, \bar{A})'$, $(A, \bar{A})'$ 由 $(A, \bar{A})$ 中关于 f 的非饱和边 E_1 和 $(\bar{A}, A)$ 中关于 f 的非零流的反向边 E_2 两部分组成. $(A, \bar{A})'$ 的容量

$$\begin{aligned} ac(A, \bar{A})' &= \sum_{\langle i, j \rangle \in E_1} \{c(i, j) - f(i, j)\} + \sum_{\langle i, j \rangle \in E_2} f(j, i) \\ &= \sum_{\langle i, j \rangle \in (A, \bar{A})} \{c(i, j) - f(i, j)\} + \sum_{\langle j, i \rangle \in (\bar{A}, A)} f(j, i) \\ &= \sum_{\langle i, j \rangle \in (A, \bar{A})} c(i, j) - \left\{ \sum_{\langle i, j \rangle \in (A, \bar{A})} f(i, j) - \sum_{\langle j, i \rangle \in (\bar{A}, A)} f(j, i) \right\} \\ &= c(A, \bar{A}) - v(f) \end{aligned}$$

由最大流最小割定理得证 $N(f)$ 的最大流量等于 $v^* - v(f)$.

引理

定义7.5 设 f 是 N 上的一个可行流, g 是 $N(f)$ 上的一个可行流, 定义 $f' = f + g$ 如下:

$$\forall \langle i, j \rangle \in E, \quad f'(i, j) = f(i, j) + g(i, j) - g(j, i)$$

这里规定, 当 $\langle i, j \rangle \notin E(f)$ 时, $g(i, j) = 0$.

引理7.5 设 f 是 N 上的一个可行流, g 是 $N(f)$ 上的一个可行流, 则 $f + g$ 是 N 上的可行流, 且 $v(f+g) = v(f) + v(g)$.

证 容量限制

$$\begin{aligned} \forall \langle i, j \rangle \in E, \quad 0 \leq g(i, j) \leq c(i, j) - f(i, j), \quad 0 \leq g(j, i) \leq f(i, j), \\ -f(i, j) \leq g(i, j) - g(j, i) \leq c(i, j) - f(i, j) \end{aligned}$$

推出
$$0 \leq f'(i, j) = f(i, j) + g(i, j) - g(j, i) \leq c(i, j).$$

引理证明（续）

平衡条件 $\forall i \in E - \{s, t\}$,

$$\begin{aligned}\sum_{\langle j, i \rangle \in E} f'(j, i) &= \sum_{\langle j, i \rangle \in E} \{f(j, i) + g(j, i) - g(i, j)\} \\ &= \sum_{\langle j, i \rangle \in E} f(j, i) + \sum_{\substack{\langle j, i \rangle \in E(f) \\ \wedge \langle j, i \rangle \in E}} g(j, i) - \sum_{\substack{\langle i, j \rangle \in E(f) \\ \wedge \langle j, i \rangle \in E}} g(i, j)\end{aligned}$$

$$\begin{aligned}\sum_{\langle i, j \rangle \in E} f'(i, j) &= \sum_{\langle i, j \rangle \in E} \{f(i, j) + g(i, j) - g(j, i)\} \\ &= \sum_{\langle i, j \rangle \in E} f(i, j) + \sum_{\substack{\langle i, j \rangle \in E(f) \\ \wedge \langle i, j \rangle \in E}} g(i, j) - \sum_{\substack{\langle j, i \rangle \in E(f) \\ \wedge \langle i, j \rangle \in E}} g(j, i)\end{aligned}$$

而

$$\sum_{\substack{\langle j, i \rangle \in E(f) \\ \wedge \langle j, i \rangle \in E}} g(j, i) - \sum_{\substack{\langle i, j \rangle \in E(f) \\ \wedge \langle j, i \rangle \in E}} g(i, j) - \sum_{\substack{\langle i, j \rangle \in E(f) \\ \wedge \langle i, j \rangle \in E}} g(i, j) + \sum_{\substack{\langle j, i \rangle \in E(f) \\ \wedge \langle i, j \rangle \in E}} g(j, i)$$

$$= \sum_{\langle j, i \rangle \in E(f)} g(j, i) - \sum_{\langle i, j \rangle \in E(f)} g(i, j) = 0$$

引理证明（续）

得

$$\sum_{\langle j,i \rangle \in E} f'(j,i) - \sum_{\langle i,j \rangle \in E} f'(i,j) = 0$$

得证 $f' = f + g$ 是 N 上的可行流. f' 的流量

$$v(f') = \sum_{\langle s,j \rangle \in E} \{f(s,j) + g(s,j) - g(j,s)\}$$

$$- \sum_{\langle j,s \rangle \in E} \{f(j,s) + g(j,s) - g(s,j)\}$$

$$= \sum_{\langle s,j \rangle \in E} f(s,j) + \sum_{\substack{\langle s,j \rangle \in E(f) \\ \wedge \langle s,j \rangle \in E}} g(s,j) - \sum_{\substack{\langle j,s \rangle \in E(f) \\ \wedge \langle s,j \rangle \in E}} g(j,s) \\ - \sum_{\langle j,s \rangle \in E} f(j,s) - \sum_{\substack{\langle j,s \rangle \in E(f) \\ \wedge \langle j,s \rangle \in E}} g(j,s) + \sum_{\substack{\langle s,j \rangle \in E(f) \\ \wedge \langle j,s \rangle \in E}} g(s,j)$$

$$= \left\{ \sum_{\langle s,j \rangle \in E} f(s,j) - \sum_{\langle j,s \rangle \in E} f(j,s) \right\} + \left\{ \sum_{\langle s,j \rangle \in E(f)} g(s,j) - \sum_{\langle j,s \rangle \in E(f)} g(j,s) \right\} \\ = v(f) + v(g)$$

分层辅助网络

- 在 $N(f)$ 中, 设 $d(i)$ 表示 s 到顶点 i 的广度优先搜索的距离, 设 $d = d(t)$ 。

- 定义**分层辅助网络**: $AN(f) = \{V(f), AE(f), ac, s, t\}$

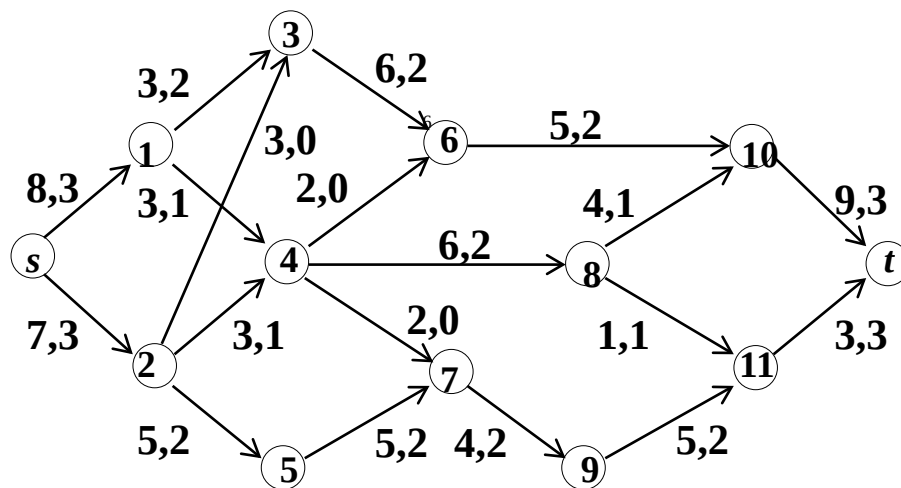
$$V_k(f) = \{i \in V \mid d(i) = k\}, 0 \leq k \leq d - 1$$

$$V_d(f) = \{t\}$$

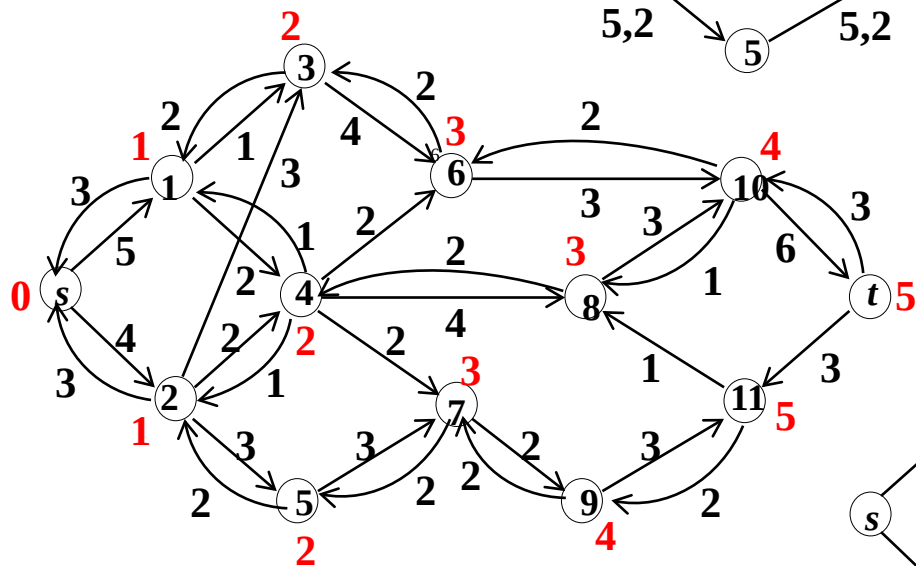
$$V(f) = \bigcup_{k=0}^{d-1} V_k(f)$$

$$AE(f) = \bigcup_{k=0}^{d-1} \{\langle i, j \rangle \mid \langle i, j \rangle \in E(f) \wedge i \in V_k(f), j \in V_{k+1}(f)\}$$

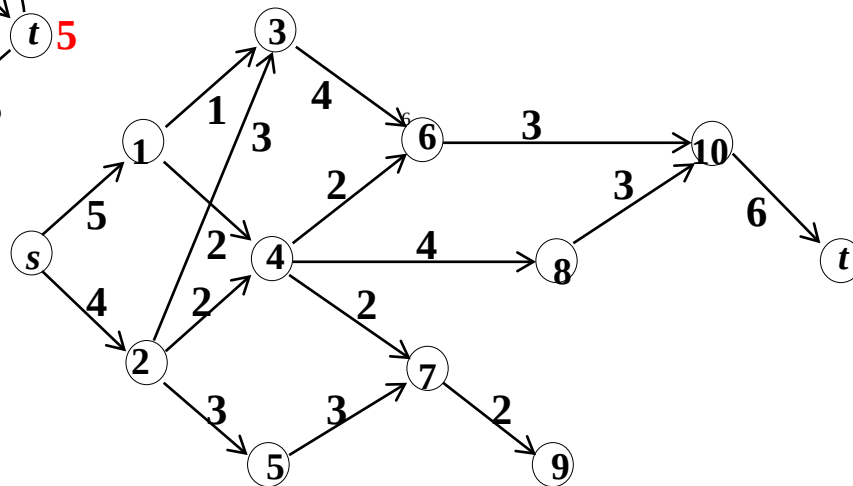
例7.2



N 与 f_1



$N(f_1)$



$AN(f_1)$

Dinic最大流算法思想

- 通过求 $AN(f)$ 的极大流，得到 $AN(f)$ 中尽可能多 $s-t$ 路径，即 N 中多条增广链
- 什么是极大流？
 - 如果不存在只包含前向边的关于 f 的 $s-t$ 增广链，则 f 是 N 中的极大流

Dinic算法

中间点 i 的流通量 $th(i)$: 以 i 为终点的边的容量之和与以 i 为始点的边的容量之和中的小者.

Dinic算法的关键步骤是求 $AN(f)$ 中的极大流, 即在 $AN(f)$ 中找尽可能多的从 s 到 t 的最短路径并给出这些路径上的流量. 删去流通量为 0 的顶点及关联的边. 找到流通量最小的顶点 k , 从 k 开始向后和向前安排流量 $th(k)$, 直到 t 和 s 为止. 修改相关边的容量. 重复进行, 直到 s 和 t 不连通为止. 得到 $AN(f)$ 的极大流 g . 令 $f' = f + g$.

Dinic最大流算法

1. 从0流 f 开始
2. 构造分层辅助网络 $AN(f)$
3. 如果 $AN(f)$ 存在 $s-t$ 最短路径
 - ① 求得 $AN(f)$ 上的极大流 g
 - ② 叠加流 $f = f + g$; GOTO 2
4. 否则, f 为所求最大流

Dinic算法

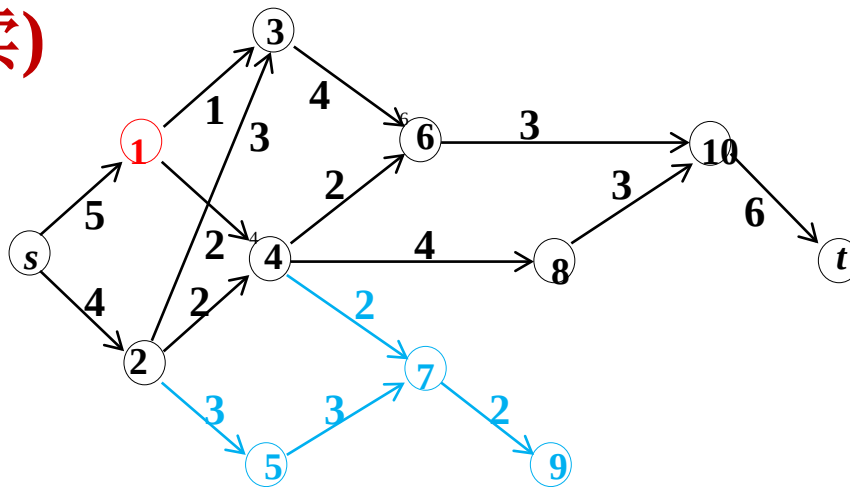
算法7.2 Dinic算法

1. $f \leftarrow 0$ // 取零流作为初始可行流
2. 构造 $AN(f)$
3. if $AN(f)$ 中不存在从 s 到 t 的路径 then return f //计算结束
4. $g \leftarrow 0$
5. if $AN(f)$ 中存在 $th(i)=0$ then
6. if $i=s \vee i=t$ then goto 15
7. else 删去 i 及其关联的边
8. 找到流通量最小的顶点 k , 从 k 开始将 $th(k)$ 个单位的流向前送到 t , 向后推到 s , 并将它们加到 g 上. // $th(k) > 0$
9. 删去 k 及其关联的边

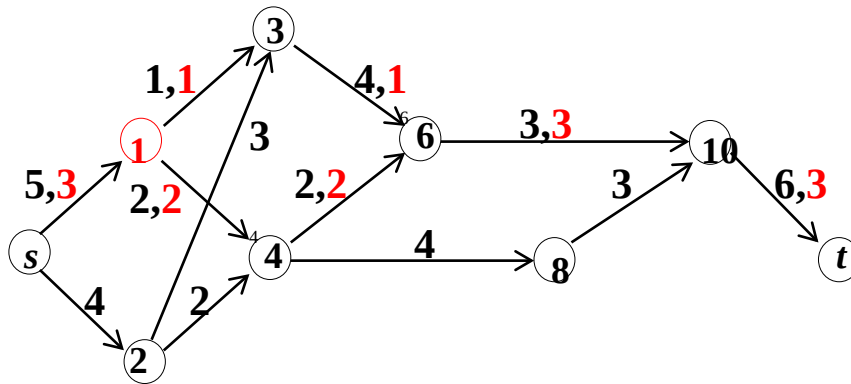
Dinic算法（续）

10. for $\langle i, j \rangle \in AE(f)$ do
11. if $g(i, j) = ac(i, j)$ then
12. 删去 $\langle i, j \rangle$
13. else $ac(i, j) \leftarrow ac(i, j) - g(i, j)$
14. goto 5
15. $f \leftarrow f + g$
16. goto 2

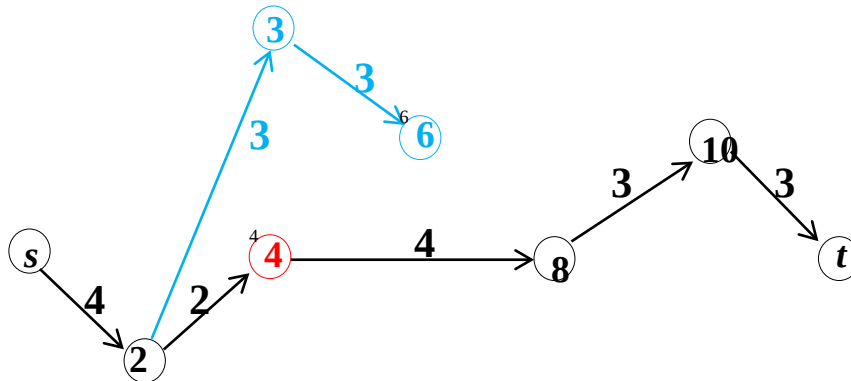
例2 (续)



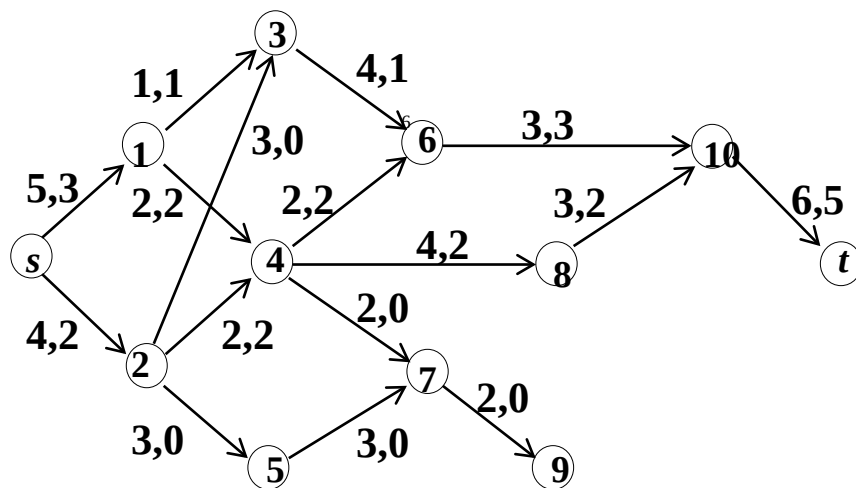
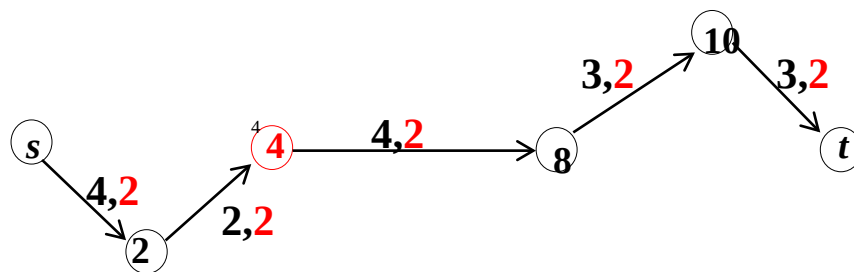
$AN(f_1)$



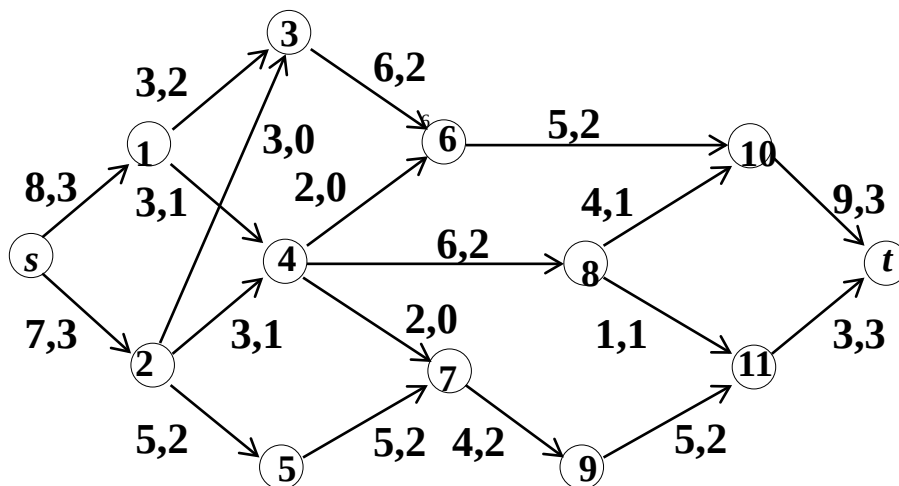
有多条边离开(进入)一个顶点时,安排流的原则是得到流的边要么成为饱和边,要么已用完流量.



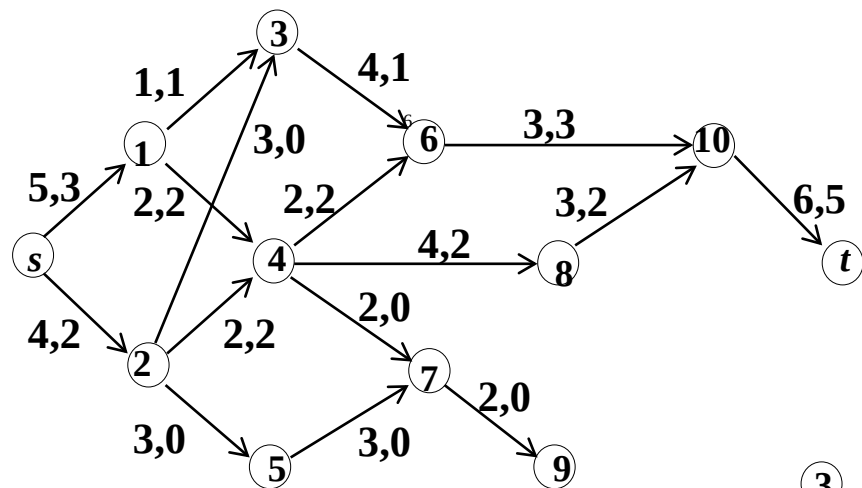
例2(续)



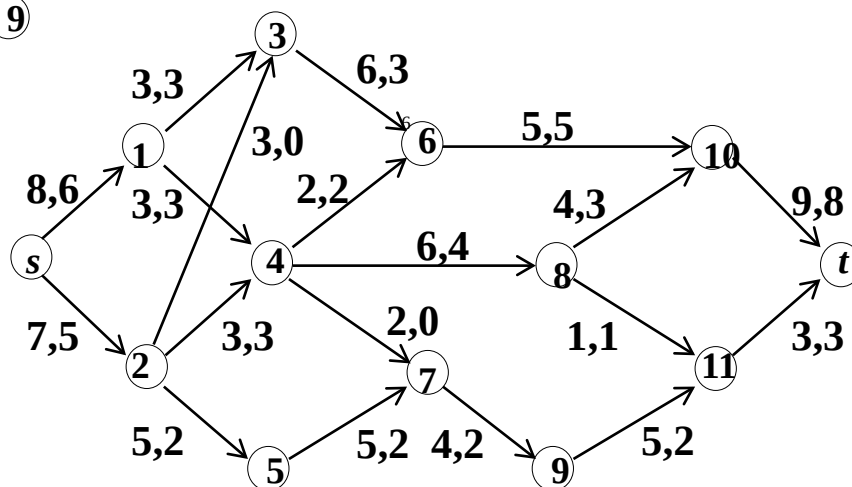
$AN(f_1)$ 与
极大流 g_1



f_1

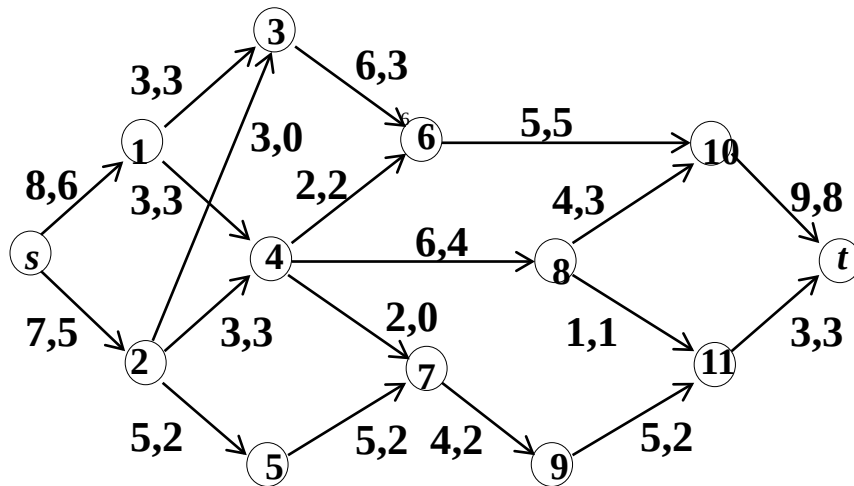


$AN(f_1)$ 与
极大流 g_1

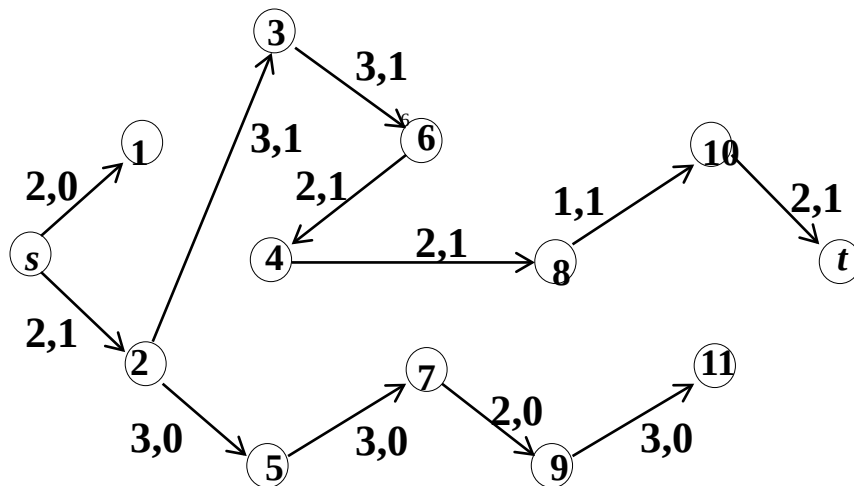


$f_2 = f_1 + g_1$

例2(续)

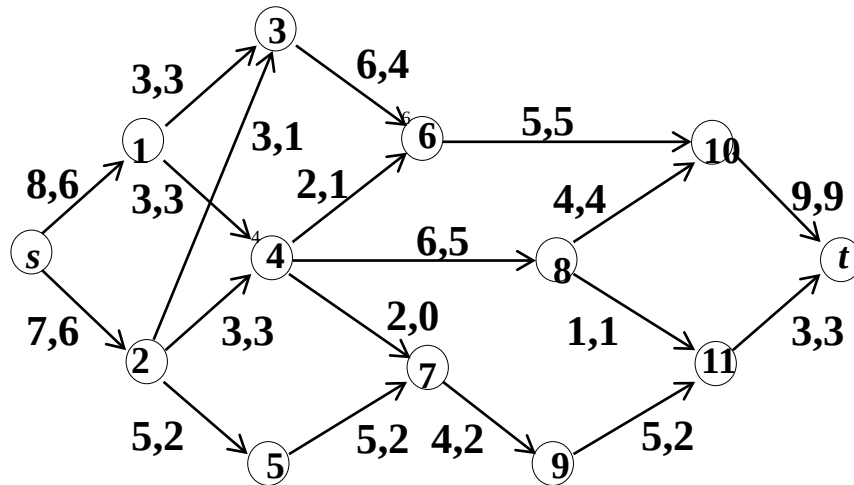


f_2

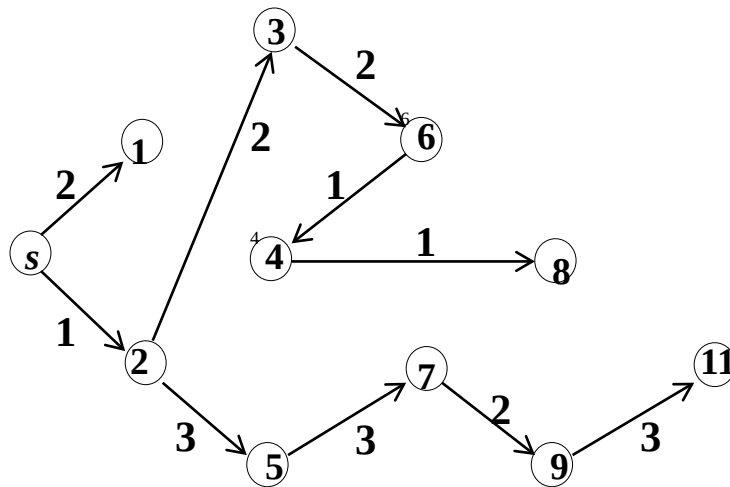


$AN(f_2)$ 与
极大流 g_2

例2(续)



$$f_3 = f_2 + g_2$$



$$AN(f_3)$$

Dinic算法的性质

引理7.6 在每个阶段结束时, g 是 $AN(f)$ 中的极大流.

证 在 $AN(f)$ 中, 只有当流通量变成 0 后, 才删去这个顶点及其关联的边. 这样的顶点和边不可能出现在关于 g 的 s - t 前向增广链上. 当每个阶段结束时, $AN(f)$ 剩余的图中已不存在从 s 到 t 的路径, 从而得到的 g 是 $AN(f)$ 的极大流.

引理7.7 在每个阶段, $AN(f+g)$ 中 s - t 距离大于前一个阶段 $AN(f)$ 中 s - t 距离. 约定, 当不存在 s - t 最短路径时, 规定其距离为 $+\infty$.

证 要证在 $N(f+g)$ 中, 而不在 $AN(f)$ 中的边都不可能从 $AN(f)$ 中的低层指向高层. 假若如此, 则 $AN(f+g)$ 的 s - t 最短路径中必有不在 $AN(f)$ 中的边 (因 g 不是极大流), 而这些边都不可能从 $AN(f)$ 的低层指向高层, 所以 $AN(f+g)$ 中 s - t 距离必大于 $AN(f)$ 中 s - t 距离.

Dinic算法的性质(续1)

考察 $N(f+g)$ 和 $AN(f)$ 中的边. 记 $f' = f+g$. $\forall \langle i,j \rangle \in AE(f)$,

若 $\langle i,j \rangle \in E$, 则 $f'(i,j) = f(i,j) + g(i,j) = c(i,j) - ac(i,j) + g(i,j)$.

当 $g(i,j) < ac(i,j)$ 时, $f'(i,j) < c(i,j)$, $\langle i,j \rangle \in E(f+g)$.

当 $g(i,j) = ac(i,j)$ 时, $f'(i,j) = c(i,j)$, $\langle i,j \rangle \notin E(f+g)$.

当 $f(i,j) = 0$ 且 $g(i,j) > 0$ 时, $N(f+g)$ 比 $N(f)$ 多一条边 $\langle j,i \rangle$.

若 $\langle j,i \rangle \in E$, 则 $f'(j,i) = ac(i,j) - g(i,j)$.

当 $g(i,j) < ac(i,j)$ 时, $f'(j,i) > 0$, $\langle i,j \rangle \in E(f+g)$.

当 $g(i,j) = ac(i,j)$ 时, $f'(j,i) = 0$, $\langle i,j \rangle \notin E(f+g)$.

当 $f(j,i) = c(j,i)$ 且 $f'(j,i) < c(j,i)$ 时, $N(f+g)$ 比 $N(f)$ 多一条边 $\langle j,i \rangle$.

因而, $AN(f)$ 中的边也是 $N(f+g)$ 中的边当且仅当它是关于 g 的非饱和边. 又由于 g 产生的在 $N(f+g)$ 中、而不在 $AN(f)$ 中的新边都是从 $AN(f)$ 的高层指向低层.

Dinic算法的性质（续2）

其余在 $N(f)$ 中,而不在 $AN(f)$ 中的边与 g 无关,故仍在 $N(f+g)$ 中. 这种边不可能从 $AN(f)$ 中的低层指向高层,只可能从高层指向低层,在同一层,或者与不在 $AN(f)$ 中的顶点关联. 总之,在 $N(f+g)$ 中而不在 $AN(f)$ 中的边都不可能从 $AN(f)$ 中的低层指向高层.

定理7.3 Dinic算法得到的 f 是最大流,且算法在 $O(n^3)$ 步内终止.

证 当算法终止时, $AN(f)$ 中不存在从 s 到 t 的路径,因此 $N(f)$ 中也不存在从 s 到 t 的路径,它的最大流为零流,最大流量为 0. 根据引理4, $v^* - v(f) = 0$, 其中 v^* 是最大流量. 推出 $v(f) = v^*$, 得证 f 是最大流.

Dinic算法的性质（续3）

由引理7.7, $AN(f)$ 中 s - t 距离每阶段至少增加 1, 而 s - t 距离不会超过 $n-1$, 故至多有 n 个阶段.

在每个阶段, 构造 $AN(f)$ 可在 $O(m)$ 步内完成, 计算顶点的流通量需 $O(m)$ 步. 在生成 g 的过程中, 找一个流通量最小的顶点需 $O(n)$ 步, 至多找 n 次, 至多需 $O(n^2)$ 步. 边操作可以分成两类. 一类是修改容量后删去这条边(已是饱和边). 这类至多有 $O(m)$ 次. 另一类是修改容量后不删去. 根据安排流量的规定, 从一个顶点送出流时, 被修改容量的边中至多有一条边是非饱和边被保留下来. 从一个流通量最小的顶点安排流至多有 n 条这样的边, 至多进行 n 次, 共有 $O(n^2)$ 次这类操作. 因此, 每个阶段的工作量为 $O(m)+O(m)+O(n^2)+O(m)+O(n^2)=O(n^2)$. 得证总的工作量为 $O(n^3)$.

Dinic算法的性质（续4）

简单容量网络：边的容量均为 1, 所有中间点或入度为1、或出度为 1 的容量网络.

引理7.8 在简单容量网络 N 中, s - t 距离 l 小于等于 n/v^*+1 .

证 把与 s 的距离为 i 的顶点集记作 V_i , $i=1,2,\dots,l-1$. 任何最大流都要经过 V_i , 而 V_i 中每个顶点或入度为1或出度为 1, 其流通量为 1, 因此 $v^* \leq |V_i|$. 于是

$$n \geq \sum_{i=1}^{l-1} |V_i| \geq (l-1)v^*,$$

得证 $l \leq n/v^*+1$.

定理7.4 对简单容量网络, Dinic算法在 $O(n^{1/2}m)$ 步内终止计算.

证 由于容量都为1, 计算过程中每条边流量为 0 或 1. 在每个阶段中, 每条边至多修改一次, 计算量为 $O(m)$.

Dinic算法的性质（续5）

证阶段数为 $O(n^{1/2})$.

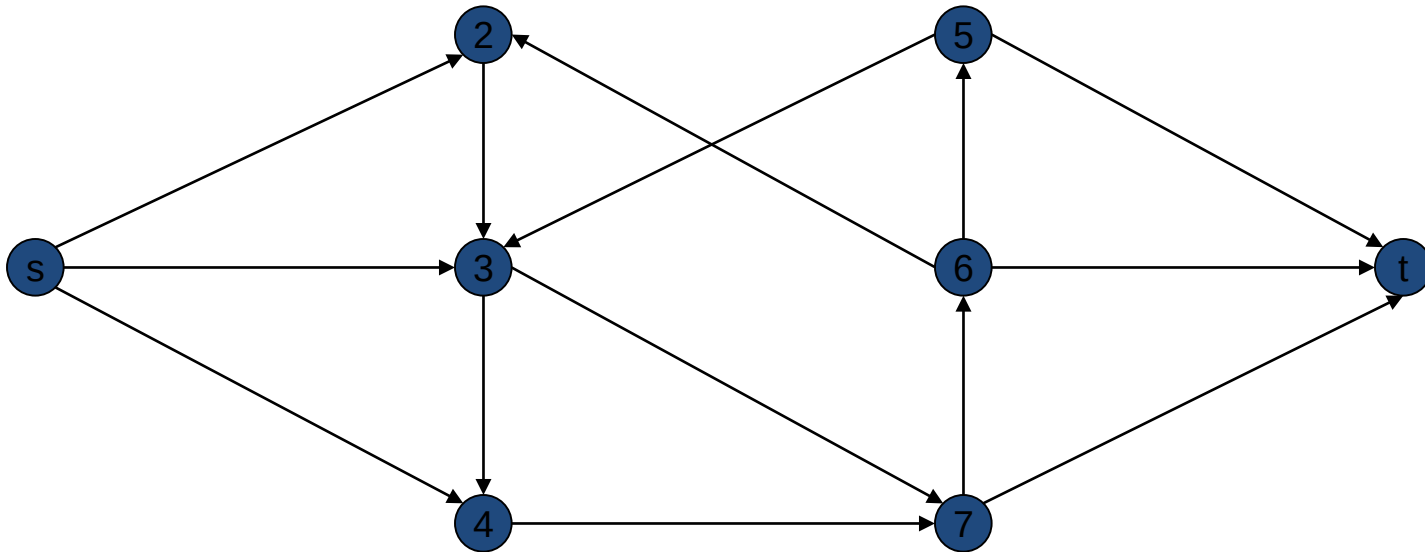
若 $v^* \leq n^{1/2}$, 则显然至多有 $n^{1/2}$ 个阶段. 若 $v^* > n^{1/2}$, 考察流量第一次超过 $v^* - n^{1/2}$ 的阶段. 设这个阶段开始时的可行流为 f , 那么 $N(f)$ 的最大流量大于 $n^{1/2}$. 不难验证, $N(f)$ 也是简单容量网络. 根据引理8, $N(f)$ 中 s - t 距离小于等于

$$n/n^{1/2} + 1 = n^{1/2} + 1,$$

从而在这个阶段之前至多有 $n^{1/2}$ 个阶段. 而这个阶段结束时流量已超过 $v^* - n^{1/2}$, 从而此后也至多有 $n^{1/2}$ 个阶段, 总阶段数不超过 $2n^{1/2} + 1$.

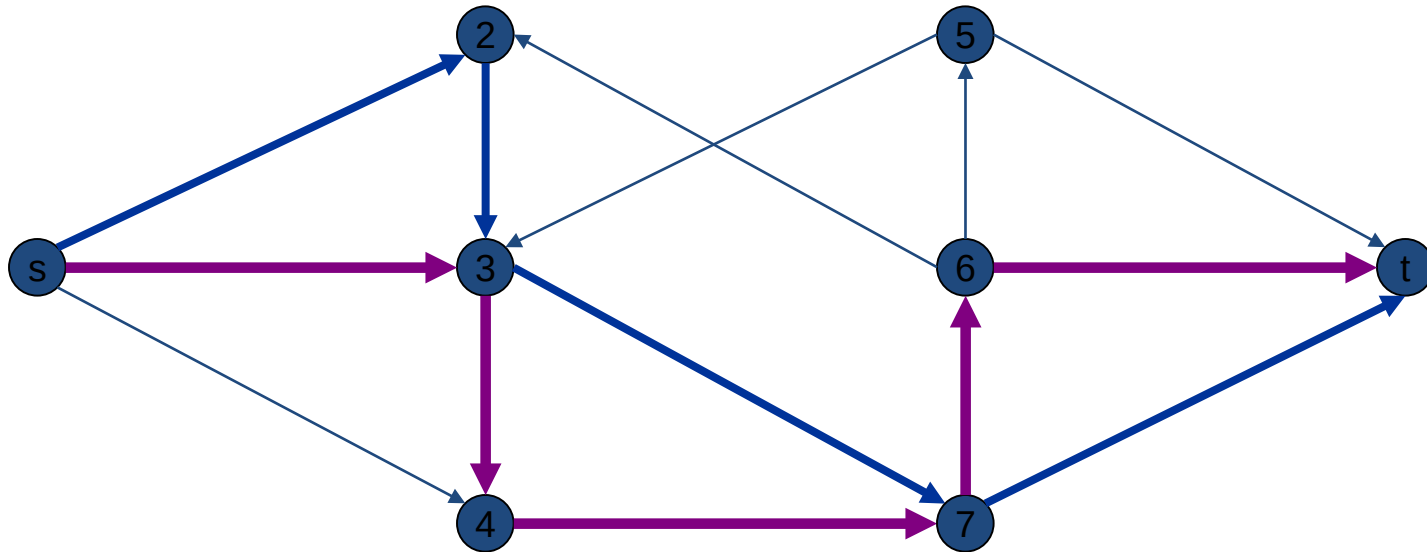
Edge Disjoint Paths

Two paths are **edge-disjoint** if they have no edge in common



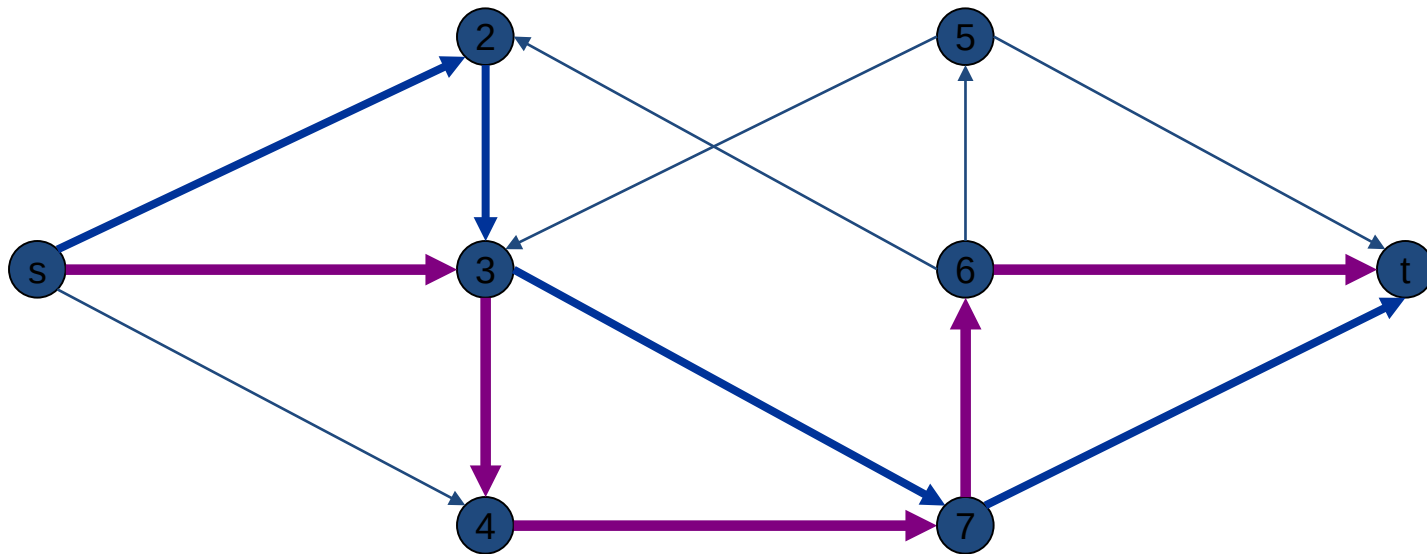
Edge Disjoint Paths

Two paths are **edge-disjoint** if they have no edge in common



Edge Disjoint Paths Problem

Given a directed graph $G = (V, E)$ and two nodes s and t , find the max number of edge-disjoint paths from s to t



Why might this be useful?

Edge Disjoint Paths Problem

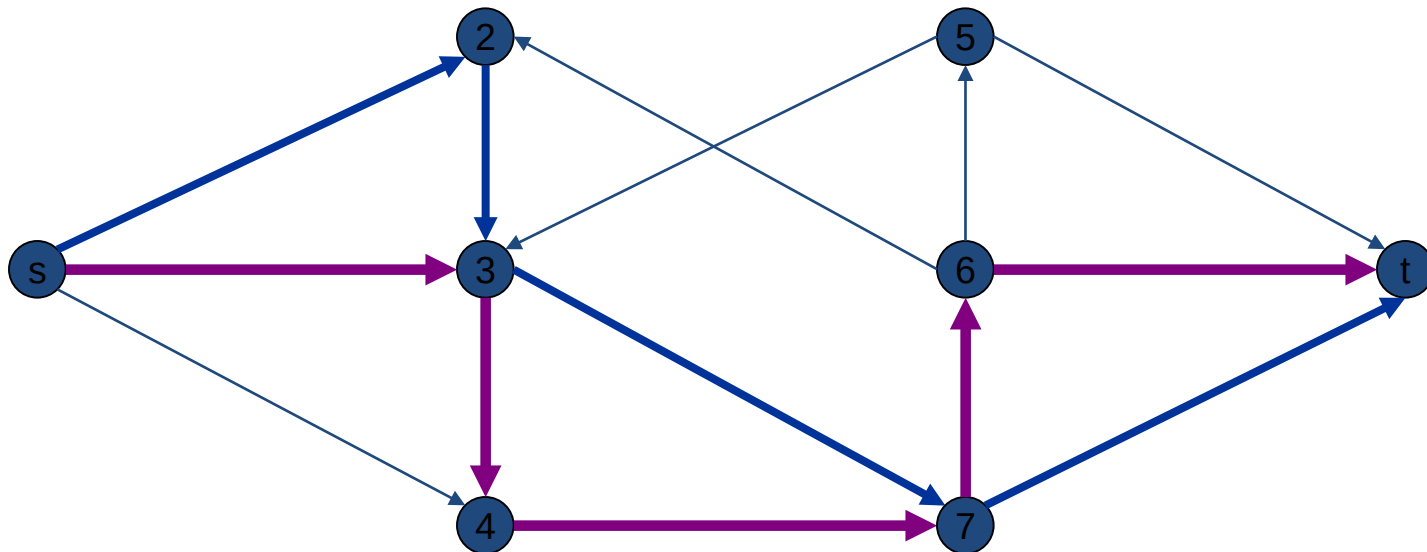
Given a directed graph $G = (V, E)$ and two nodes s and t , find the max number of edge-disjoint paths from s to t

Why might this be useful?

- edges are unique resources (e.g. communications, transportation, etc.)
- how many *concurrent (non-conflicting)* paths do we have from s to t

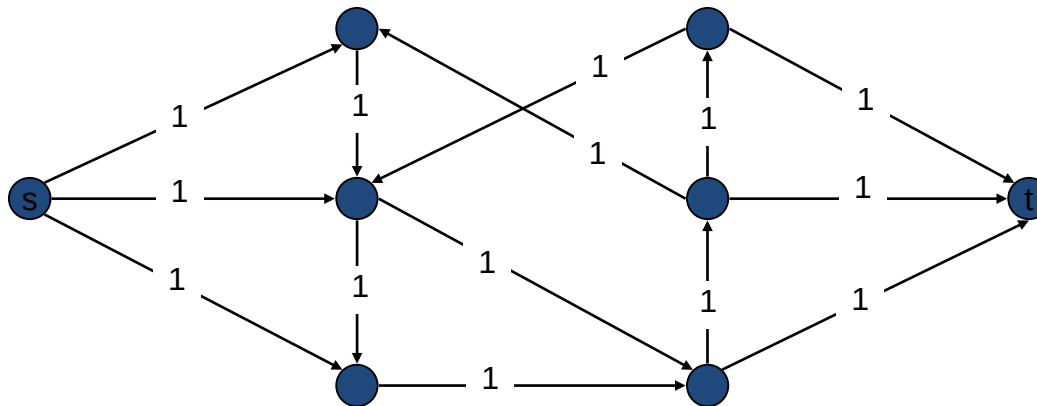
Edge Disjoint Paths

Algorithm ideas?



Edge Disjoint Paths

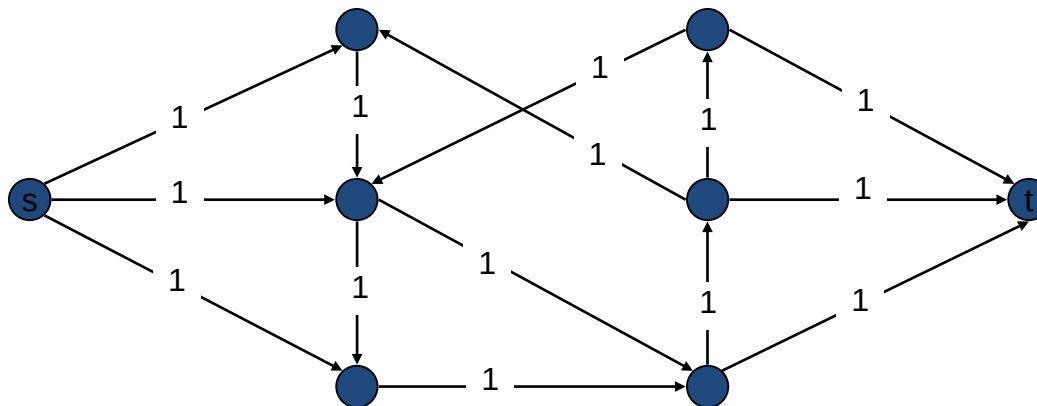
Max flow formulation: assign unit capacity to every edge



What does the max flow represent?
Why?

Edge Disjoint Paths

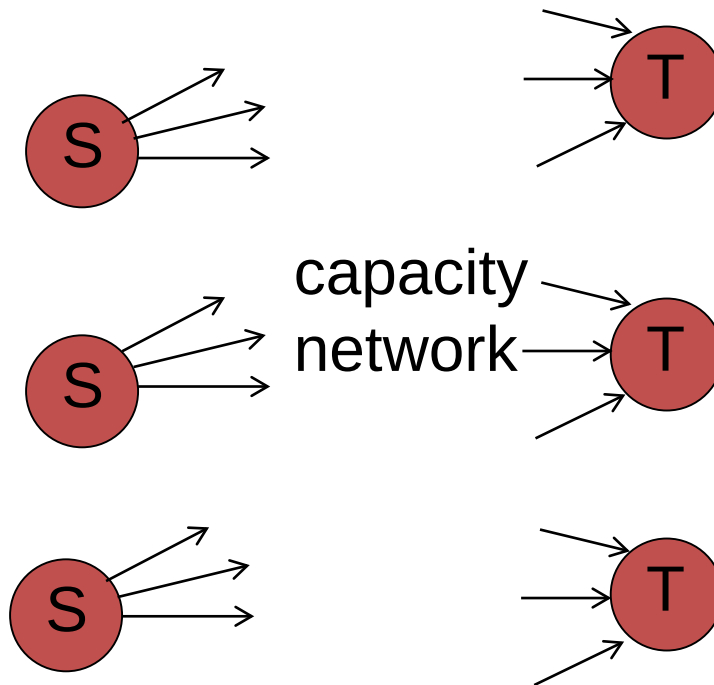
Max flow formulation: assign unit capacity to every edge



- max-flow = maximum number of disjoint paths
- correctness:
 - each edge can have at most flow = 1, so can only be traversed once
 - therefore, each unit out of s represents a separate path to t

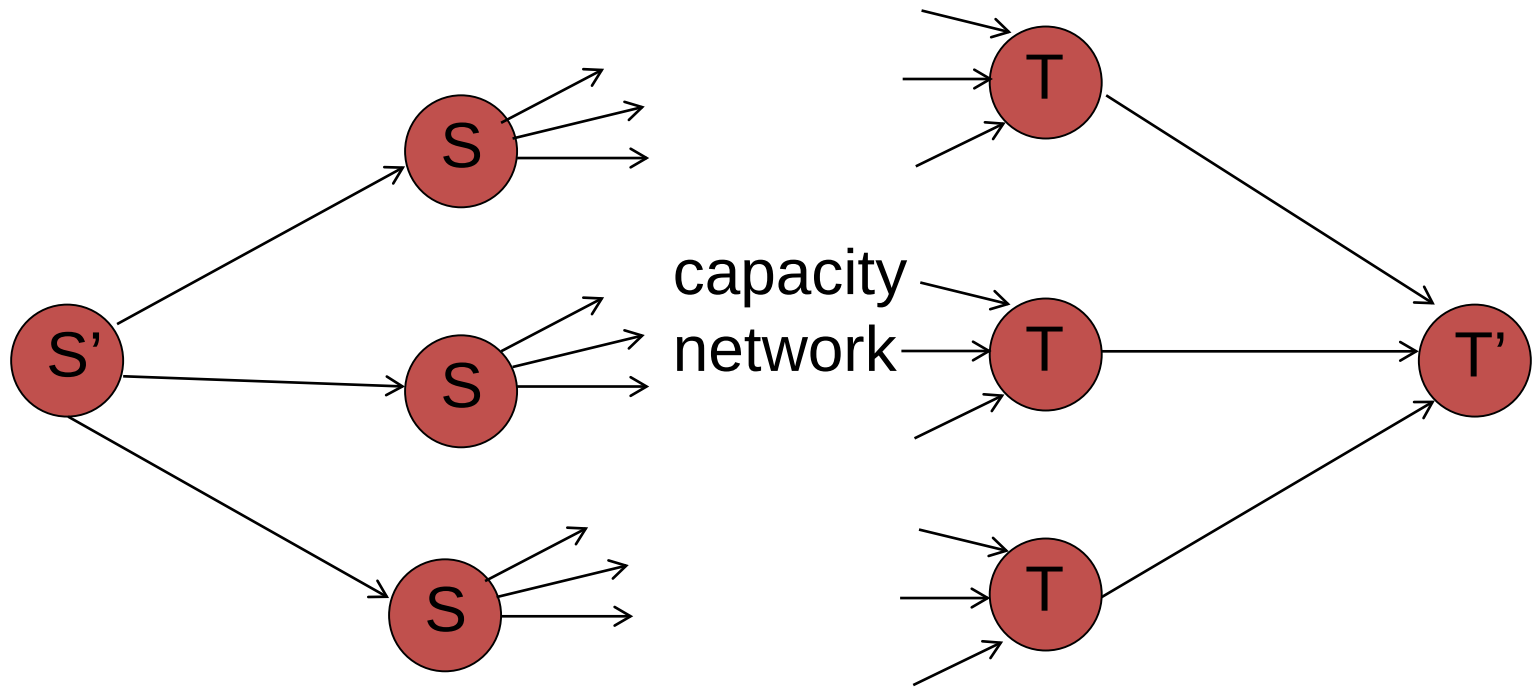
Max-flow variations

What if we have multiple sources and multiple sinks?



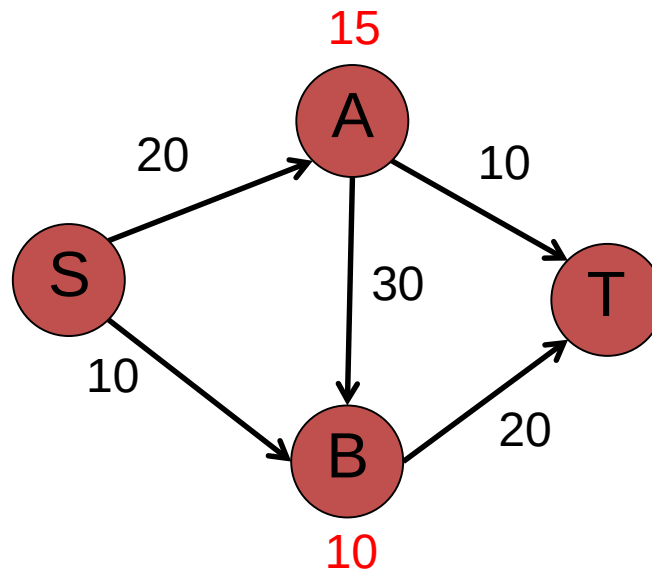
Max-flow variations

Create a new source and sink and connect up with infinite capacities...



Max-flow variations

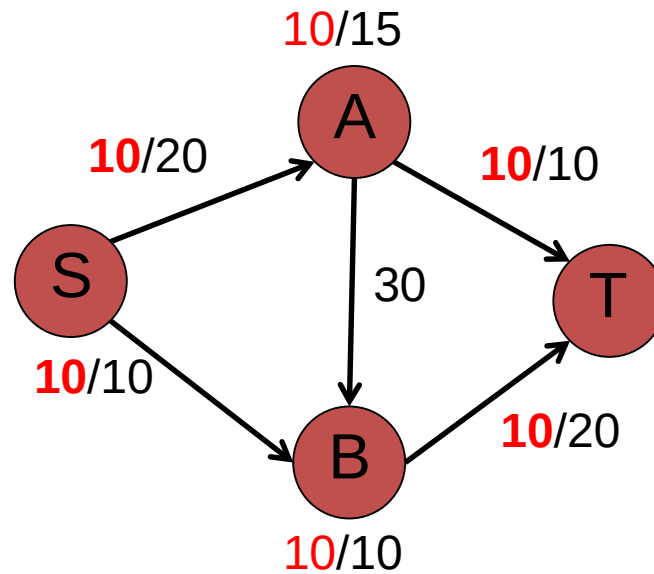
Vertex capacities: in addition to having edge capacities we can also restrict the amount of flow through each vertex



What is the max-flow now?

Max-flow variations

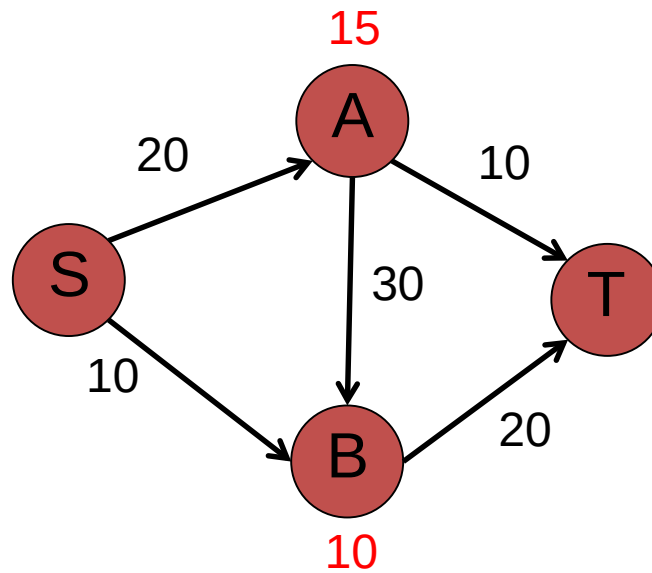
Vertex capacities: in addition to having edge capacities we can also restrict the amount of flow through each vertex



20 units

Max-flow variations

Vertex capacities: in addition to having edge capacities we can also restrict the amount of flow through each vertex

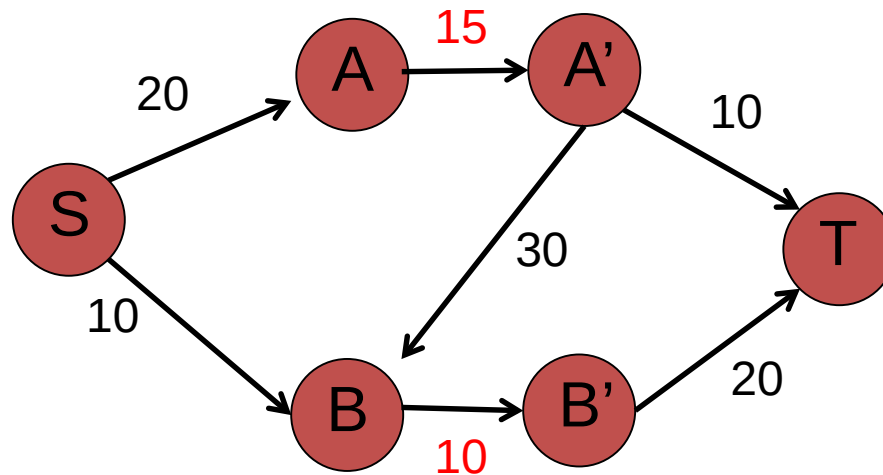


How can we solve this problem?

Max-flow variations

For each vertex v

- create a new node v'
- create an edge with the vertex capacity from v to v'
- move all outgoing edges from v to v'



Can you now prove it's correct?

Max-flow variations

Proof:

1. show that if a solution exists in the original graph, then a solution exists in the modified graph
2. show that if a solution exists in the modified graph, then a solution exists in the original graph

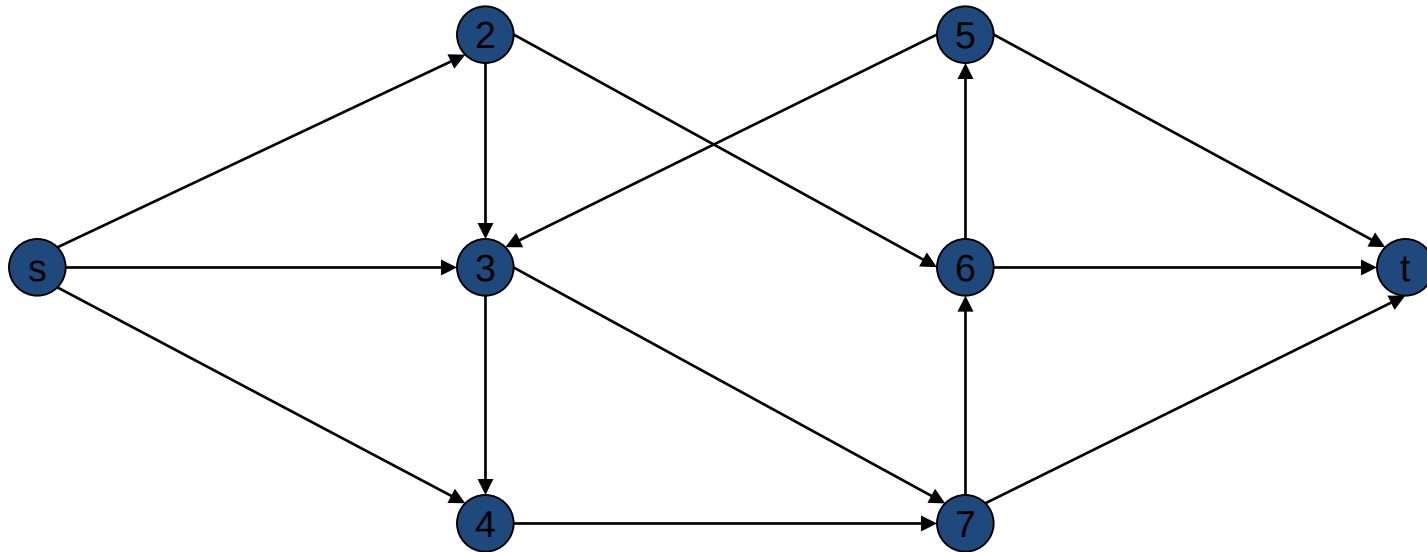
Max-flow variations

Proof:

- we know that the vertex constraints are satisfied
 - no incoming flow can exceed the vertex capacity since we have a single edge with that capacity from v to v'
- we can obtain the solution, by collapsing each v and v' back to the original v node
 - in-flow = out-flow since there is only a single edge from v to v'
 - because there is only a single edge from v to v' and all the in edges go in to v and out to v' , they can be viewed as a single node in the original graph

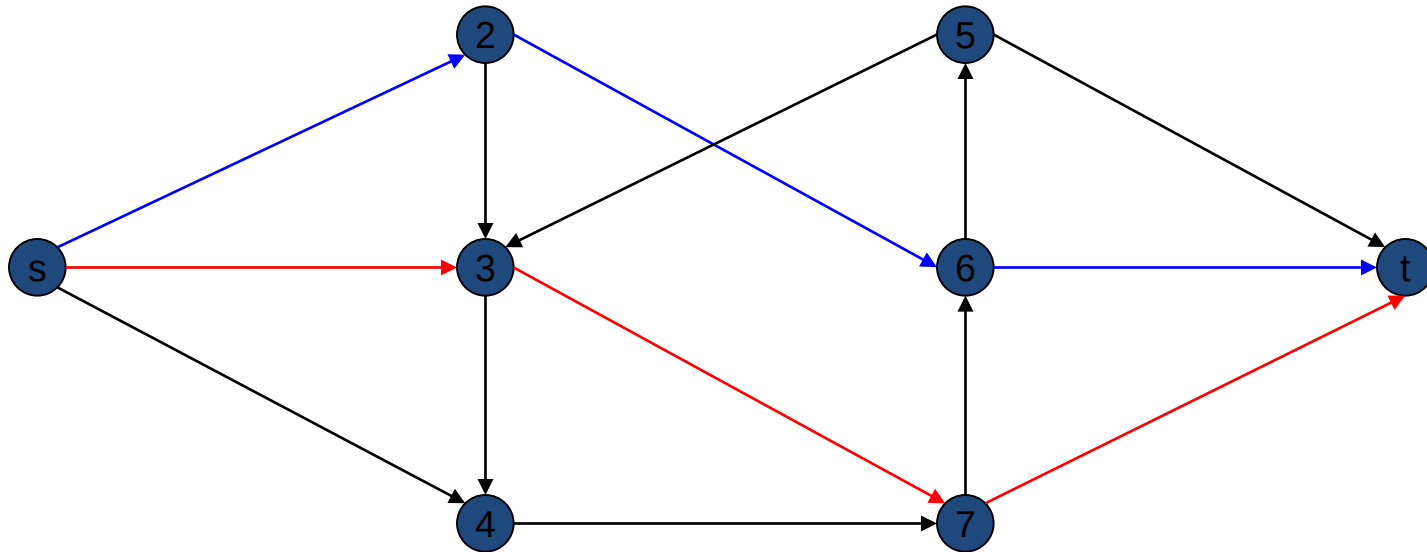
More problems: maximum independent path

Two paths are **independent** if they have no *vertices* in common



More problems: maximum independent path

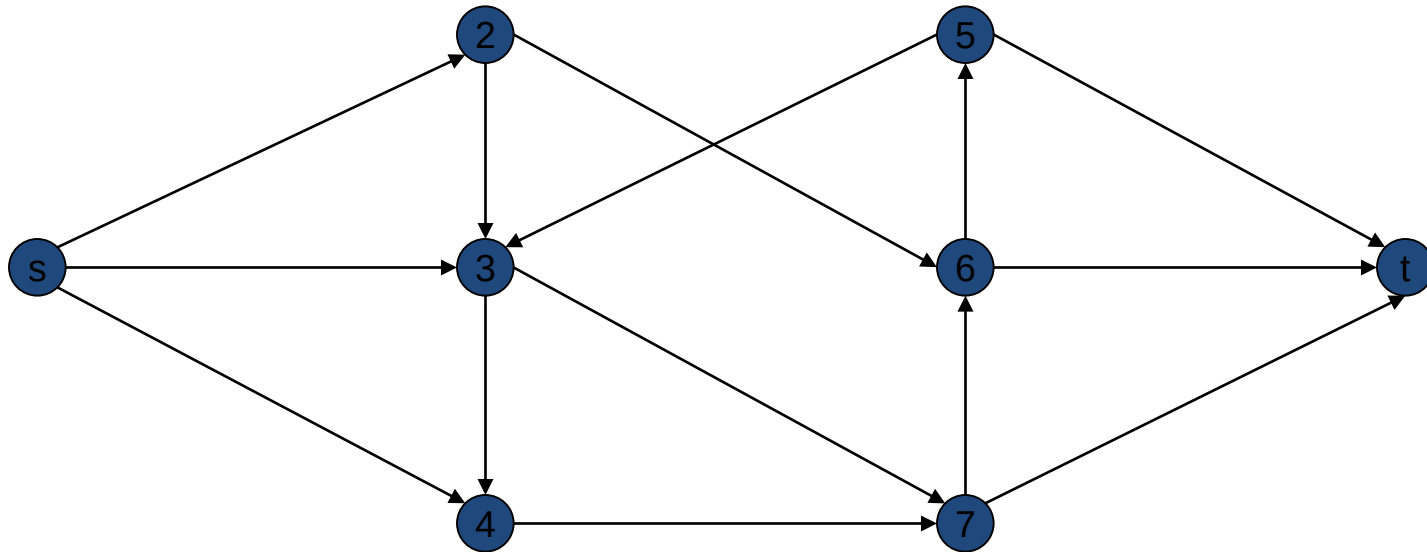
Two paths are *independent* if they have no *vertices* in common



More problems: maximum independent path

Find the maximum number of independent paths

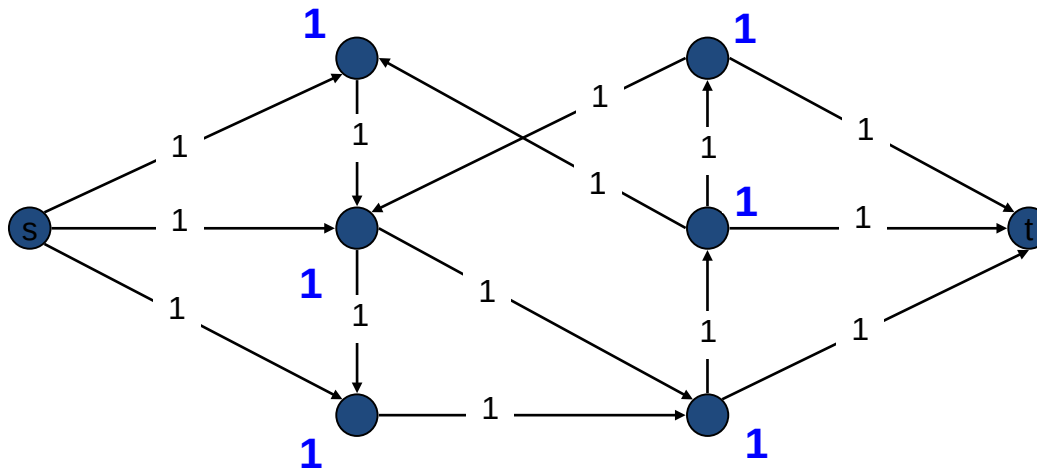
Ideas?



maximum independent path

Max flow formulation:

- assign unit capacity to every edge (though any value would work)
- assign unit capacity to every vertex



Same idea as the maximum edge-disjoint paths, but now we also constrain the vertices

7.2.1 Floyd算法

求带负权的最短路径和检测负回路的算法

负回路 赋权有向图 $D=\langle V,E,w\rangle$ 中权为负数的回路

Dijkstra最短路算法不再适用（算法要求所有边的权值均为非负）

命题7.1 赋权有向图 D 中任意两点之间都有最短路径或不存在路径当且仅当 D 中不含负回路.

证 假设 D 中有负回路 C , i 是 C 上的一个顶点, 那么从 i 到 j 的路径可以先重复走 C 若干次再到 j , 因而 i 到 j 的路径的权可以任意小. 反之, 假设 D 中不存在负回路 C . 对任意两点 i 和 j , 如果从 i 到 j 的路径中有两个相同的顶点, 那么删去这两个顶点之间的路径(回路)后仍是从 i 到 j 的路径, 其权不会增加, 因而只需考虑从 i 到 j 顶点都不相同的路径. 而从 i 到 j 顶点不同的路径只有有限条, 故一定存在最短路径.

Floyd-Warshall算法

算法采用动态规划方法.

$d^{(k)}(i,j)$:从 i 到 j 经过号码不大于 k 的最短路径的长度.

递推方程

$$d^{(0)}(i, j) = w(i, j), \quad 1 \leq i, j \leq n$$

$$d^{(k)}(i, j) = \min\{d^{(k-1)}(i, j), d^{(k-1)}(i, k) + d^{(k-1)}(k, j)\},$$

$$1 \leq i, j \leq n \text{ 且 } i, j \neq k, 1 \leq k \leq n$$

规定: $\forall i \in V, w(i, i) = 0$; $\forall \langle i, j \rangle \notin E, w(i, j) = +\infty$.

当 D 中不存在负回路时, i 到 j 的距离 $d(i, j) = d^{(n)}(i, j)$. 当 D 中存在负回路时, 设负回路 C 经过 i , 除 i 外顶点的最大号码是 k , 则必有 $d^{(k)}(i, i) < 0$.

Floyd-Warshall算法

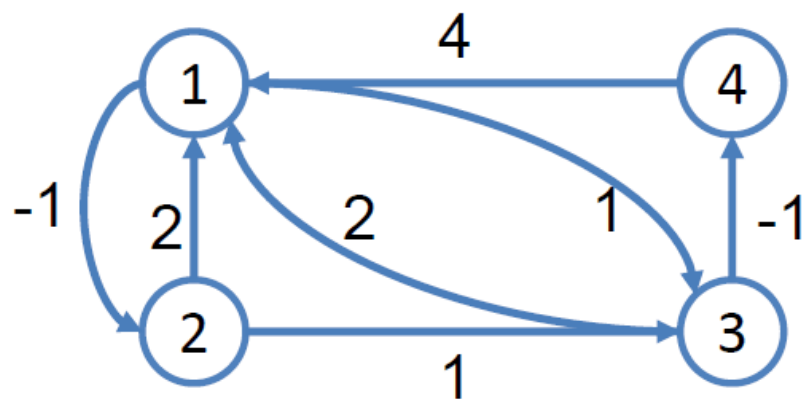
$h^{(k)}(i,j)$:从 i 到 j 经过号码不大于 k 的最短路径中 i 的下一个顶点.
递推关系

$$h^{(0)}(i, j) = \begin{cases} j, & \text{若 } \langle i, j \rangle \in E, \\ 0, & \text{否则,} \end{cases} \quad 1 \leq i, j \leq n$$
$$h^{(k)}(i, j) = \begin{cases} h^{(k-1)}(i, j), & \text{若 } d^{(k-1)}(i, j) \leq d^{(k-1)}(i, k) \\ & + d^{(k-1)}(k, j), \\ h^{(k-1)}(i, k), & \text{否则,} \end{cases}$$
$$1 \leq i, j \leq n \text{ 且 } i, j \neq k, 1 \leq k \leq n$$

Floyd算法——算法示例

$$d^{(0)} = \begin{bmatrix} 0 & -1 & 1 & +\infty \\ 2 & 0 & 1 & +\infty \\ 2 & +\infty & 0 & -1 \\ 4 & +\infty & +\infty & 0 \end{bmatrix}$$

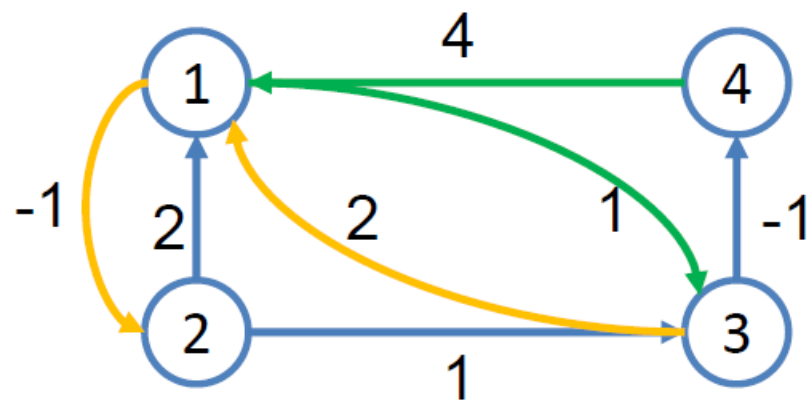
$$h^{(0)} = \begin{bmatrix} 1 & 2 & 3 & 0 \\ 1 & 2 & 3 & 0 \\ 1 & 0 & 3 & 4 \\ 1 & 0 & 0 & 4 \end{bmatrix}$$



Floyd算法——算法示例

$$d^{(1)} = \begin{bmatrix} 0 & -1 & 1 & +\infty \\ 2 & 0 & 1 & +\infty \\ 2 & \color{red}{1} & 0 & -1 \\ 4 & \color{red}{3} & \color{red}{5} & 0 \end{bmatrix}$$

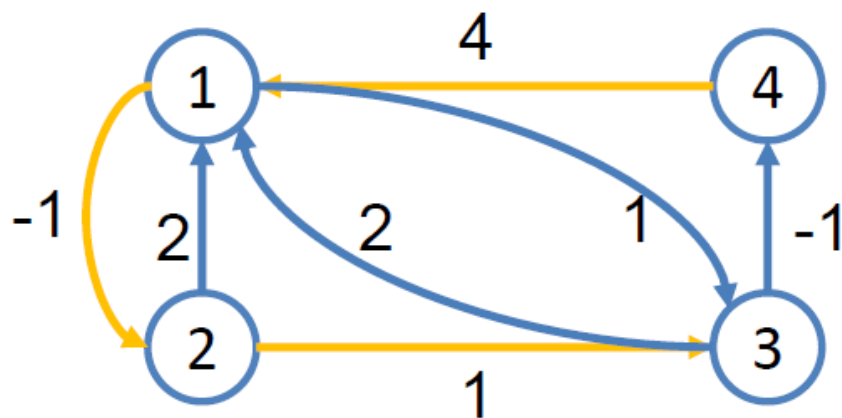
$$h^{(1)} = \begin{bmatrix} 1 & 2 & 3 & 0 \\ 1 & 2 & 3 & 0 \\ 1 & \color{red}{1} & 3 & 4 \\ 1 & \color{red}{1} & \color{red}{1} & 4 \end{bmatrix}$$



Floyd算法——算法示例

$$d^{(2)} = \begin{bmatrix} 0 & -1 & 0 & +\infty \\ 2 & 0 & 1 & +\infty \\ 2 & 1 & 0 & -1 \\ 4 & 3 & 4 & 0 \end{bmatrix}$$

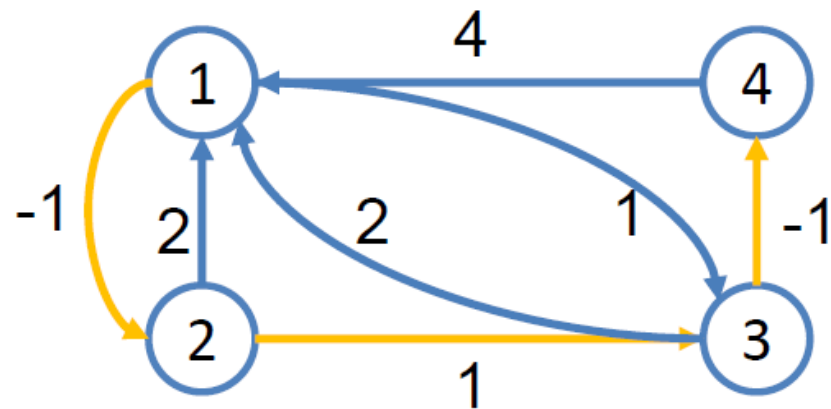
$$h^{(2)} = \begin{bmatrix} 1 & 2 & 2 & 0 \\ 1 & 2 & 3 & 0 \\ 1 & 1 & 3 & 4 \\ 1 & 1 & 1 & 4 \end{bmatrix}$$



Floyd算法——算法示例

$$d^{(3)} = \begin{bmatrix} 0 & -1 & 0 & -1 \\ 2 & 0 & 1 & 0 \\ 2 & 1 & 0 & -1 \\ 4 & 3 & 4 & 0 \end{bmatrix}$$

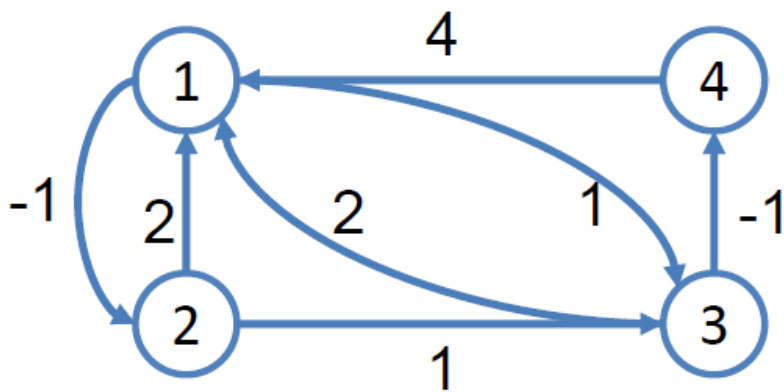
$$h^{(3)} = \begin{bmatrix} 1 & 2 & 2 & 2 \\ 1 & 2 & 3 & 3 \\ 1 & 1 & 3 & 4 \\ 1 & 1 & 1 & 4 \end{bmatrix}$$



Floyd算法——算法示例

$$d^{(4)} = \begin{bmatrix} 0 & -1 & 0 & -1 \\ 2 & 0 & 1 & 0 \\ 2 & 1 & 0 & -1 \\ 4 & 3 & 4 & 0 \end{bmatrix}$$

$$h^{(4)} = \begin{bmatrix} 1 & 2 & 2 & 2 \\ 1 & 2 & 3 & 3 \\ 1 & 1 & 3 & 4 \\ 1 & 1 & 1 & 4 \end{bmatrix}$$



例 (续)

根据 $d^{(4)}$ 和 $h^{(4)}$, 可以得到任意两点之间的最短路径和距离.

例如, 由 $d^{(4)}(1,4)=-1$, $h^{(4)}(1,4)=2$, $h^{(4)}(2,4)=3$, $h^{(4)}(3,4)=4$, 得到1到4的最短路径是1-2-3-4, 距离为-1.

又如, 由 $d^{(4)}(3,2)=1$, $h^{(4)}(3,2)=1$, $h^{(4)}(1,2)=2$, 得到3到2的最短路径是3-1-2, 距离为1.

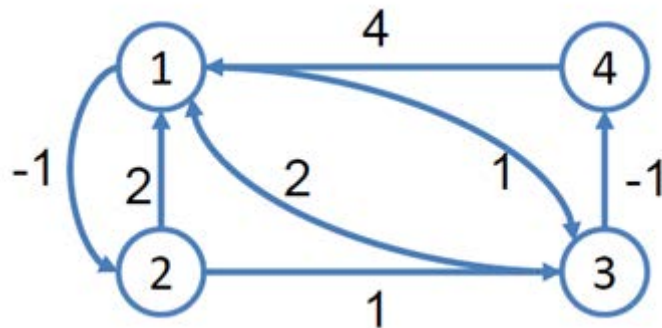
Floyd最短路径算法有 n 个阶段, 每个阶段都要检查或更新所有点对的当前最短路径长度, 更新的开销是常数, Floyd算法的时间复杂度为 $O(n^3)$

7.2 最小费用流

定义7.7 在容量网络 $N = \langle V, E, c, s, t \rangle$ 中添加单位费用 $w: E \rightarrow R^*$, 称作**容量-费用网络**, 记作 $N = \langle V, E, c, w, s, t \rangle$.

设 f 是 N 上的一个可行流, 称 $w(f) = \sum_{\langle i, j \rangle \in E} w(i, j) f(i, j)$ 为 f 的**费用**. 在所有流量为 v_0 的可行流中费用最小的称作流量 v_0 的**最小费用流**.

最小费用流问题 给定容量-费用网络 N 和流量 v_0 , 求流量 v_0 的最小费用流.



7.2.2 最小费用流的负回路算法

设容量-费用网络 $N = \langle V, E, c, w, s, t \rangle$, 关于可行流 f 的辅助网络 $N(f) = \langle V, E(f), ac, aw, s, t \rangle$, 其中 $E(f)$ 和 ac 的定义与最大流辅助网络一致

辅助费用

$$aw(i, j) = \begin{cases} w(i, j), & \text{若 } \langle i, j \rangle \in E^+(f) \\ -w(j, i), & \text{若 } \langle i, j \rangle \in E^-(f) \end{cases}$$

下述引理是引理5在容量-费用网络上的继续.

引理7.9 设 f 是容量-费用网络 N 上的可行流, g 是辅助网络 $N(f)$ 上的可行流, $f' = f + g$, 则

$$w(f') = w(f) + aw(g)$$

证明:略, 类似引理7.5的证明。

最小费用流的负回路算法

定义7.8 设 C 是 N 中一条边不重复的回路, C 上的圈流 h^C 定义如下: $\forall \langle i, j \rangle \in E(C), h^C(i, j) = \delta$; $\forall \langle i, j \rangle \in E - E(C), h^C(i, j) = 0$, 其中 $\delta > 0$ 称作 h^C 的环流量. $\delta = \min_{\langle i, j \rangle \in E(C)} \{c(i, j)\} > 0$

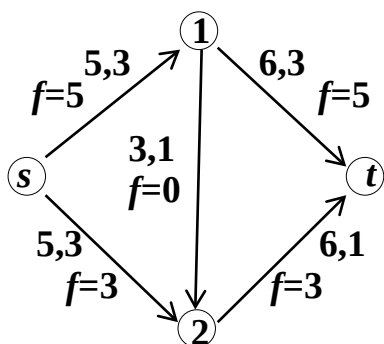
$v(h^C) = 0$, $w(h^C) = \delta \cdot w(C)$, 其中 $w(C) = \sum_{\langle i, j \rangle \in E(C)} w(i, j)$

设 f 是 N 上的一个可行流, h^C 是 $N(f)$ 上的一个圈流, $f' = f + h^C$. f' 是 N 上的可行流, 且 $v(f') = v(f)$, $w(f') = w(f) + \delta \cdot w(C)$. 如果 $w(C) < 0$, 则 $w(f') < w(f)$.

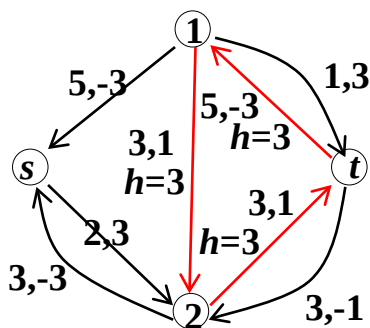
求最小费用流的构想: 首先求一个流量 v_0 的可行流 f . 如果 $N(f)$ 中存在权 w 的负回路 C , 令 $f' \leftarrow f + h^C$, 重复进行, 直至辅助网络中不存在权 w 的负回路为止

最小费用流的负回路算法

例7.4 旁边的 2个数依次是容量和费用

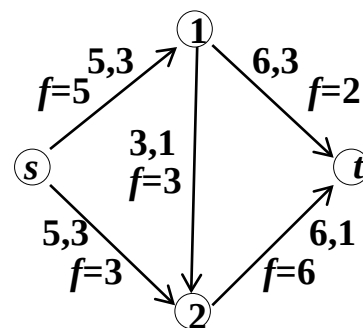


$$v_0=8, w(f)=42.$$



$$aw(C) = -1, \delta=3$$

$$aw(h^C) = -3.$$



$$f_1 = f + h^C$$

$$w(f_1) = 42 - 3 = 39.$$

关键：证明当 $N(f)$ 中不存在权 aw 的负回路时, f 是最小费用流.

最小费用流的负回路算法

定义7.9 设 f_1, f_2 是两个可行流, 在 $N(f_2)$ 上定义 $g = f_1 - f_2$:

$\forall \langle i, j \rangle \in E$,

若 $f_1(i, j) > f_2(i, j)$, 则 $g(i, j) = f_1(i, j) - f_2(i, j)$;

若 $f_1(i, j) < f_2(i, j)$, 则 $g(j, i) = f_2(i, j) - f_1(i, j)$.

在 $N(f_2)$ 的其它边上 g 都为 0.

引理7.10 设 f_1, f_2 是两个可行流, $g = f_1 - f_2$, 则 g 是 $N(f_2)$ 上的可行流且 $f_1 = f_2 + g$.

最小费用流的负回路算法

引理7.11 如果 f 不是零流且 $v(f) = 0$, 则 f 等于若干个圈流之和.

证 记 $E_1 = \{ \langle i, j \rangle \in E \mid f(i, j) > 0 \}$, D_1 是 E_1 的导出子图. 由于 f 不是零流, $E_1 \neq \emptyset$. 又由于 $v(f) = 0$, D_1 中的每个顶点的出度大于 0. 不难证明 D_1 中必存在边不重复的回路 C_1 . 设

$$\delta_1 = \min \{ f(i, j) \mid \langle i, j \rangle \in E(C_1) \},$$

h^{C_1} 是 C_1 上环流量 δ_1 的圈流.

令 $f_1 = f - h^{C_1}$, $v(f_1) = 0$ 且 f_1 比 f 至少多一条零流边. 重复进行, 得到圈流 $h^{C_2}, \dots, h^{C_l}$ 使得 $f_l = f - h^{C_1} - \dots - h^{C_l}$ 为零流, 从而

$$f = h^{C_1} + h^{C_2} + \dots + h^{C_l}.$$

最小费用流的负回路算法

定理7.5 设 f 是 N 上流量 v_0 的可行流, 则 f 是最小费用流当且仅当 $N(f)$ 中不存在以辅助费用 aw 为权的负回路.

证 充分性. 假设 $N(f)$ 中不存在关于 aw 的负回路, f' 是 N 上任一流量 v_0 的可行流. $G = f' - f$ 是 $N(f)$ 上的可行流且 $v(g) = 0$. 不妨设 $f' \neq f$, 于是 g 不是零流, 从而 $N(f)$ 中存在回路 C_j 及其圈流 h^{C_j} ($1 \leq j \leq l$) 使得 $g = h^{C_1} + h^{C_2} + \dots + h^{C_l}$. g 的辅助费用

$$aw(g) = \delta_1 \cdot aw(C_1) + \delta_2 \cdot aw(C_2) + \dots + \delta_l \cdot aw(C_l)$$

由于 $N(f)$ 中不存在关于权 aw 的负回路, 所有 $aw(C_r) \geq 0$, 从而 $aw(g) \geq 0$. 又由引理7.9, $f' = f + g$, $w(f') = w(f) + aw(g) \geq w(f)$. 得证 f 是最小费用流.

最小费用流的负回路算法

算法7.4 最小费用流的负回路算法

1. 调用最大流算法, 若求得流量 v_0 的可行流 f , 则转2; 若最大流量小于 v_0 , 则输出“无流量 v_0 的可行流”, 计算结束.
2. 构造 $N(f)$.
3. 用 Floyd算法检测 $N(f)$ 中是否存在权 c_{ij} 的负回路. 若存在负回路 C , 则转4; 若不存在负回路, 则输出 f , 计算结束.
4. 令 $\delta \leftarrow \min\{ c_{ij} \mid \langle i, j \rangle \in E(C) \}$ // h^C 的环流量为 δ
5. $\forall \langle i, j \rangle \in E(C)$, 若 $\langle i, j \rangle \in E$, 令 $f(i, j) \leftarrow f(i, j) + \delta$; 若 $\langle j, i \rangle \in E$, 令 $f(j, i) \leftarrow f(j, i) - \delta$.
// $f \leftarrow f + h^C$
6. 转2

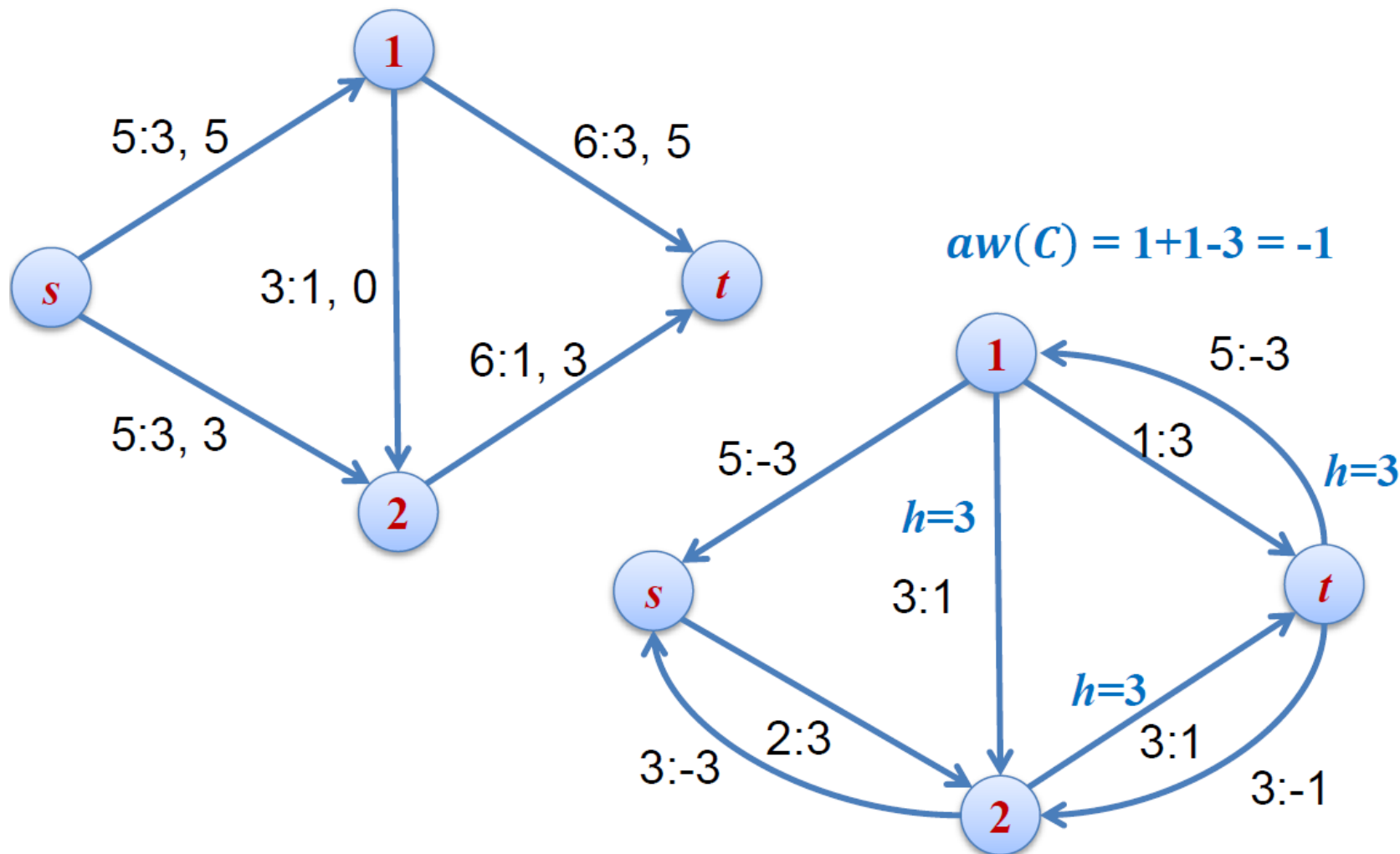
算法7.4的有限终止性

假设所有的容量都是整数, 算法计算过程中的可行流都是整数值, 圈流的环流量也是整数, 大于等于 1.

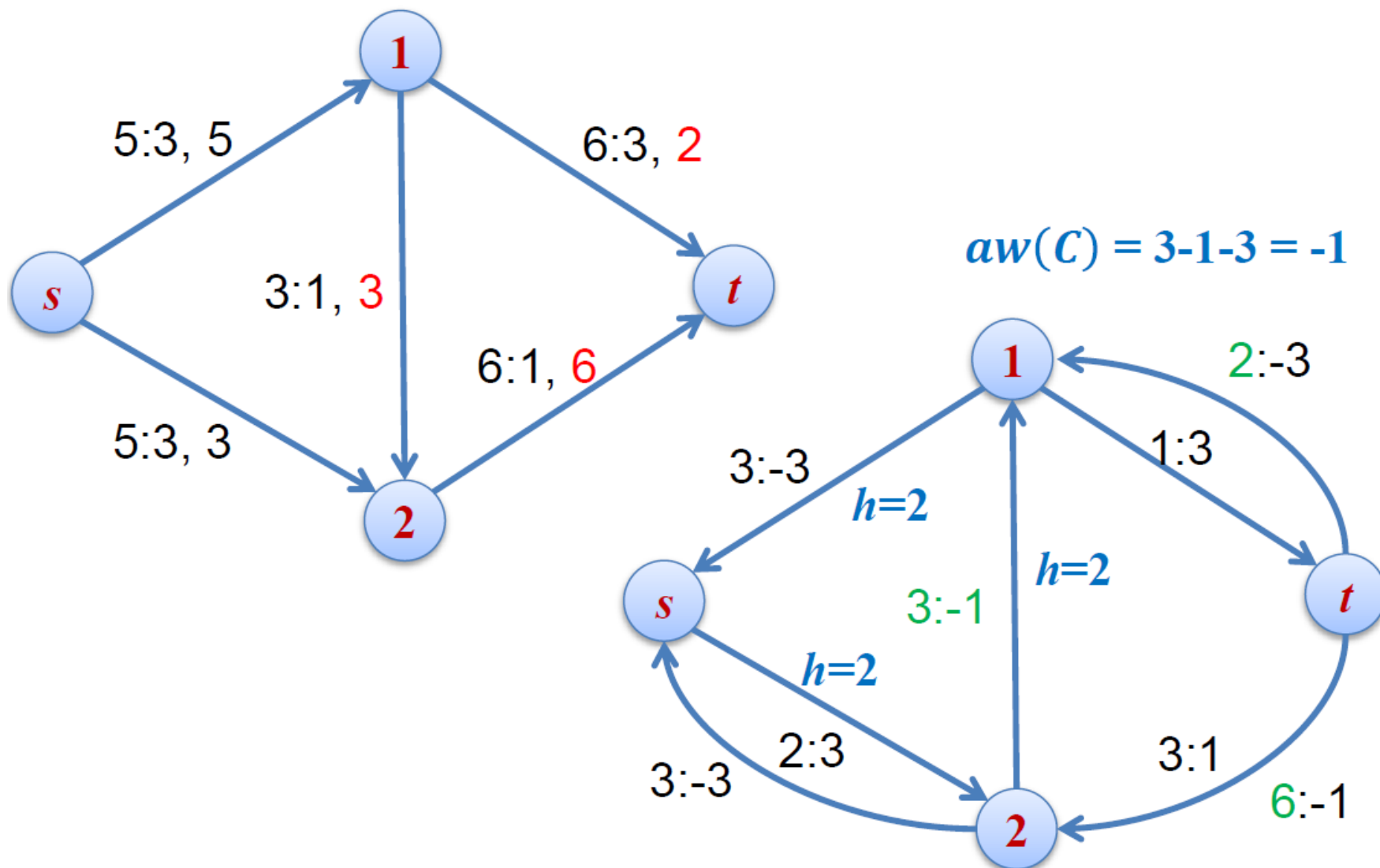
以 N 的顶点集 V 为顶点集的完全有向图 D 中只有有限条边不重的回路, 记以 αw 为权的所有负回路的权的绝对值的最小值为 w^* . 每个 $N(f)$ 都是 D 的子图, 其负回路的权的绝对值不小于 w^* .

设 f 是流量 v_0 的初始可行流, 那么 2~6 的循环至多进行 $w(f)/w^*$ 次. 因此, 算法必在有限步内终止.

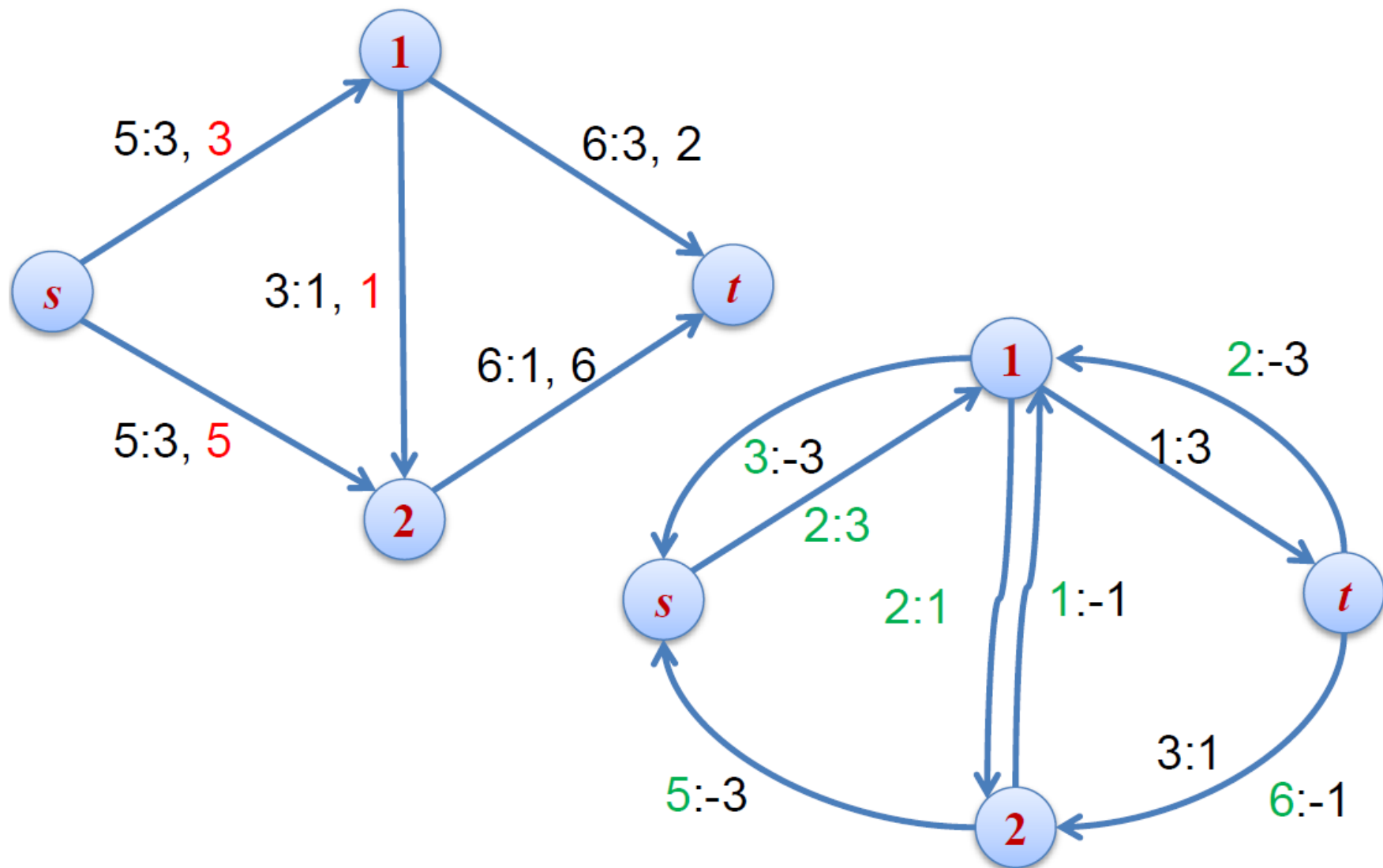
示例：负回路算法（1）



示例：负回路算法（2）



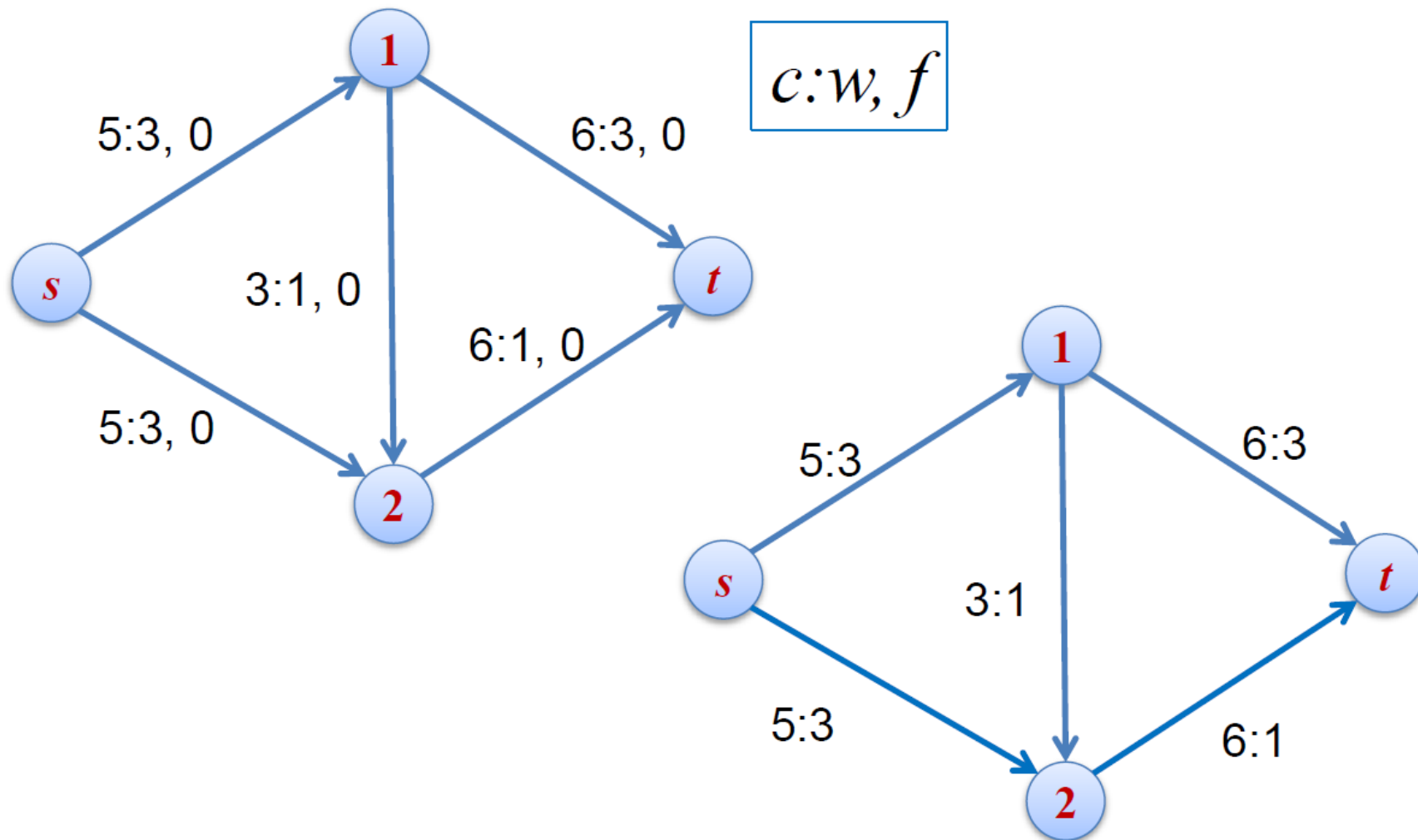
示例：负回路算法（3）



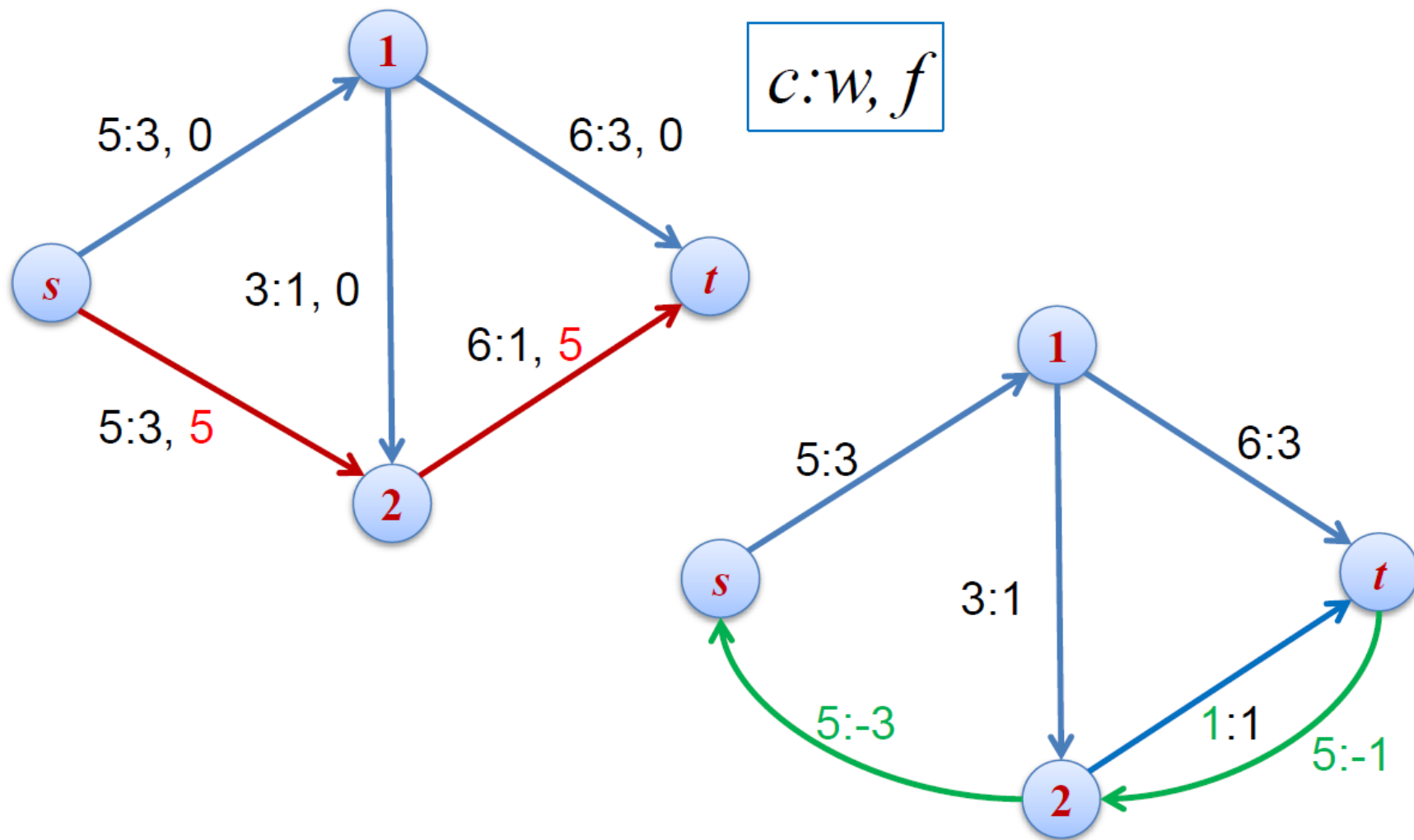
7.2.3 最小费用流的最短路径算法

算法基本想法: 从一个初始最小费用流 f (如零流) 开始, 如果 $v(f) < v_0$, 找一条费用最少的 s - t 增广链 P , 需要用Floyd算法计算 P . 修改 P 上的流量, 得到新的可行流 f' . 重复进行, 直至流量等于 v_0 为止. 关键是要证明这样得到的 f' 也是最小费用流.

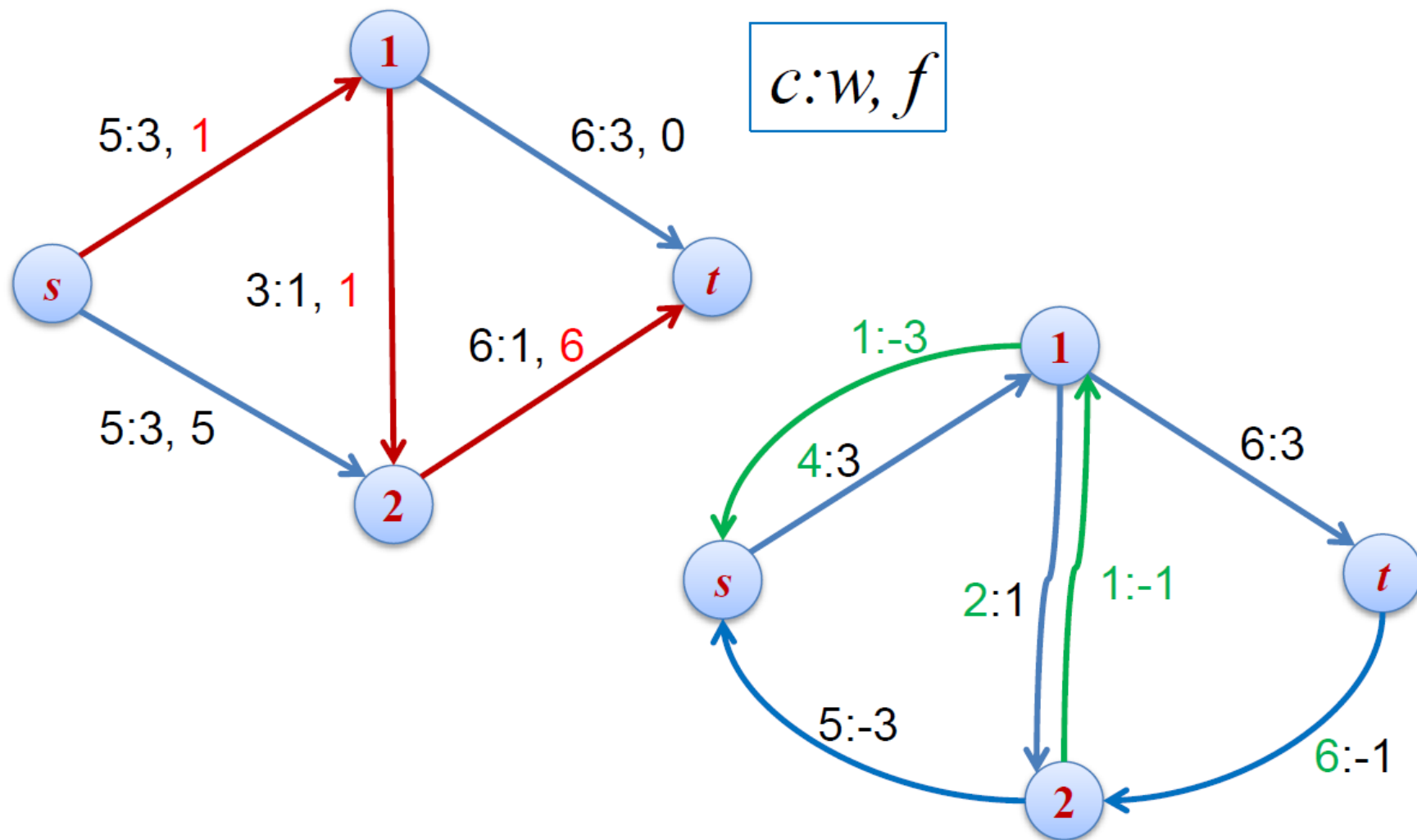
示例：求最小费用最大流



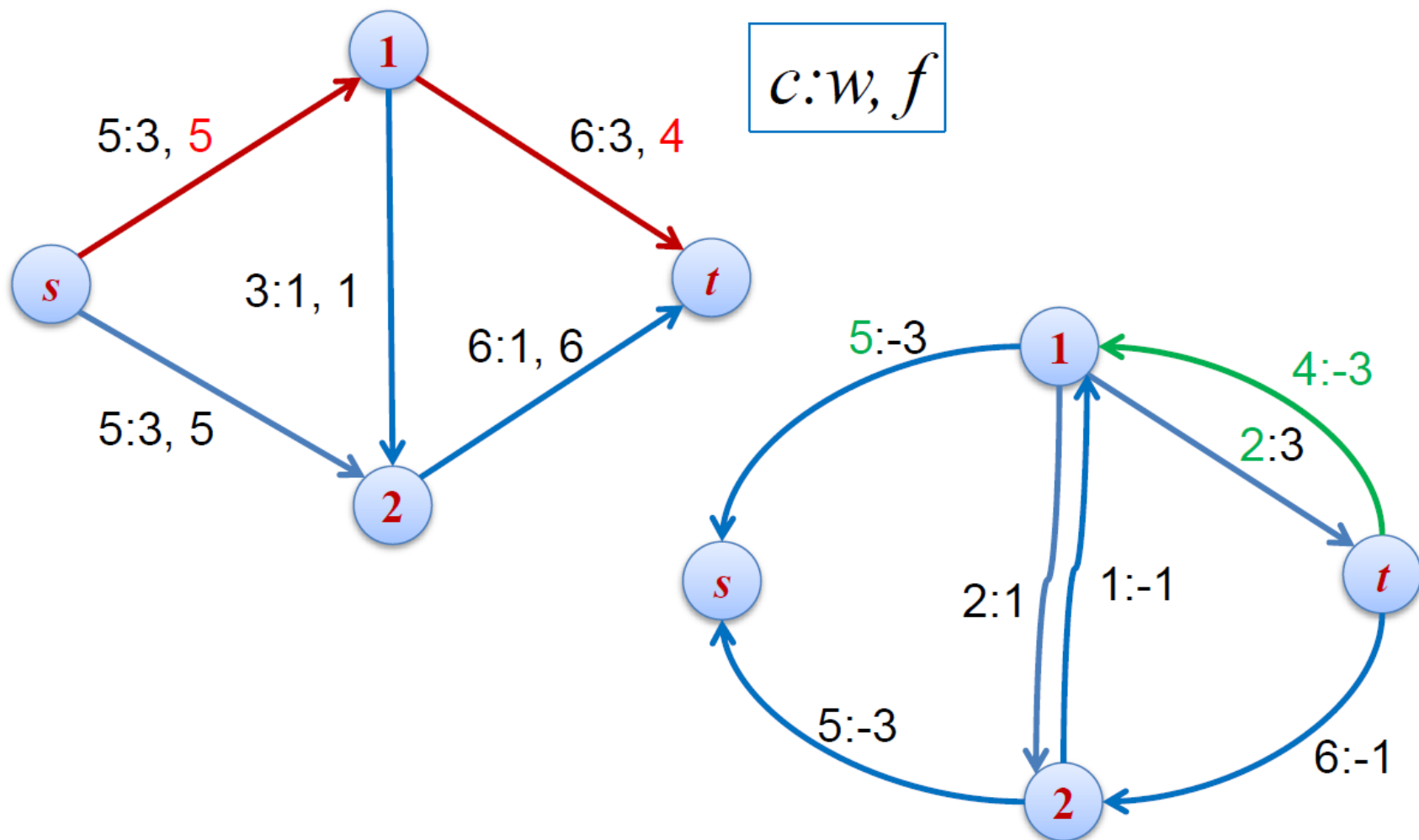
延最短路增加流再求辅助网络



再延最短路增加流并求辅助网络



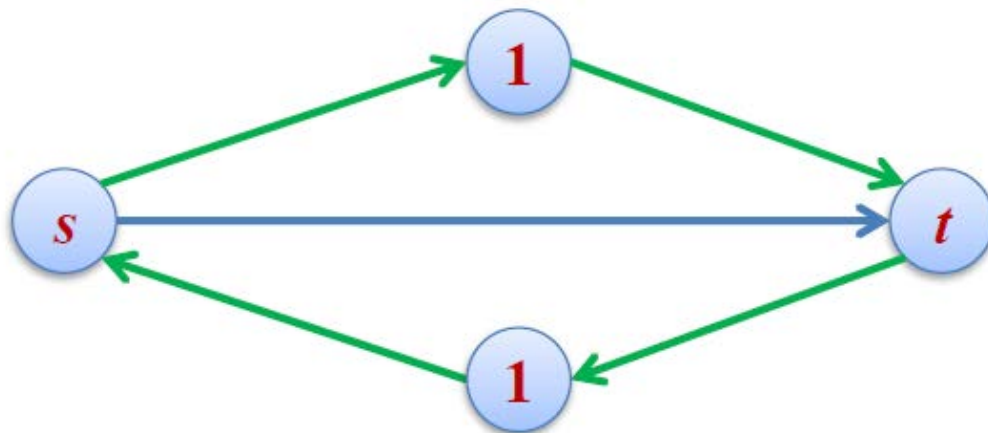
最后得到最小费用最大流



最小费用流的最短路径算法

引理7.12 设有向图 $D=<V,E>$ 没有孤立点, 顶点 s 的出度比入度大1, t 的入度比出度大1, 其余顶点的出度等于入度, 则 D 可表成一条 $s-t$ 路径与若干条回路的并.

证 从 s 开始, 沿一条边到另一个顶点. 由于除 s, t 外的顶点的入度等于出度, 到一个顶点后总可从一条没走过的边到另一个顶点, 直到 t 为止. 这样得到一条 $s-t$ 路径 P . 删去 P 和孤立点, 剩下的图中每个顶点的入度等于出度, 它由若干条回路组成.



延最短路扩张一定得到最小费用流

定理 7.6

设 f 是 N 上流量为 v_0 的最小费用流,

P 是 $N(f)$ 中权 aw 的 s - t 最短路径,

g 是 P 上流量为 θ 的可行流,

则 $f' = f + g$ 是流量为 $v_0 + \theta$ 的最小费用流。

其中对 $\forall \langle i, j \rangle \in E(f)$

$$g(i, j) = \begin{cases} \theta & \langle i, j \rangle \in E(P) \\ 0 & \text{else} \end{cases}$$

$$0 < \theta \leq \min\{ac(i, j) | \langle i, j \rangle \in E(P)\}$$

定理 7.6 证明

反正法：假设 f' 不是最小费用流，
则 $N(f')$ 中存在权 aw 的负回路 C 。

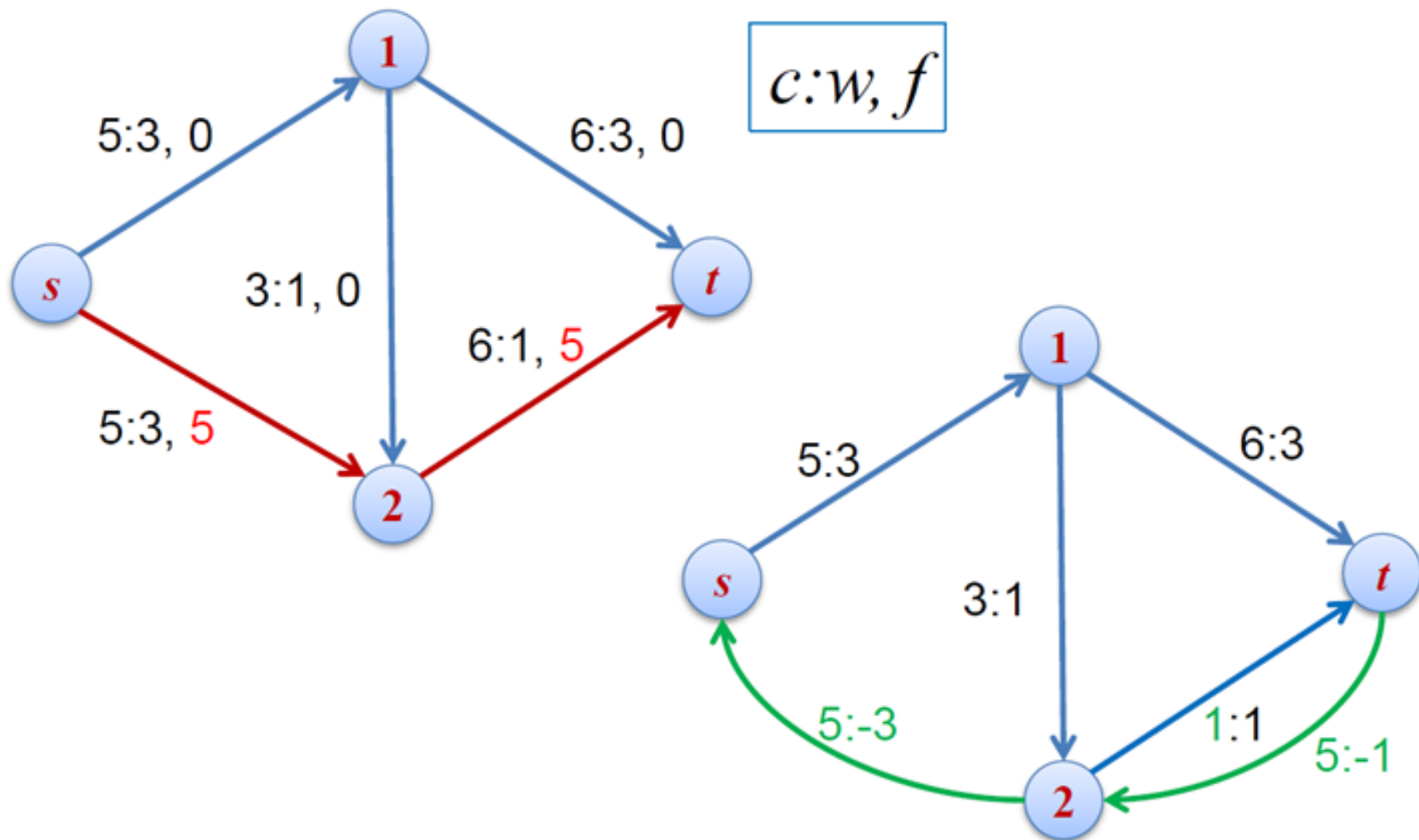
考察 $N(f)$ 与 $N(f')$ 间的差异

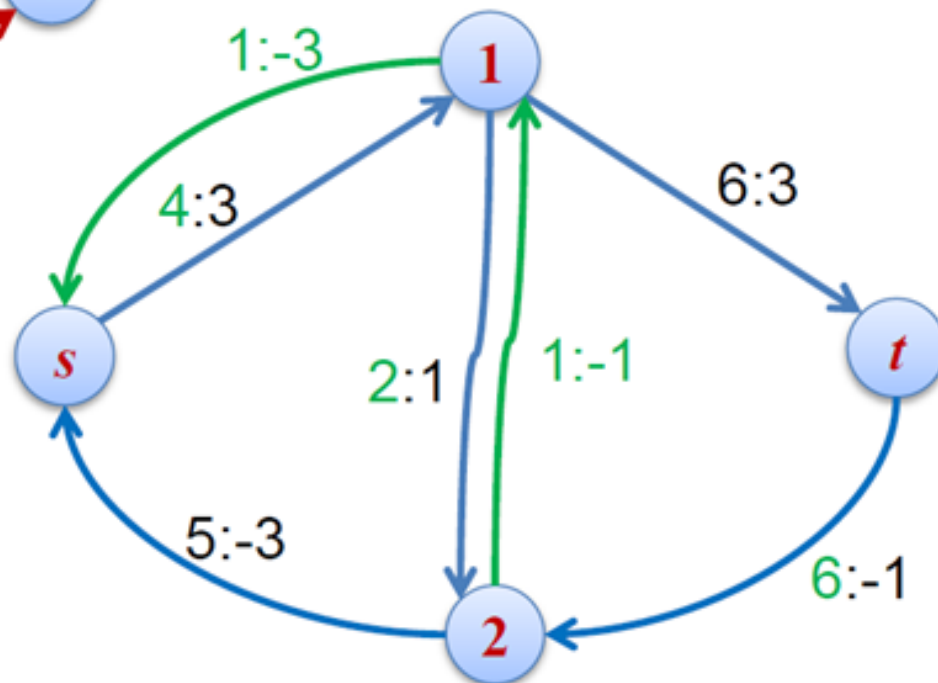
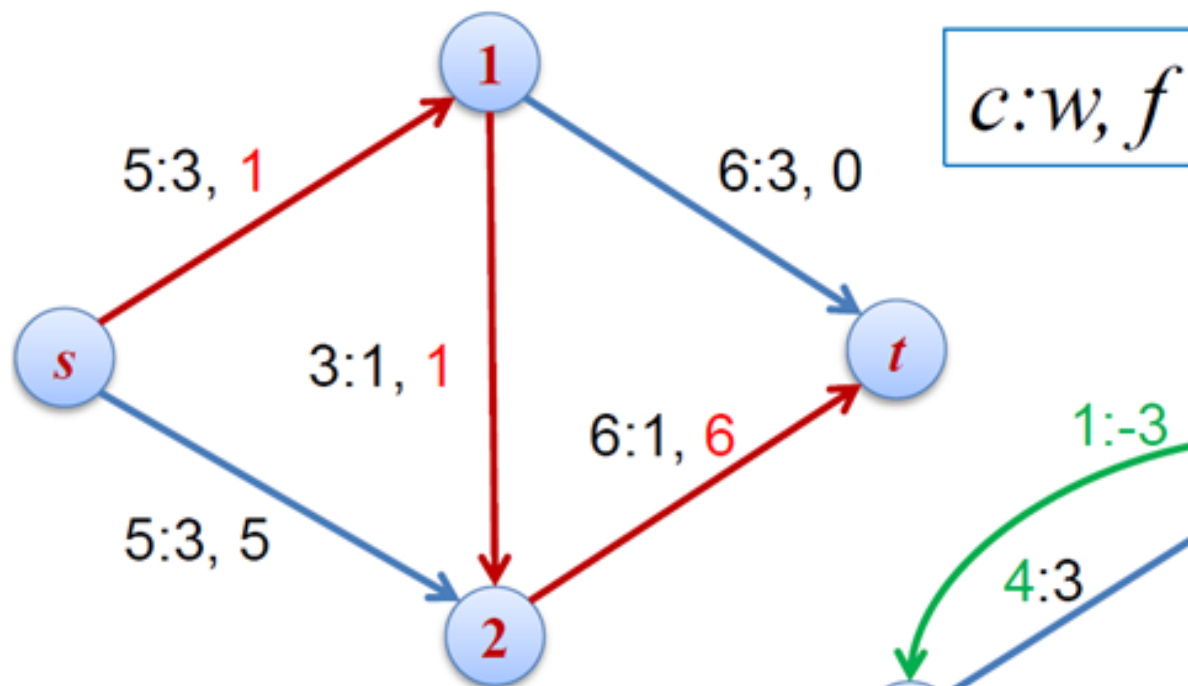
f 与 f' 仅在对 P 的增广链上不同

$E(f') - E(f)$ 定与 P 有关，即有

$$\langle i, j \rangle \in E(f') - E(f) \Rightarrow \langle j, i \rangle \in E(P)$$

回顾刚才的例子





定理 7.6 证明（续）

负回路 C 中必有 $E(f') - E(f)$ 中的边

因为 f 是最小费用流， $N(f)$ 中无负回路

设这些边为

$$\langle i_1, j_1 \rangle, \langle i_2, j_2 \rangle, \dots, \langle i_r, j_r \rangle$$

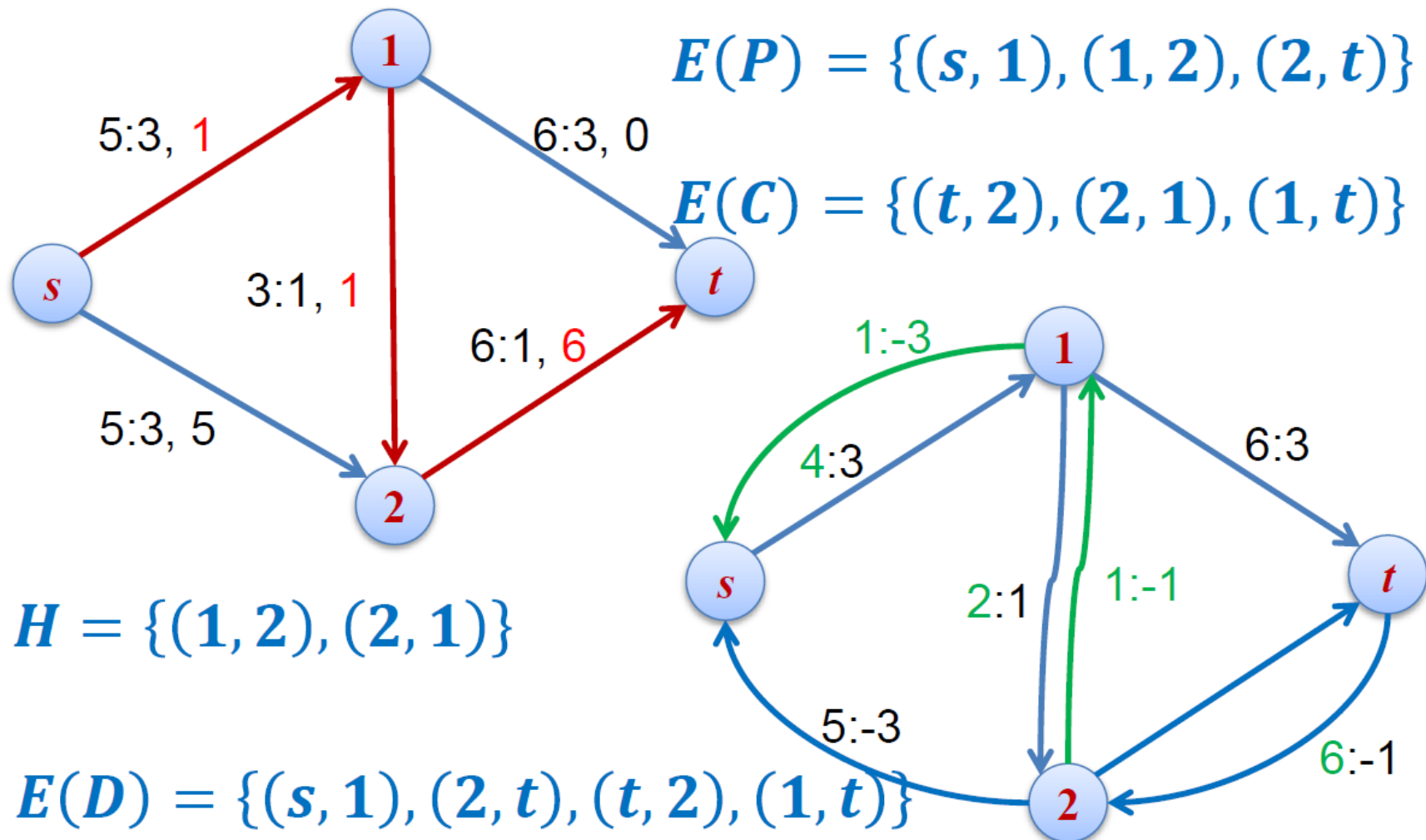
则有

$$\langle j_1, i_1 \rangle, \langle j_2, i_2 \rangle, \dots, \langle j_r, i_r \rangle \in E(P)$$

记这 $2r$ 条的边集为 H

从 P 和 C 构成的子图中删除 H （以及孤立点）
记为 D

再回顾刚才的例子



定理 7.6 证明 (续)

子图 D 满足引理7.12

它由一条 s - t 路径 P'

与若干条回路 $C_1, C_2, \dots, C_l$ 组成

于是

$$aw(D) = aw(P') + \sum_{i=1}^l aw(C_i)$$

而 $aw(H) = 0$, 于是

$$aw(P) + aw(C) = aw(D) + aw(H)$$

定理 7.6 证明 (续)

$$aw(P') = aw(P) + aw(C) - \sum_{i=1}^l aw(C_i)$$

假设条件

$$aw(C) < 0$$

而

$$aw(C_i) \geq 0, i \in \{1, 2, \dots, l\}$$

于是

$$aw(P') < aw(P)$$

与 P 是 $N(f)$ 中权 aw 的 s - t 最短路径矛盾

算法7.5 最小费用流的最短路径算法

1. $f \leftarrow 0, v \leftarrow v_0$ // 取零流作为初始最小费用流
2. 构造 $N(f)$
3. 调用 Floyd 算法计算 $N(f)$ 中以 aw 为权的 s - t 最短路径
4. if $N(f)$ 中不存在 s - t 路径 then
5. return “无流量 v_0 的可行流” // f 是最大流且 $v(f) < v_0$
6. else 设求得的最短路径为 P
7. $\theta \leftarrow \min\{v, \min\{ac(i, j) \mid \langle i, j \rangle \in E(P)\}\}$
8. for $\langle i, j \rangle \in E(P)$ do
9. if $\langle i, j \rangle \in E$ then $f(i, j) \leftarrow f(i, j) + \theta$
10. else $f(j, i) \leftarrow f(j, i) - \theta$
11. $v \leftarrow v - \theta$
12. if $v > 0$ then goto 2
13. return f

运输问题的定义

- 有 m 个产地 $A_1, A_2, \dots, A_m$,
和 n 个销地 $B_1, B_2, \dots, B_n$,
- A_i 的产量为 a_i , B_j 的销量为 b_j ,
- A_i 到 B_j 的单位运费为 w_{ij} 。
- 假设产销平衡, 即

$$\sum_{i=1}^m a_i = \sum_{j=1}^n b_j$$

- 如何制定调运方案使得总运费最少?

运输问题的线性规划描述

以 $x = \{x_{ij}\}$ 表示一个调运方案

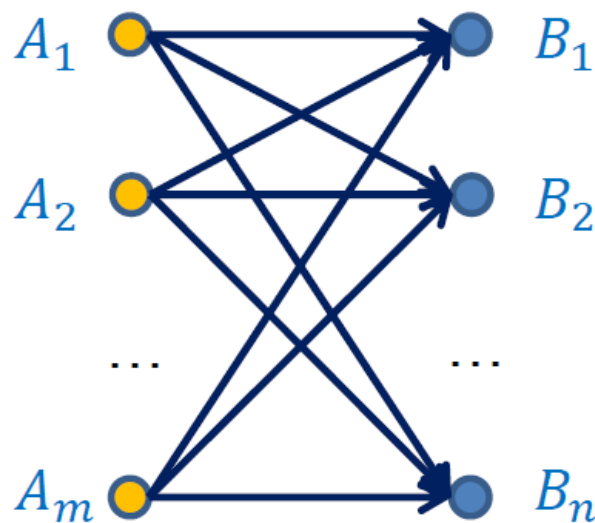
$$\min. w(x) = \sum_{i=1}^m \sum_{j=1}^n x_{ij} w_{ij}$$

st.

$$\sum_{j=1}^n x_{ij} = a_i, 1 \leq i \leq m$$

$$\sum_{i=1}^m x_{ij} = b_j, 1 \leq j \leq n$$

$$x_{ij} \geq 0, 1 \leq i \leq m, 1 \leq j \leq n$$



产销不平衡的处理

当**产大于销** ($\sum_{i=1}^m a_i > \sum_{j=1}^n b_j$) 时

引入虚拟销地 B_{n+1} , 其销量 $b_{n+1} = \sum_{i=1}^m a_i - \sum_{j=1}^n b_j$,

供给 B_{n+1} 的产品实际上是未售出的剩余产品, 其单位运费

$w_{i,n+1} = 0$ ($1 \leq i \leq m$). 问题转化成 m 个产地和 $n+1$ 个销地的运输问题, 满足产销平衡要求.

当**销大于产** ($\sum_{i=1}^m a_i < \sum_{j=1}^n b_j$) 时

引入虚拟产地 A_{m+1} , 其产量 $a_{m+1} = \sum_{j=1}^n b_j - \sum_{i=1}^m a_i$.

A_{m+1} 的供货实际上是缺货, 其单位运费 $w_{m+1,j} = 0$ ($1 \leq j \leq n$).

问题转化成 $m+1$ 个产地和 n 个销地的运输问题, 满足产销平衡要求.

模型

运输问题可看作最小费用流问题. 添加发点 s 和收点 t , 作容量-费用网络 $N=\langle V, E, c, w, s, t \rangle$, 其中

$$V=\{s, t, A_1, A_2, \dots, A_m, B_1, B_2, \dots, B_n\}$$

$$E=\{ \langle s, A_i \rangle \mid 1 \leq i \leq m \} \cup \{ \langle B_j, t \rangle \mid 1 \leq j \leq n \}$$

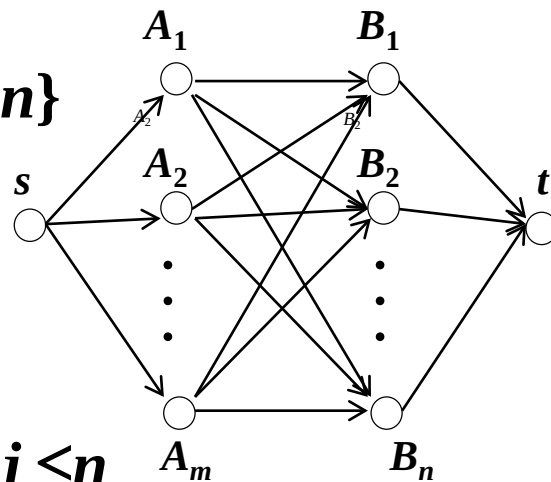
$$\cup \{ \langle A_i, B_j \rangle \mid 1 \leq i \leq m, 1 \leq j \leq n \}$$

$$c(s, A_i) = a_i, \quad w(s, A_i) = 0, \quad 1 \leq i \leq m$$

$$c(B_j, t) = b_j, \quad w(B_j, t) = 0, \quad 1 \leq j \leq n$$

$$c(A_i, B_j) = +\infty, \quad w(A_i, B_j) = w_{ij}, \quad 1 \leq i \leq m, 1 \leq j \leq n$$

流量 $v_0 = \sum_{i=1}^m a_i$ 的可行流恰好对应运输问题的调运方案. 总运费最小的调运方案就是流量 v_0 的最小费用流.



最小费用流思路求解运输问题

1. 首先类似求最大流，
确定一个对应的初始运输方案
2. 其次以初始方案为基础，
类似消减负回路的方式，
不断优化运输方案，
3. 直到不能优化为止，获得最优运输方案

7.3.1 确定初始调运方案

记 G 为以 $A=\{A_1, A_2, \dots, A_m\}$, $B=\{B_1, B_2, \dots, B_n\}$ 为互补顶点子集的赋权完全二部图, (A_i, B_j) 的权为 w_{ij} .

初始调运方案 选取边 (A_i, B_j) , 令 $x_{ij} \leftarrow \min\{a_i, b_j\}$.

若 $a_i < b_j$, 删去 A_i 及其关联边, 令 $b_j \leftarrow b_j - a_i$;

若 $a_i > b_j$, 删去 B_j 及其关联的边, 令 $a_i \leftarrow a_i - b_j$;

若 $a_i = b_j$, 则再选取一条与 A_i 关联或与 B_j 关联的边, 比如 (A_i, B_k) , $k \neq j$, 令 $x_{ik} = 0$, 删去 A_i 和 B_j 及其关联的边.

重复进行, 直到选取了 $m+n-2$ 条边. 此时只剩下一条边 (A_p, B_q) 且 $a_p = b_q$, 选取 (A_p, B_q) , 令 $x_{pq} \leftarrow a_p$. 其余边上的 x 值均为 0.

记选中的 $m+n-1$ 条边集合为 T , 称作**选中边集**.

最小元素法 每次在剩余边中选取 w_{ij} 最小的边

西北角法 每次在剩余边中选取西北角的边

调运方案的性质

命题7.2 初始调运方案选的 $m+n-1$ 条边 T 构成 G 的一棵生成树.

证 只需证 T 的导出子图(仍记作 T)连通. 对 $m+n$ 作归纳证明.
当 $m+n=2$ 时, $m=1, n=1$, 结论显然成立.

假设 $m+n \leq k$ ($k \geq 2$) 时结论成立. 考虑 $m+n=k+1$. 设选中第一条边是 (A_i, B_j) . 若 $a_i < b_j$, G 删去 A_i 及关联的边记作 G' , T 删去 A_i 和 (A_i, B_j) 记作 T' . G' 是 k 阶完全二部图, T' 是从 G' 中选中的 $k-1$ 条边. 由归纳假设, T' 是连通的. 而 (A_i, B_j) 连接 T' 和 A_i , 故 T 连通. 若 $a_i > b_j$, 类似可证 T 连通. 若 $a_i = b_j$, 设选中2条边: (A_i, B_j) , 另一条与 A_i 或与 B_j 关联, 设为 (A_i, B_k) , $k \neq j$. G 删去 A_i 和 B_j 及其关联的边记作 G' , T 删去 A_i 和 B_j 及 (A_i, B_j) 和 (A_i, B_k) 记作 T' . G' 是 $k-1$ 阶完全二部图, T' 是从 G' 中选中的 $k-2$ 条边. 由归纳假设, T' 连通. 而 (A_i, B_k) 连接 T' 和 (A_i, B_j) , 故 T 连通. 即当 $m+n=k+1$ 时结论成立.

7.3.2 改进调运方案

设 $T = \{(A_{i_r}, B_{j_r}) | 1 \leq r \leq m + n - 1\}$. 有 $m+n$ 个变量, u_i 对应 A_i , v_j 对应 B_j . 对应每一条选中边 (A_{i_r}, B_{j_r}) 有一个方程

$$u_{i_r} + v_{j_r} = w_{i_r j_r}, \quad 1 \leq r \leq m + n - 1$$

有无穷多组解. 它的一组解称作 G 的顶点的位势. 记

$$\lambda_{ij} = w_{ij} - u_i - v_j$$

是边 (A_i, B_j) 或 x_{ij} 的检验数, $1 \leq i \leq m, 1 \leq j \leq n$. 选中边的检验数为 0.

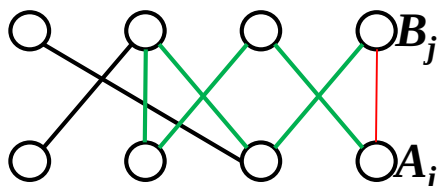
调运方案 x 对应 N 上流量 v_0 的可行流, 也记作 x . 由于离开 s 的边和进入 t 的边都是饱和边, 不在 $N(x)$ 中出现, 故 $N(x)$ 中的回路 C 在 G 中. 又, 非选中边都是零流边, 只能从 A 到 B . 选中边既是非饱和边又是非零流边, 可以从 A 到 B (前向边), 也能从 B 到 A (后向边).

改进调运方案

在 G 中对应每一条非选中边 (A_i, B_j) , 存在唯一一条由 (A_i, B_j) 和 T 的边(选中边)组成的基本回路 C_{ij} . 记 $i_0 = i, j_0 = j$.

$$C_{ij} = A_{i_0} B_{j_0} A_{i_1} B_{j_1} \cdots B_{j_p} A_{i_0}$$

$$\begin{aligned} W(C_{ij}) &= w_{i_0 j_0} - w_{i_1 j_0} + w_{i_1 j_1} - w_{i_2 j_1} + \cdots + w_{i_p j_p} - w_{i_0 j_p} \\ &= w_{i_0 j_0} - (u_{i_1} + v_{j_0}) + (u_{i_1} + v_{j_1}) - (u_{i_2} + v_{j_1}) + \cdots + (u_{i_p} + v_{j_p}) - (u_{i_0} + v_{j_p}) \\ &= w_{i_0 j_0} - u_{i_0} - v_{j_0} = \lambda_{ij} \end{aligned}$$



如果 $\lambda_{ij} < 0$, 设 δ 是 C_{ij} 中第偶数条边上 x 的最小值((A_i, B_j) 是第一条边), C_{ij} 中第奇数条边上的 x 加 δ , 第偶数条边上的 x 减 δ , 其余的 x 不变, 得到 x' .

改进调运方案

选中边变换: 设 e 是 C_{ij} 上第偶数条边中 x 值最小 (等于 δ) 的边 (若有多条, 任取一条). x' 在 e 上的值为 0. 从 T 中删去 e , 加入 (A_i, B_j) , 记作 T' . (A_i, B_j) 称作**换入边**, e 称作**换出边**, x_{ij} 是**换入变量**, e 上的 x 是**换出变量**.

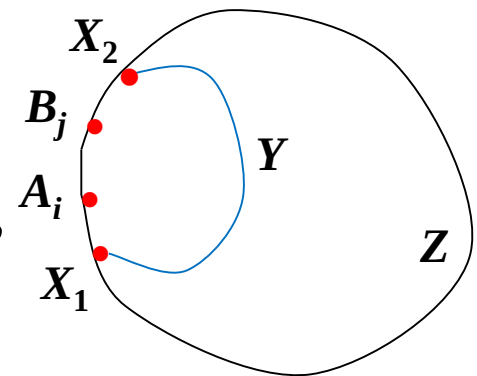
引理7.13 设对调运方案 x 和对应的选中边集 T 作以 (A_i, B_j) 为换入边的选中边变换得到 x' 和 T' , 换出变量的值为 δ , 则 x' 也是一个调运方案, 对应的选中边集为 T' , 且

$$w(x') = w(x) + \delta \cdot \lambda_{ij}$$

定理7.7 调运方案的总费用是最小的当且仅当所有检验数大于等于 0.

定理证明

证 充分性. 设所有检验数 $\lambda_{ij} \geq 0$, 只需证 G 没有权 w 的负回路. 由 $w(C_{ij}) = \lambda_{ij}$, 基本回路都不是负回路. 假设 C 是负回路, 那么 C 至少有2条非选中边, 任取其中一条 (A_i, B_j) , 对应的基本回路 $C_{ij} = X_1 \dots A_i B_j \dots X_2 \dots Y \dots X_1$, 其中 X_1 和 X_2 分别是边 (A_i, B_j) 两侧与 C 连续重合部分的最远端点, $C = X_1 \dots A_i B_j \dots X_2 \dots Z \dots X_1$. 令 $C' = X_2 \dots Z \dots X_1 \dots Y \dots X_2$, C' 中 $X_1 \dots Y \dots X_2$ 与 C_{ij} 中 $X_2 \dots Y \dots X_1$ 是方向相反的同一段, $w(C_{ij}) + w(C') = w(C)$, 推出 $w(C') \leq w(C)$. C_{ij} 中除 (A_i, B_j) 外都是选中边, 因此 C' 是比 C 少一条非选中边的负回路. 重复进行, 最后得到一个只含一条非选中边的负回路, 这是权为负的基本回路, 矛盾.



位势算法

算法7.6 位势算法

1. 求初始调运方案 x 和选中边集 T
2. 求位势 u, v
3. 计算检验数 $\lambda_{ij} = w_{ij} - u_i - v_j$
4. if $\lambda_{pq} = \min\{\lambda_{ij}\} \geq 0$ then return x
5. 以 (A_p, B_q) 为换入边作选中边集变换, 变换结果仍记作 x 和 T
6. goto 2

运输问题可以表成线性规划.

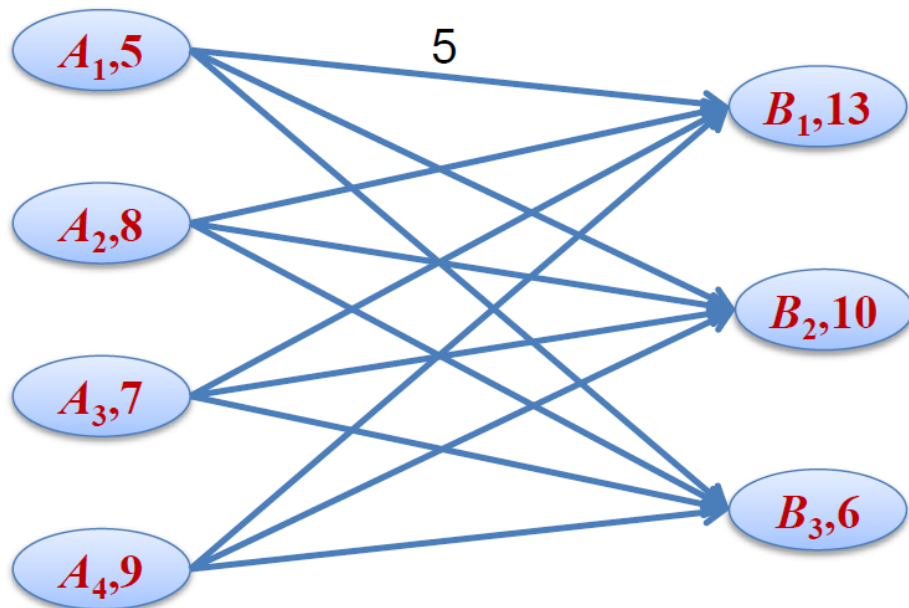
$T \leftrightarrow$ 可行基, $x \leftrightarrow$ 基本可行解,

检验数 $\leftrightarrow$ 检验数, 位势 $u, v \leftrightarrow$ 对偶变量,

选中边集变换 $\leftrightarrow$ 基变换.

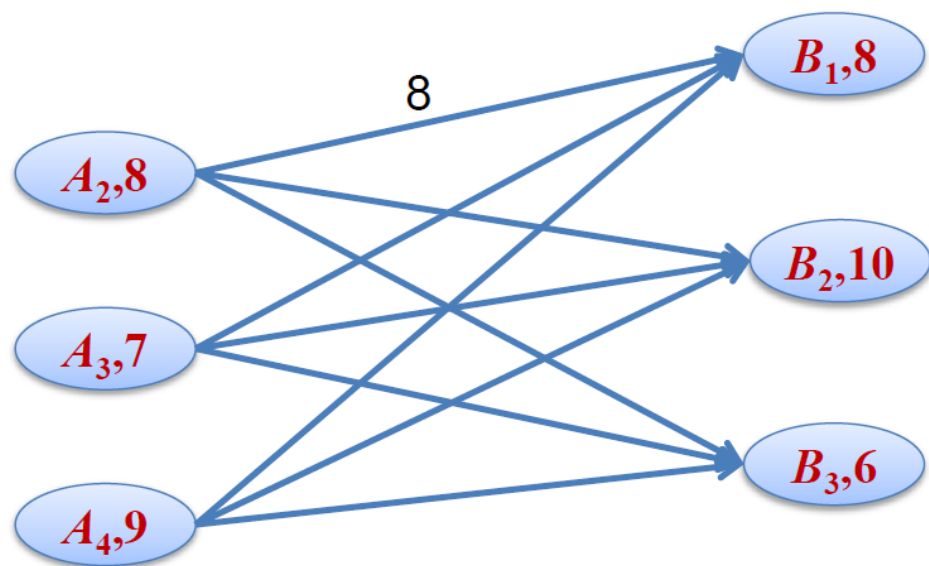
若选中边变量恒大于0 (问题非退化), 位势算法在有限步终止.

确定初始调度方案



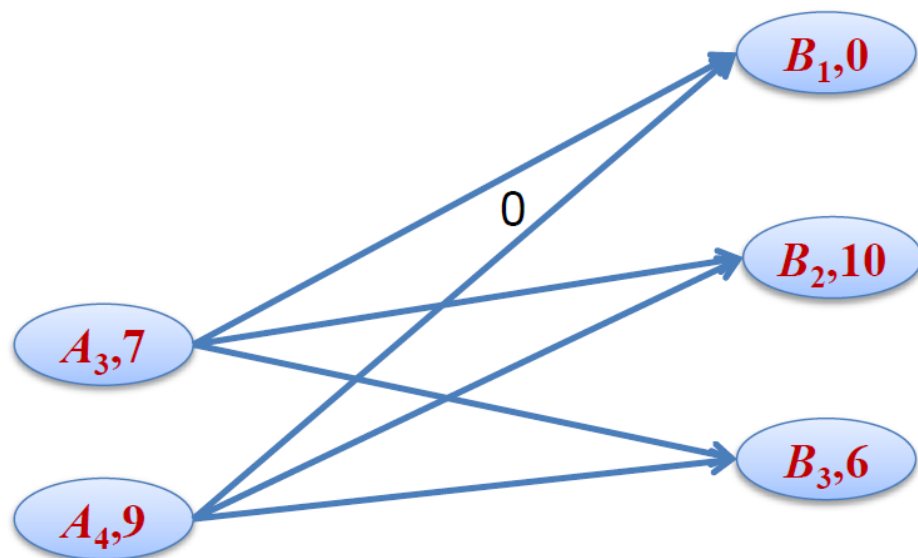
w	B_1	B_2	B_3
A_1	3	6	2
A_2	5	5	3
A_3	2	3	4
A_4	1	4	1

确定初始调度方案



w, x	B_1	B_2	B_3
A_1	3, 5	6	2
A_2	5	5	3
A_3	2	3	4
A_4	1	4	1

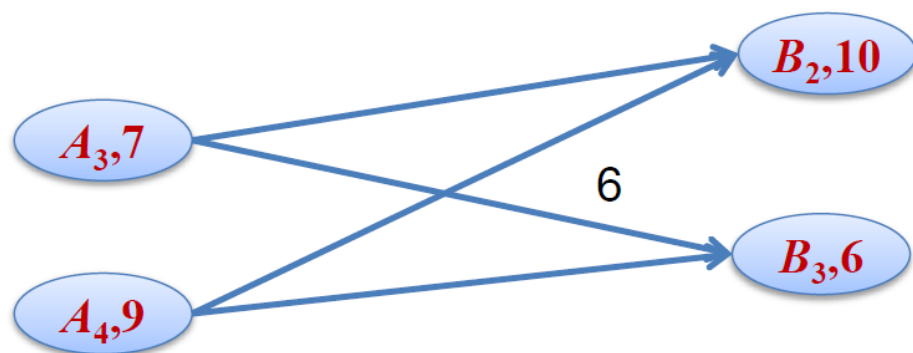
确定初始调度方案



w, x	B_1	B_2	B_3
A_1	3, 5	6	2
A_2	5, 8	5	3
A_3	2	3	4
A_4	1	4	1

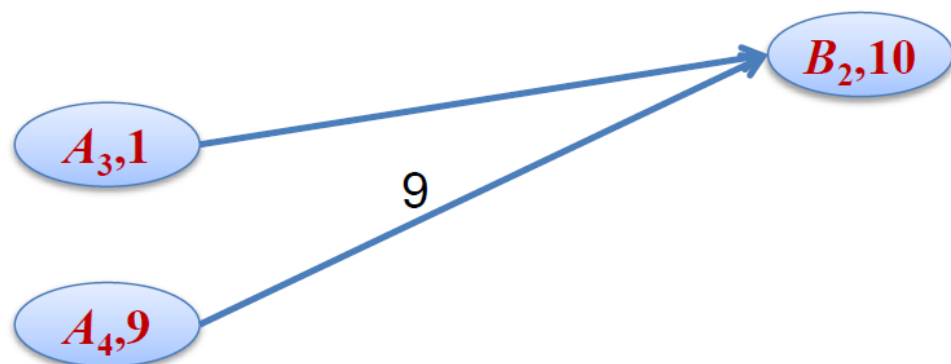
确定初始调度方案

w, x	B_1	B_2	B_3
A_1	3, 5	6	2
A_2	5, 8	5	3
A_3	2	3	4
A_4	1, 0	4	1



确定初始调度方案

w, x	B_1	B_2	B_3
A_1	3, 5	6	2
A_2	5, 8	5	3
A_3	2	3	4, 6
A_4	1, 0	4	1

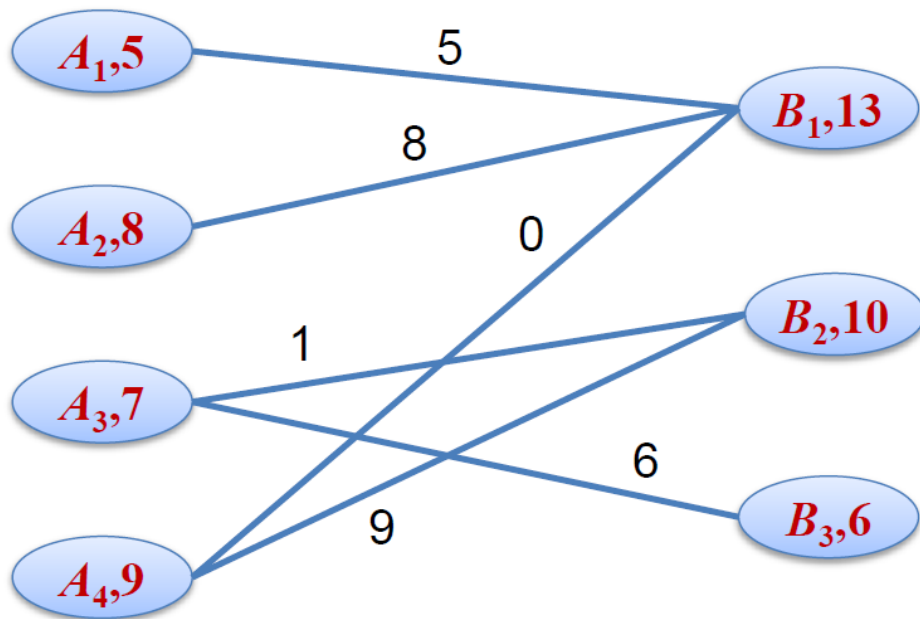


确定初始调度方案

w, x	B_1	B_2	B_3
A_1	3, 5	6	2
A_2	5, 8	5	3
A_3	2	3, 1	4, 6
A_4	1, 0	4, 9	1



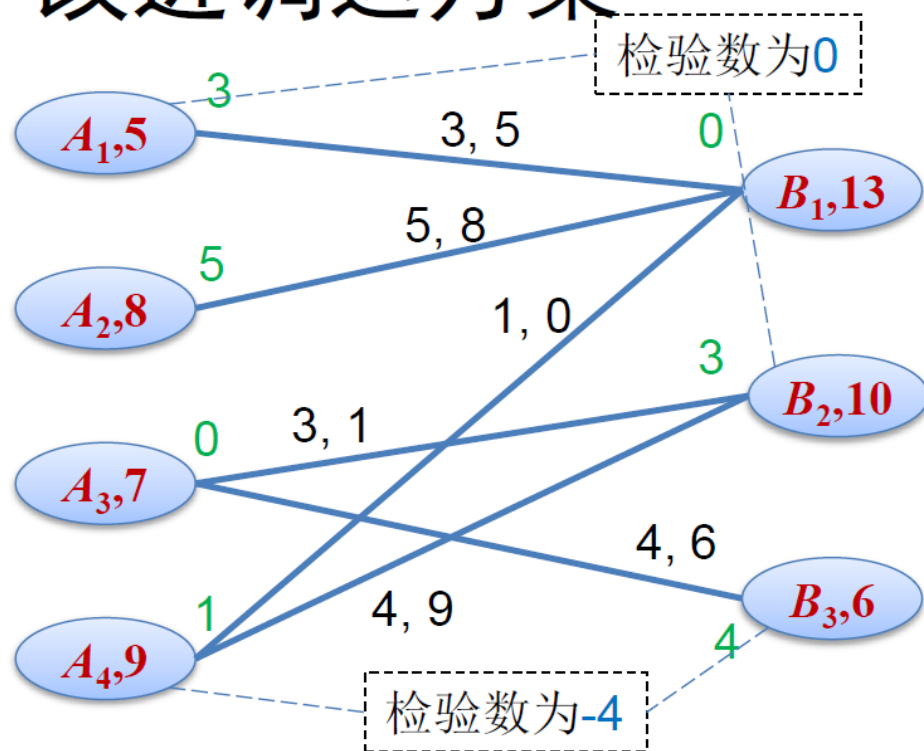
确定初始调度方案



w, x	B_1	B_2	B_3
A_1	3, 5	6, 0	2, 0
A_2	5, 8	5, 0	3, 0
A_3	2, 0	3, 1	4, 6
A_4	1, 0	4, 9	1, 0

选中边集： $m+n-1$ 条边，构成一个生成树

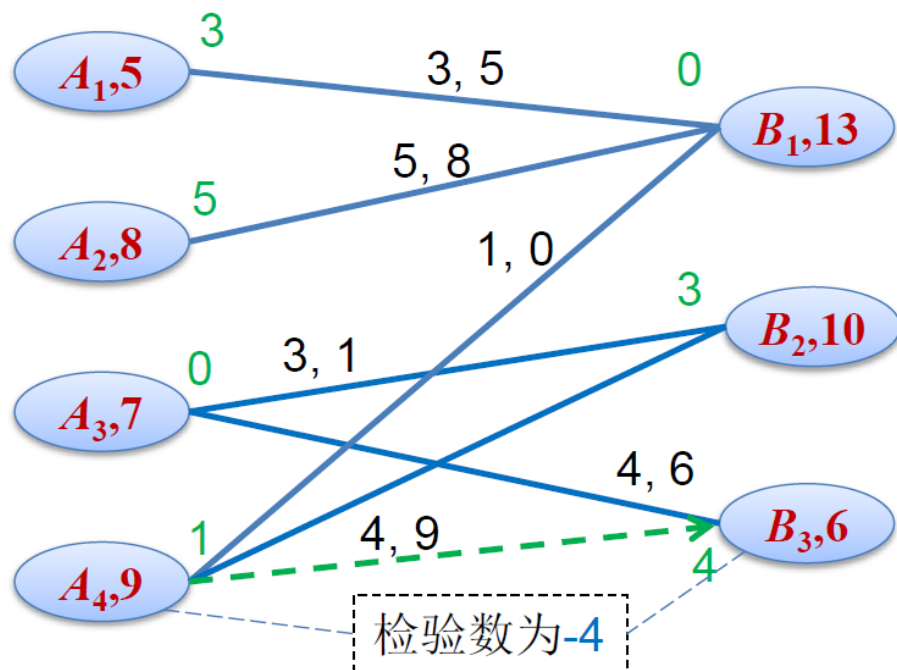
改进调运方案



w, x	B_1	B_2	B_3
A_1	3, 5	6, 0	2, 0
A_2	5, 8	5, 0	3, 0
A_3	2, 0	3, 1	4, 6
A_4	1, 0	4, 9	1, 0

- 选中边集两端点的位势之和等于边的权重值
- 边的检验数为边权重值减去边的两端点位势和

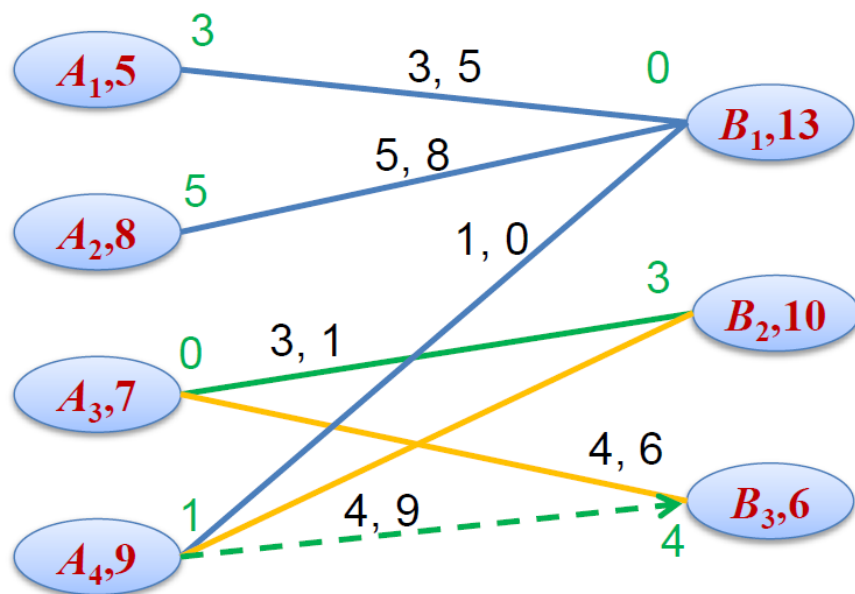
改进调运方案



w, x	B_1	B_2	B_3
A_1	3, 5	6, 0	2, 0
A_2	5, 8	5, 0	3, 0
A_3	2, 0	3, 1	4, 6
A_4	1, 0	4, 9	1, 0

以检验数小于零的一条边开始在选中边集中广搜
一定能找到一个负回路

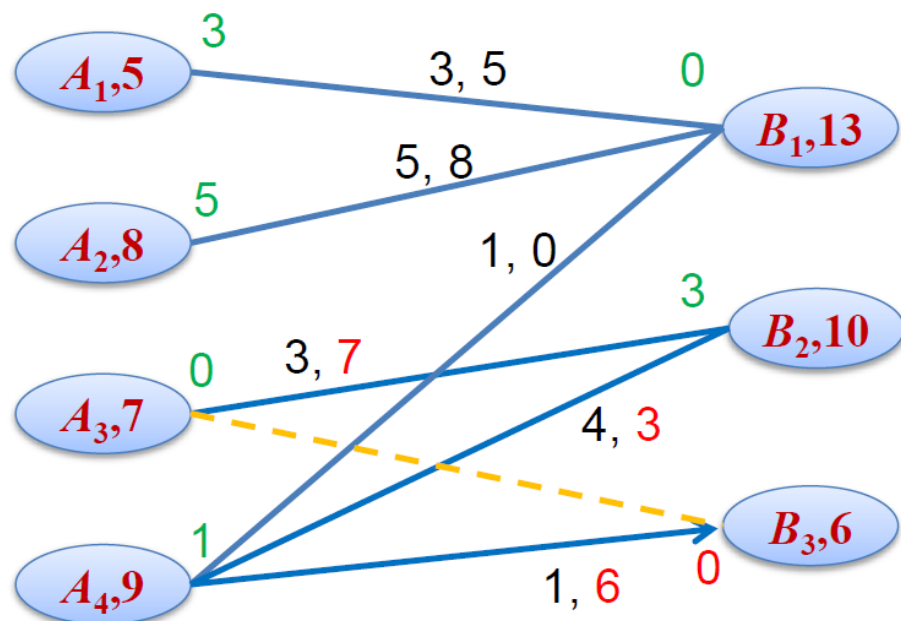
改进调运方案



w, x	B_1	B_2	B_3
A_1	3, 5	6, 0	2, 0
A_2	5, 8	5, 0	3, 0
A_3	2, 0	3, 1	4, 6
A_4	1, 0	4, 9	1, 0

绿边为正向边，费用为正；黄边为反向边，费用为负
以反向边对应的最小流量更新流，得到新的选中集

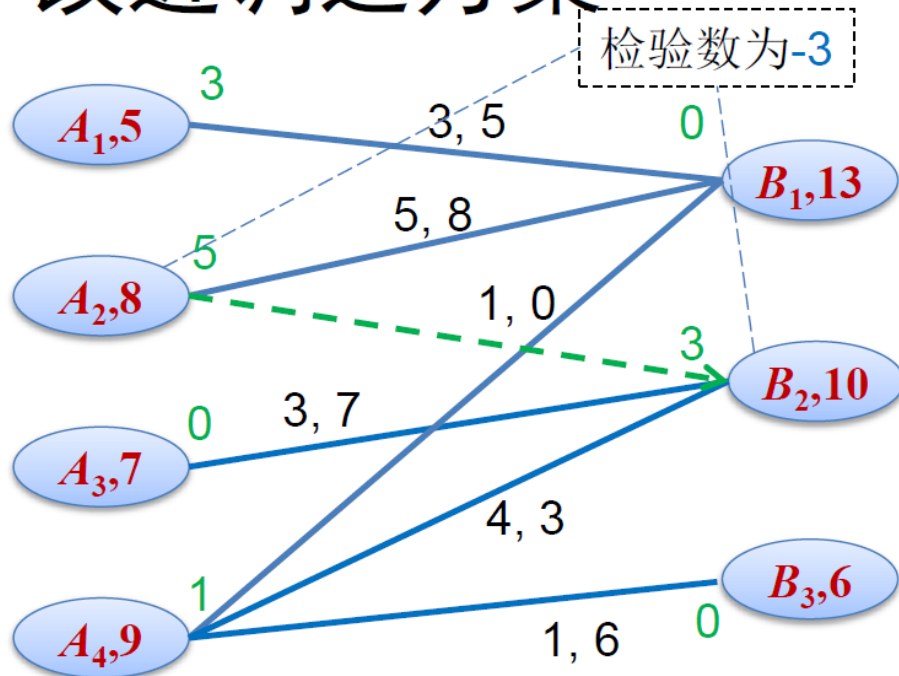
改进调运方案



w, x	B_1	B_2	B_3
A_1	3, 5	6, 0	2, 0
A_2	5, 8	5, 0	3, 0
A_3	2, 0	3, 7	4, 0
A_4	1, 0	4, 3	1, 6

更新边的流量值，删除零流边，加入新边，得到新的选中集，并适当更新顶点的位势

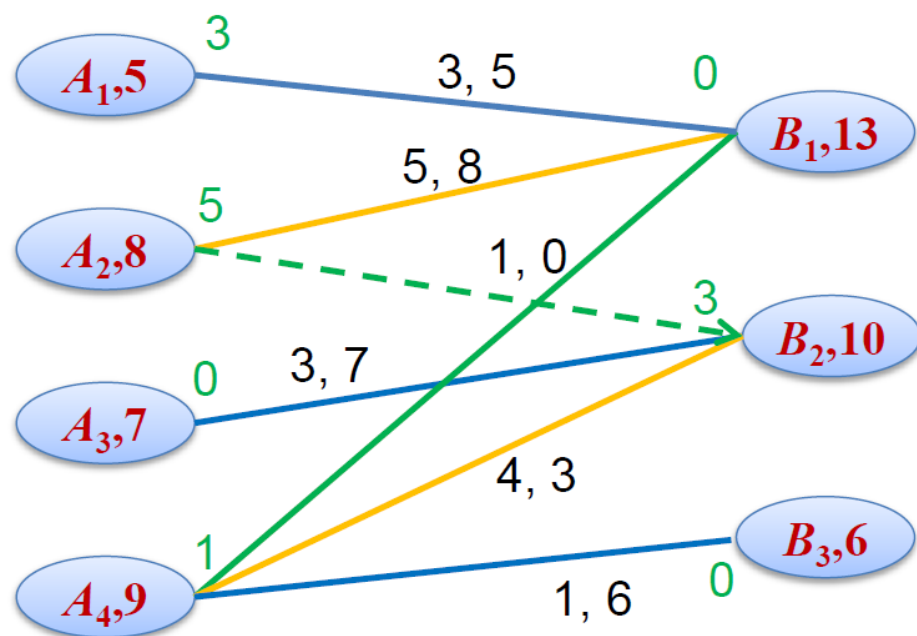
改进调运方案



w, x	B_1	B_2	B_3
A_1	3, 5	6, 0	2, 0
A_2	5, 8	5, 0	3, 0
A_3	2, 0	3, 7	4, 0
A_4	1, 0	4, 3	1, 6

以检验数小于零的一条边开始在选中边集中广搜
一定能找到一个负回路

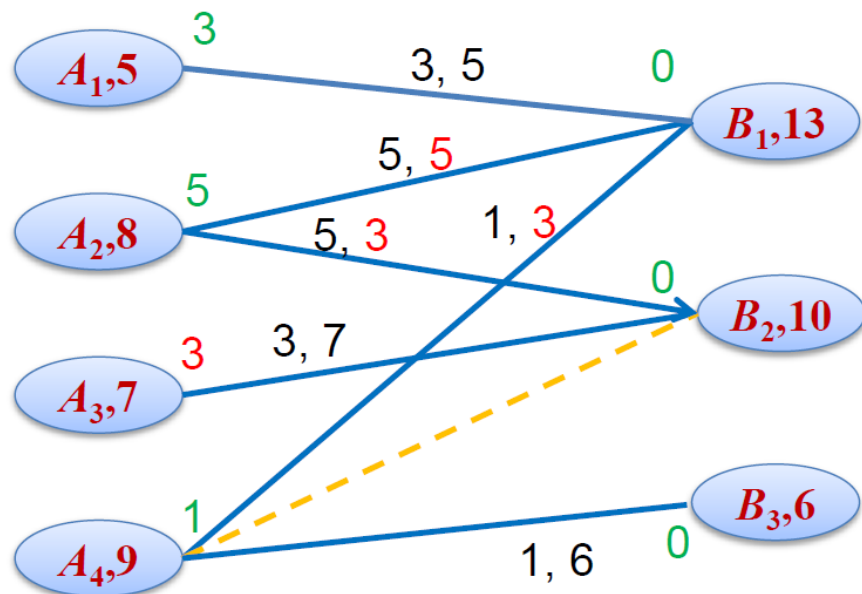
改进调运方案



w, x	B_1	B_2	B_3
A_1	3, 5	6, 0	2, 0
A_2	5, 8	5, 0	3, 0
A_3	2, 0	3, 7	4, 0
A_4	1, 0	4, 3	1, 6

绿边为正向边，费用为正；黄边为反向边，费用为负
以反向边对应的最小流量更新流，得到新的选中集

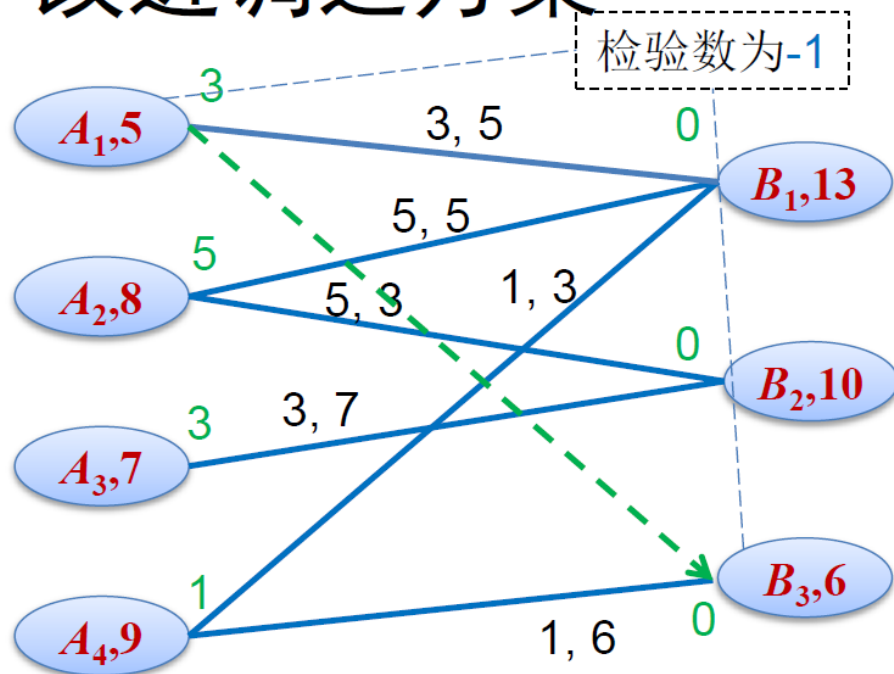
改进调运方案



w, x	B_1	B_2	B_3
A_1	3, 5	6, 0	2, 0
A_2	5, 5	5, 3	3, 0
A_3	2, 0	3, 7	4, 0
A_4	1, 3	4, 0	1, 6

更新边的流量值，删除零流边，加入新边，得到新的选中集，并适当更新顶点的位势

改进调运方案

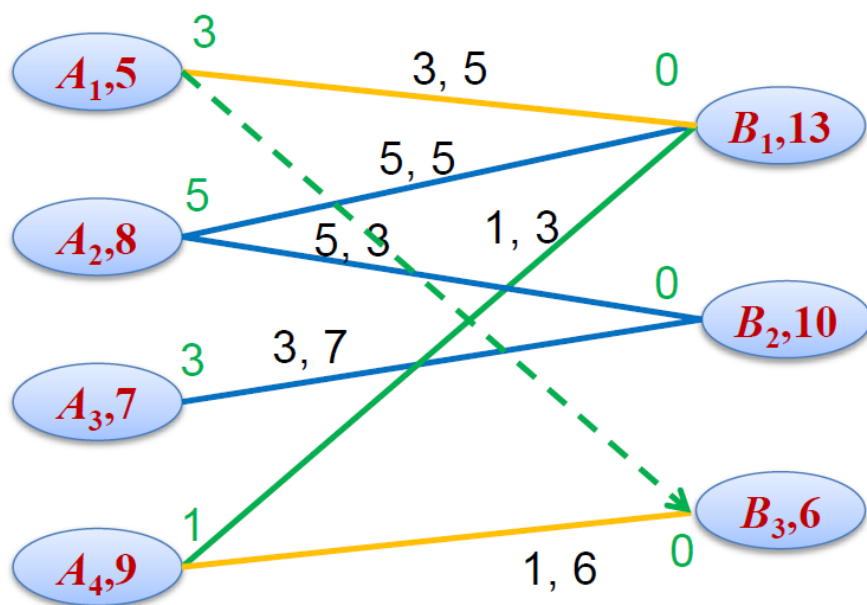


w, x	B_1	B_2	B_3
A_1	3, 5	6, 0	2, 0
A_2	5, 5	5, 3	3, 0
A_3	2, 0	3, 7	4, 0
A_4	1, 3	4, 0	1, 6

以检验数小于零的一条边开始在选中边集中广搜
一定能找到一个负回路

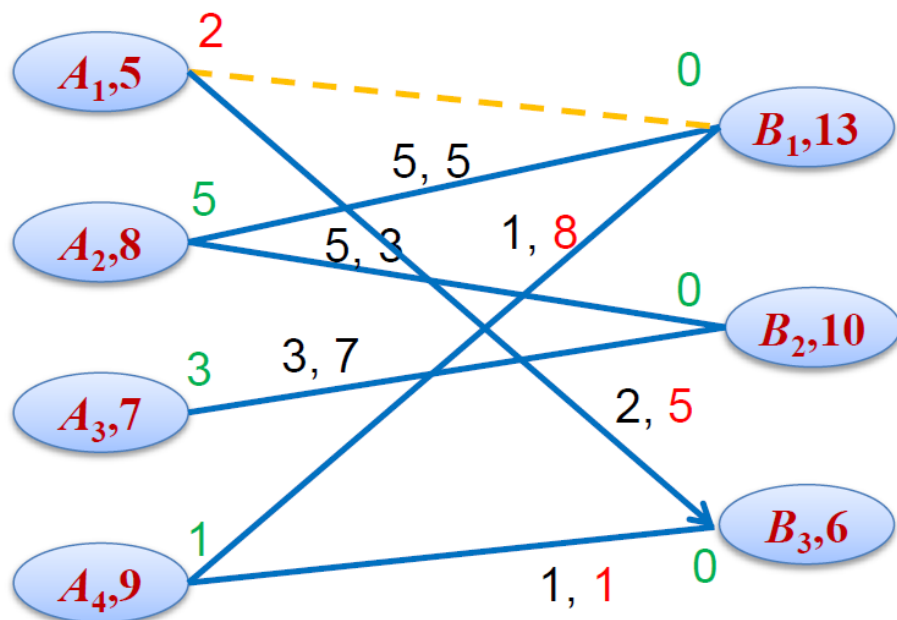
改进调运方案

w, x	B_1	B_2	B_3
A_1	3, 5	6, 0	2, 0
A_2	5, 5	5, 3	3, 0
A_3	2, 0	3, 7	4, 0
A_4	1, 3	4, 0	1, 6



绿边为正向边，费用为正；黄边为反向边，费用为负
以反向边对应的最小流量更新流，得到新的选中集

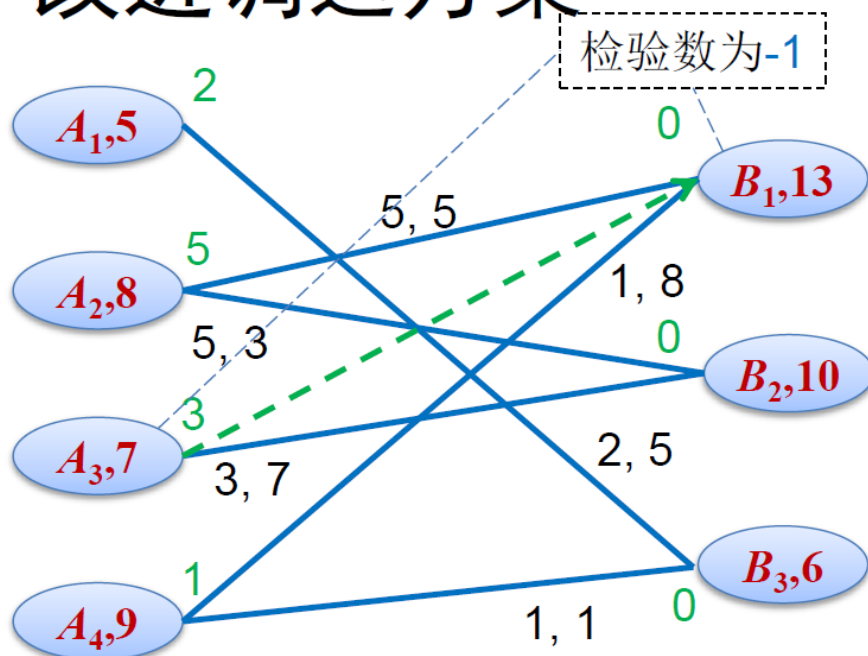
改进调运方案



w, x	B_1	B_2	B_3
A_1	3, 0	6, 0	2, 5
A_2	5, 5	5, 3	3, 0
A_3	2, 0	3, 7	4, 0
A_4	1, 8	4, 0	1, 1

更新边的流量值，删除零流边，加入新边，得到新的选中集，并适当更新顶点的位势

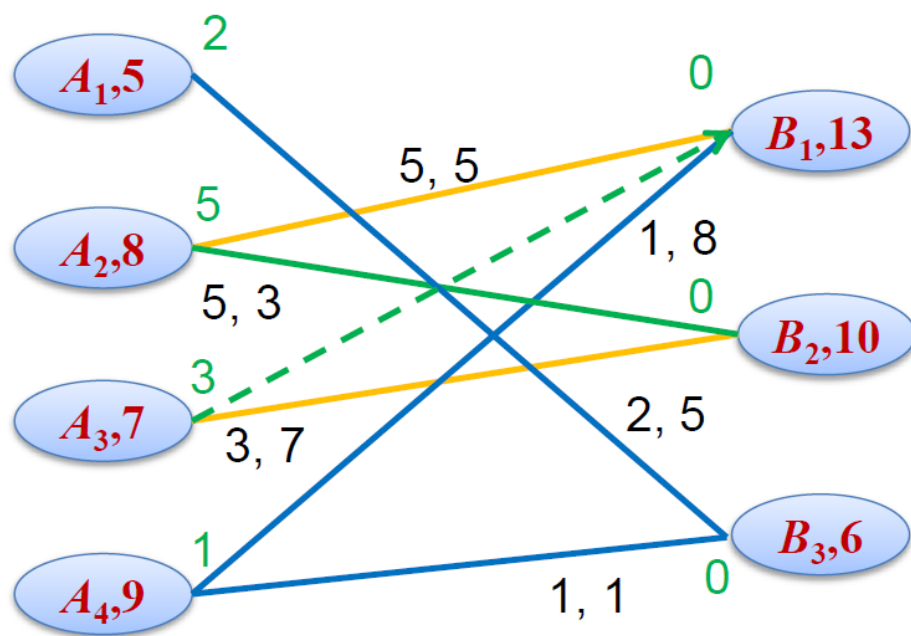
改进调运方案



w, x	B_1	B_2	B_3
A_1	3, 0	6, 0	2, 5
A_2	5, 5	5, 3	3, 0
A_3	2, 0	3, 7	4, 0
A_4	1, 8	4, 0	1, 1

以检验数小于零的一条边开始在选中边集中广搜
一定能找到一个负回路

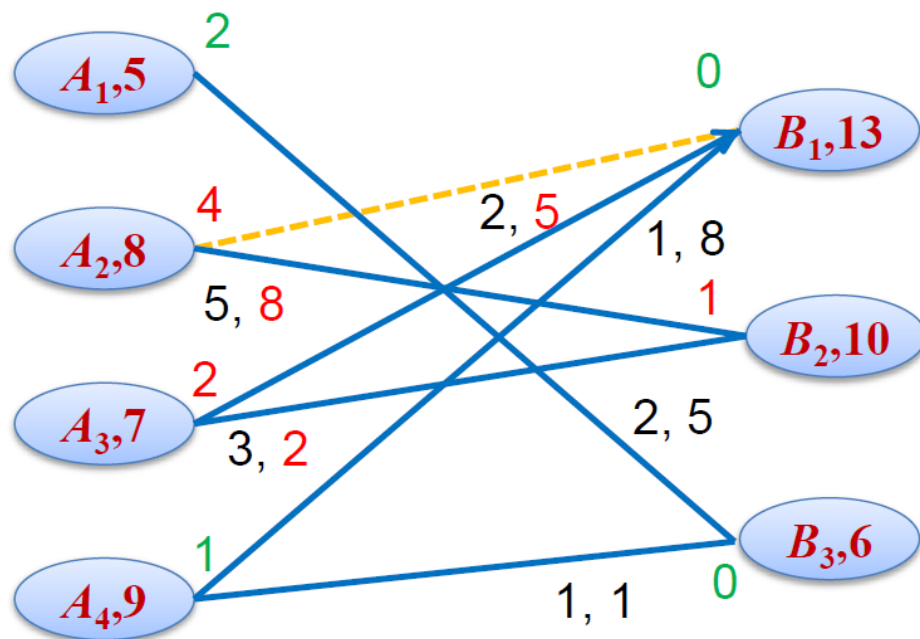
改进调运方案



w, x	B_1	B_2	B_3
A_1	3, 0	6, 0	2, 5
A_2	5, 5	5, 3	3, 0
A_3	2, 0	3, 7	4, 0
A_4	1, 8	4, 0	1, 1

绿边为正向边，费用为正；黄边为反向边，费用为负
以反向边对应的最小流量更新流，得到新的选中集

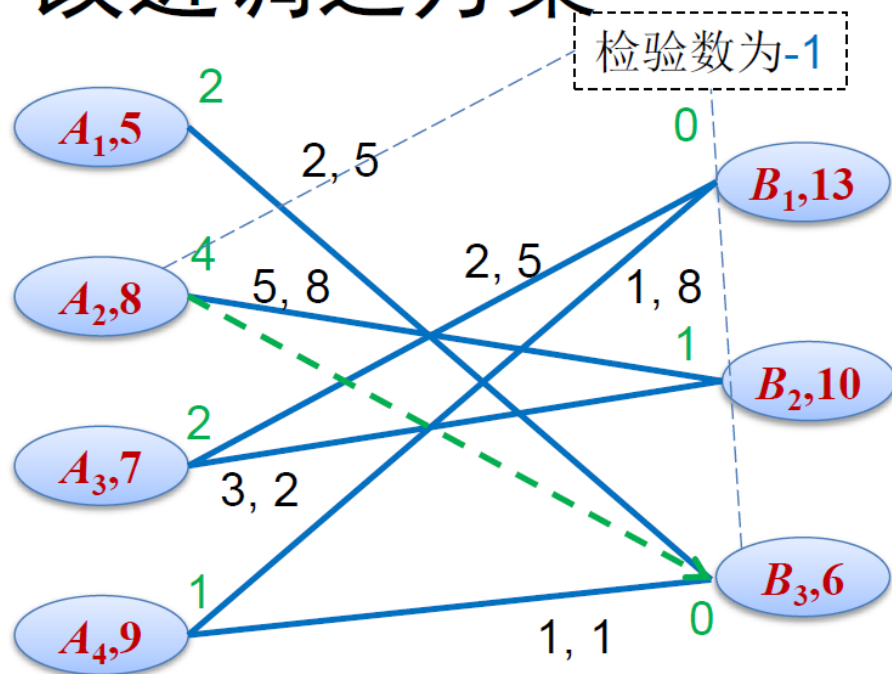
改进调运方案



w, x	B_1	B_2	B_3
A_1	3, 0	6, 0	2, 5
A_2	5, 0	5, 8	3, 0
A_3	2, 5	3, 2	4, 0
A_4	1, 8	4, 0	1, 1

更新边的流量值，删除零流边，加入新边，得到新的选中集，并适当更新顶点的位势

改进调运方案

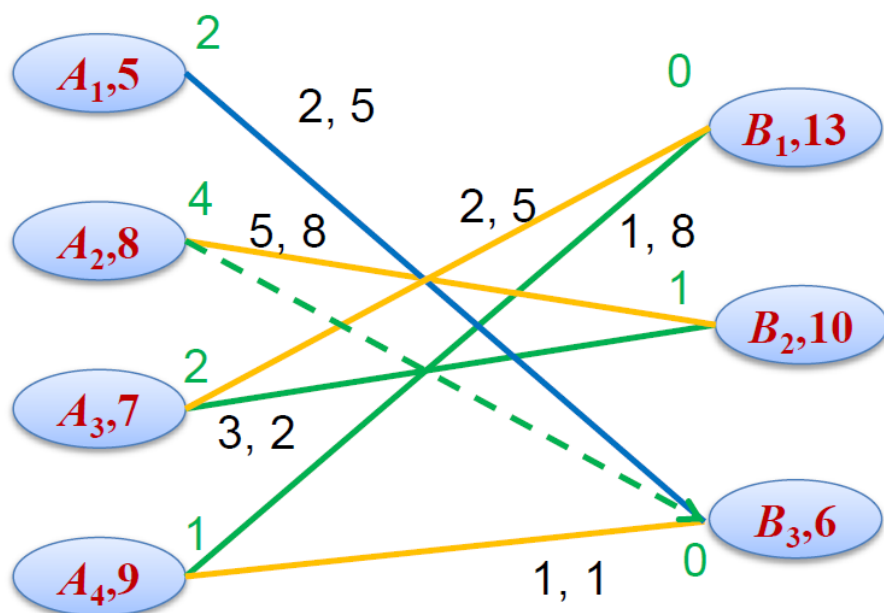


w, x	B_1	B_2	B_3
A_1	3, 0	6, 0	2, 5
A_2	5, 0	5, 8	3, 0
A_3	2, 5	3, 2	4, 0
A_4	1, 8	4, 0	1, 1

以检验数小于零的一条边开始在选中边集中广搜
一定能找到一个负回路

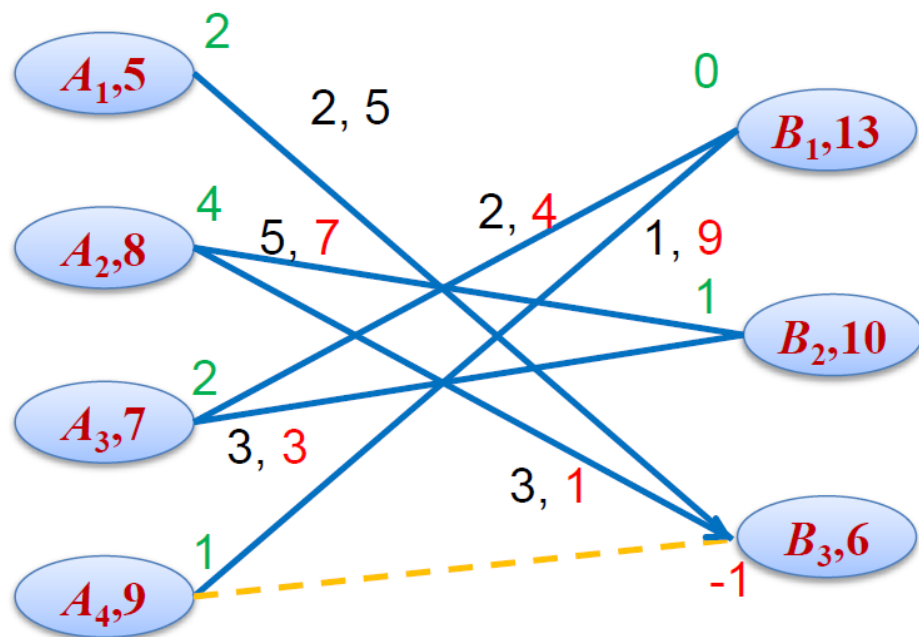
改进调运方案

w, x	B_1	B_2	B_3
A_1	3, 0	6, 0	2, 5
A_2	5, 0	5, 8	3, 0
A_3	2, 5	3, 2	4, 0
A_4	1, 8	4, 0	1, 1



绿边为正向边，费用为正；黄边为反向边，费用为负
以反向边对应的最小流量更新流，得到新的选中集

改进调运方案

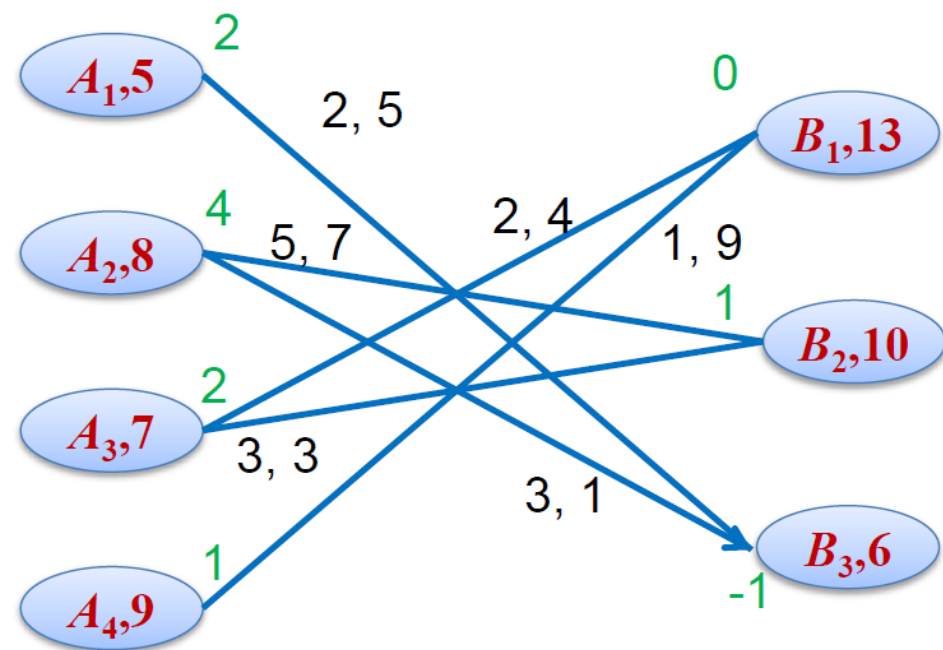


w, x	B_1	B_2	B_3
A_1	3, 0	6, 0	2, 5
A_2	5, 0	5, 7	3, 1
A_3	2, 4	3, 3	4, 0
A_4	1, 9	4, 0	1, 0

更新边的流量值，删除零流边，加入新边，得到新的选中集，并适当更新顶点的位势

改进调运方案

w, x	B_1	B_2	B_3
A_1	3, 0	6, 0	2, 5
A_2	5, 0	5, 7	3, 1
A_3	2, 4	3, 3	4, 0
A_4	1, 9	4, 0	1, 0



不再有检验数小于零的边，算法结束

7.3.3

表上作业法

初始调运方案 $x^{(0)}$ 和位势

	B_1	B_2	B_3	产量 a_i	u_i
A_1	1 3 ③	4 2 ①	7 7	5 1	2
A_2	5 7 ④	5 5	1 2 ②	6 5	6
销量 b_j	6 5	4	1	11	
v_j	1	0	-4		

$x^{(0)}$ 的检验数和回路

	B_1	B_2	B_3
A_1	1 0	4 0	9 9
A_2	5 0	-1 -1	1 0

$$\min z = 3x_{11} + 2x_{12} + 7x_{13} + 7x_{21}$$

$$+ 5x_{22} + 2x_{23}$$

$$\text{s.t. } x_{11} + x_{12} + x_{13} = 5000$$

$$x_{21} + x_{22} + x_{23} = 6000$$

$$x_{11} + x_{21} = 6000$$

$$x_{12} + x_{22} = 4000$$

$$x_{13} + x_{23} = 1000$$

例
6.3

例6.3 (续)

调运方案 $x^{(1)}$ 和位势

	B_1	B_2	B_3	产量 a_i	u_i
A_1	5 3	2	7	5	2
A_2	1 7	4 5	1 2	6	6
v_j	1	-1	-4		

$x^{(1)}$ 的检验数

	B_1	B_2	B_3
A_1	1 0	4 1	9
A_2	5 0	0	1 0

$x^{(1)}$ 是最优解

7.4 二部图匹配

7.4.1 二部图的最大匹配

定义7.10 设简单二部图 $G = \langle A, B, E \rangle$, $M \subseteq E$, 如果 M 中任意两条边都不相邻, 则称 M 是 G 的**匹配**. 边数最多的匹配称作**最大匹配**. 当 $|A|=|B|=n$ 时, 边数为 n 的匹配称作**完美匹配**.

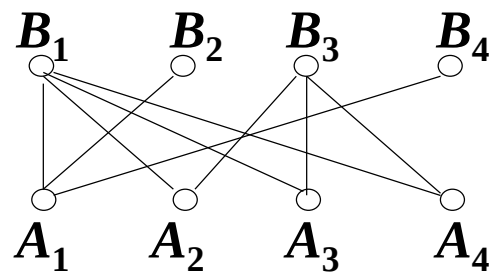
例7.5 有4名新入学的硕士生和4位导师, A_1 申请 B_1, B_2 或 B_4 作导师, A_2, A_3 和 A_4 都申请 B_1 或 B_3 作导师. 每名硕士生有一位导师, 每位导师只收一名新生. 如何分配才能尽可能满足学生要求.

作二部图 $G = \langle A, B, E \rangle$, 其中

$$A = \{ A_i \mid 1 \leq i \leq 4 \}$$

$$B = \{ B_j \mid 1 \leq j \leq 4 \}$$

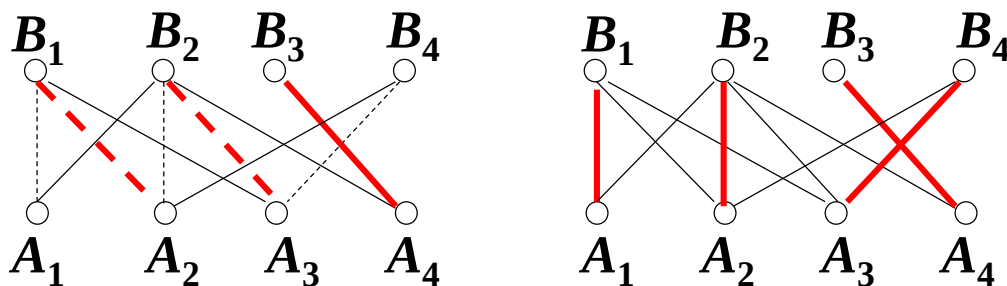
$$E = \{ (A_i, B_j) \mid A_i \text{ 申请 } B_j \text{ 作导师}, 1 \leq i \leq 4, 1 \leq j \leq 4 \}$$



增广交错路径

定义7.11 设 M 是二部图 G 的匹配, 称 M 中的边为**匹配边**, 不属于 M 的边为**非匹配边**, 与匹配边关联的顶点为**饱和点**, 不与匹配边关联的顶点为**非饱和点**. G 中由匹配边和非匹配边交替构成的路径称为**交错路径**, 起点和终点都是非饱和点的交错路径称为**增广交错路径**.

引理7.14 设 M 是二部图 G 的一个匹配, P 是一条关于 M 的增广交错路径, 则 $M' = M \oplus E(P)$ 是一个匹配且 $|M'| = |M| + 1$.



定理7.8 二部图的匹配是最大匹配当且仅当不存在关于它的增广交错路径.

匈牙利算法

匈牙利算法设计思想: 从初始匹配 M 开始, 每次找一条增广交错路径 P , 令 $M \leftarrow M \oplus E(P)$, 直到不存在增广交错路径为止.

1. 设当前匹配为 M , 对 A 中每一个非饱和点 A_i 加标号 $l(A_i) = 0$, A_i 为已标号未检查
2. 对任意已标号未检查的顶点 A_i , 把所有相邻但未标号的 B_j 标号为 $l(B_j) = A_i$
3. 如果 B_j 是非饱和点, 则找到增广交错路径, 修改匹配, 重新开始标号
4. 如果 B_j 是饱和点, 则设 $(A_k, B_j) \in M$, 令 $l(A_k) = B_j$, A_k 成为已标号未检查, A_i 称为已标号已检查
5. 重复算法, 直到 A 中不存在已标号未检查顶点

匈牙利算法

match存储 M , $\text{match}(A_i)=B_j$ 和 $\text{match}(B_j)=A_i$ 表示 $(A_i, B_j) \in M$.
 $\text{match}(A_i)=0$ ($\text{match}(B_j)=0$) 表示 A_i (B_j) 是非饱和点.

算法7.7 匈牙利算法

输入 二部图 $G=\langle A, B, E \rangle$

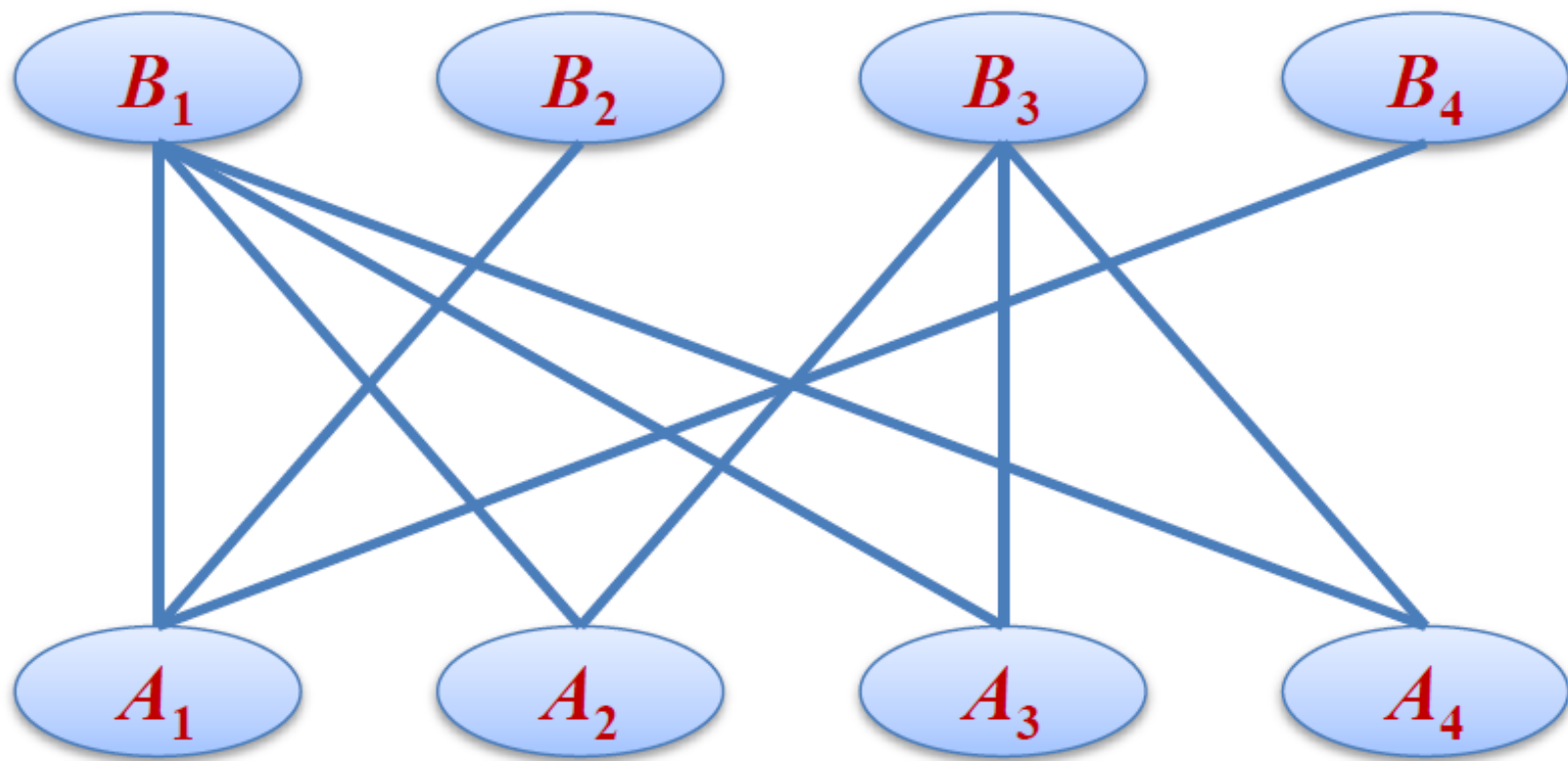
输出 match

1. for $x \in A \cup B$ do $\text{match}(x) \leftarrow 0$
2. $X \leftarrow \emptyset$ //标号开始, X 是 A 中已标号未检查顶点集
 - 2.1 for $A_i \in A \wedge \text{match}(A_i)=0$ do
 - 2.2 $l(A_i) \leftarrow 0, X \leftarrow X \cup \{A_i\}$ // l 是标号
 - 2.3 $Y \leftarrow B$ // Y 是 B 中未标号顶点集

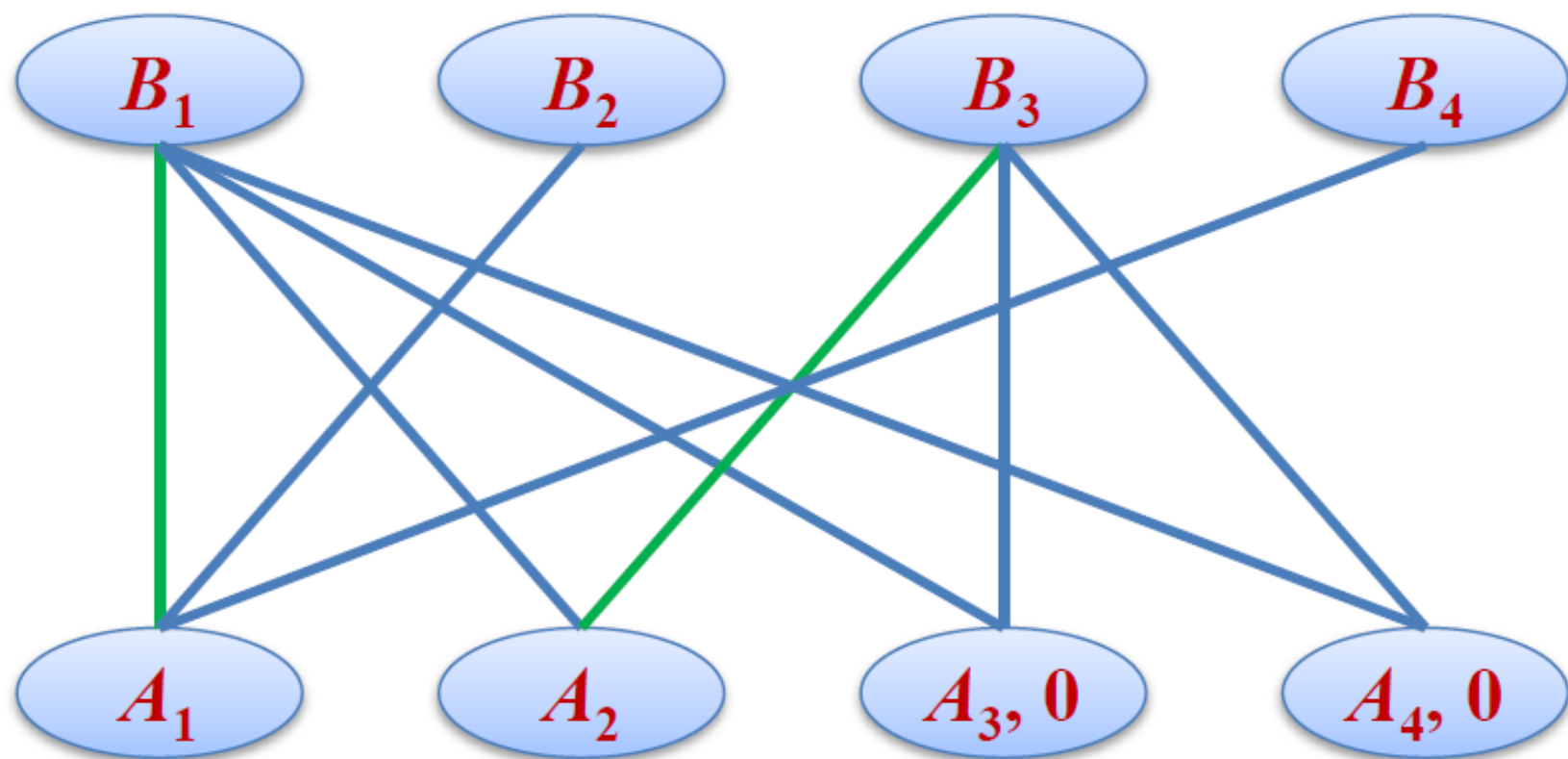
匈牙利算法（续）

3. while $X \neq \emptyset$ do
 - 3.1 任取 $A_i \in X$, $X \leftarrow X - \{A_i\}$
 - 3.2 for $(A_i, B_j) \in E \wedge B_j \in Y$ do
 - 3.3 $l(B_j) \leftarrow A_i$, $Y \leftarrow Y - \{B_j\}$
 - 3.4 if $\text{match}(B_j) = 0$ then goto 5 //存在增广交错路径 P
 - 3.5 else $A_k \leftarrow \text{match}(B_j)$, $l(A_k) \leftarrow B_j$, $X \leftarrow X \cup \{A_k\}$
4. return match //计算结束
5. $A_i \leftarrow l(B_j)$, $\text{match}(B_j) \leftarrow A_i$, $\text{match}(A_i) \leftarrow B_j$
//回溯找到 P , $M \leftarrow M \oplus P$
 - 5.1 if $l(A_i) = 0$ then goto 2 //返回开始下一阶段标号
 - 5.2 else $B_j \leftarrow l(A_i)$, goto 5

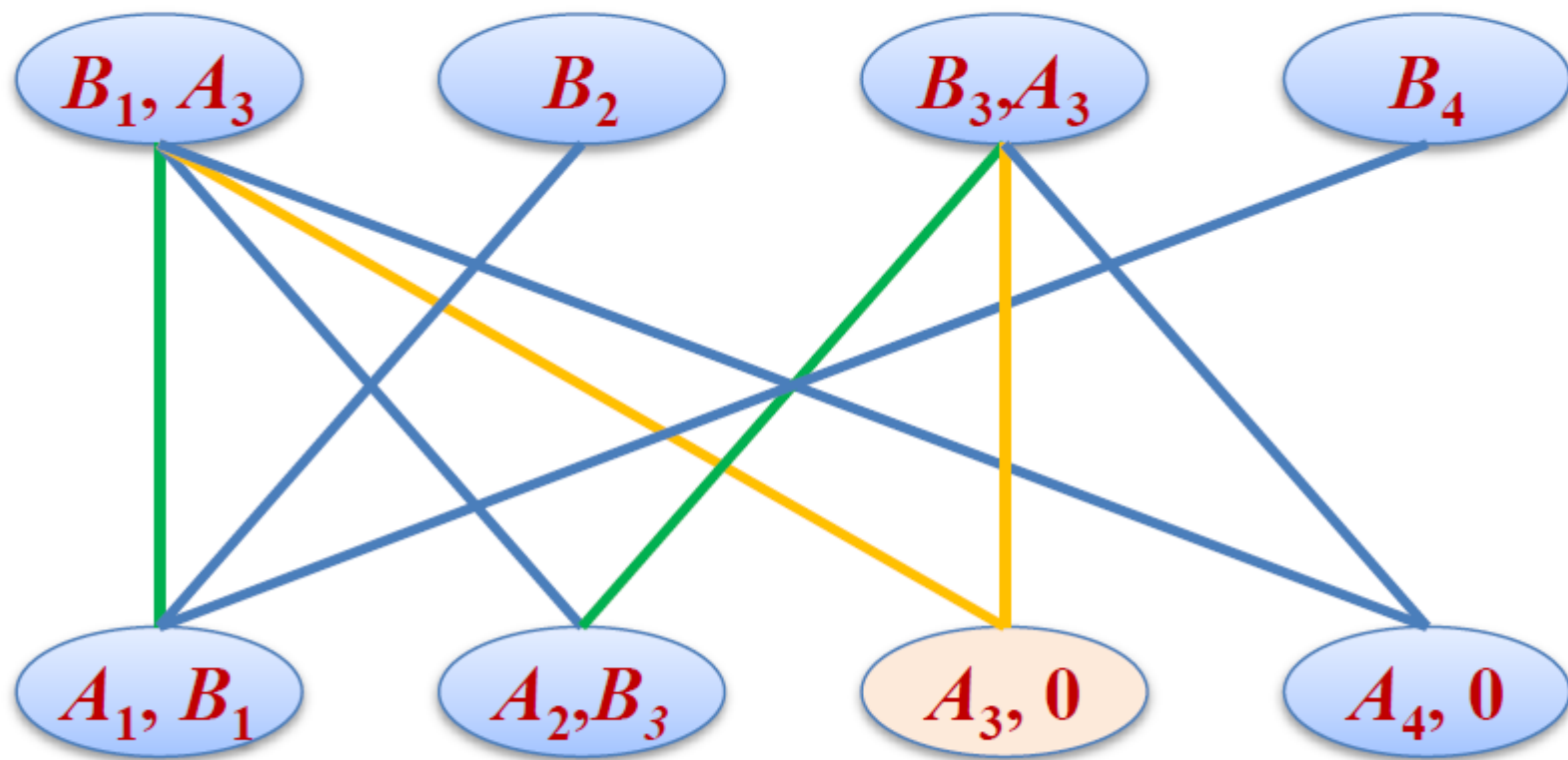
匈牙利算法示例(1)



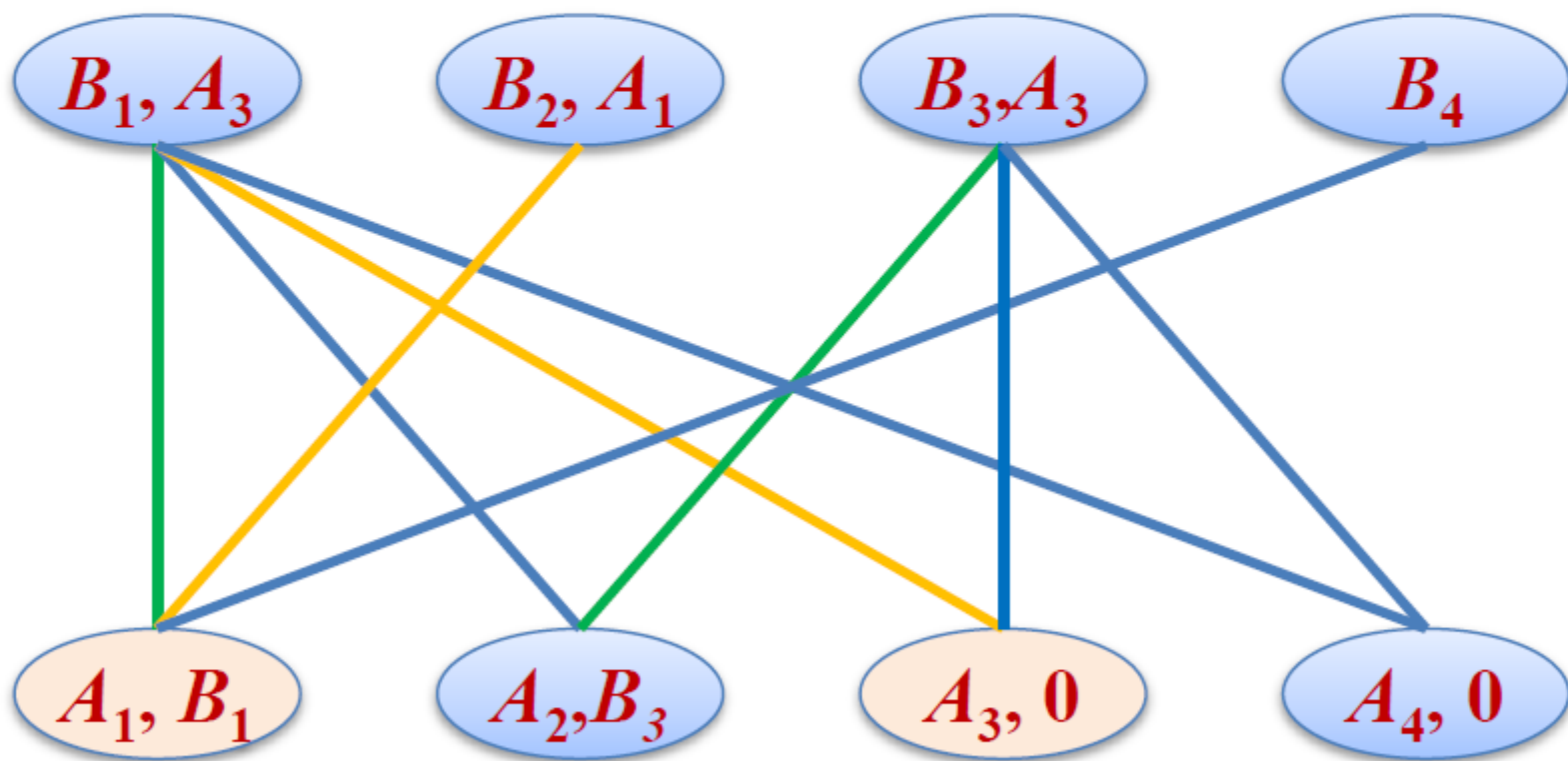
匈牙利算法示例(2)



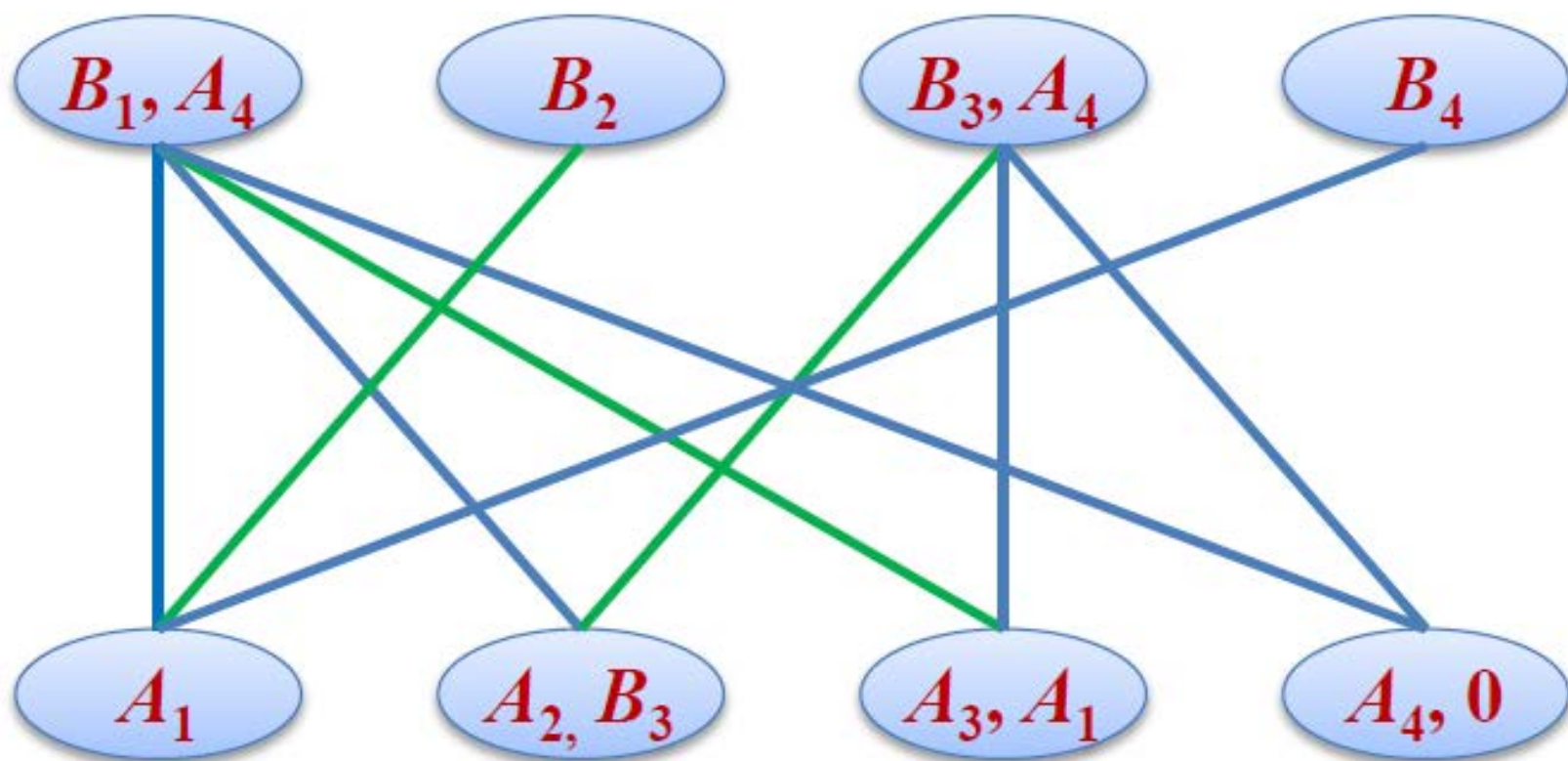
匈牙利算法示例 (3)



匈牙利算法示例 (4)



匈牙利算法示例 (5)



匈牙利算法的时间复杂度

定理7.9 对二部图 $G=\langle A,B,E\rangle$, 匈牙利算法终止时 M 是 G 的最大匹配, 并且算法在 $O(\min\{|A|,|B|\}\cdot|E|)$ 步内终止.

证 开始 $M=\emptyset$, 是 G 的匹配, 每次令 $M\leftarrow M\oplus E(P)$. 可归纳证明最后得到的 M 是 G 的匹配. 当计算终止时, 如果存在增广交错路径 P , 它的端点 A_i 和 B_j 都是非饱和的, 从 A_i 开始沿 P 最后必给 B_j 标号, 计算终止时 B 中已标号顶点都是饱和点, 矛盾.

除最后阶段外, 算法每阶段 (标号和修改 M) 增加一条匹配边, 最多有 $\min\{|A|,|B|\}$ 条匹配边, 故最多有 $\min\{|A|,|B|\}+1$ 个阶段. 在每个阶段标号过程中, 每条边最多检查一次, 增广交错路径的长不超过 $2|M|+1$, 故每个阶段可在 $O(|E|)$ 内完成. 得证计算必在 $O(\min\{|A|,|B|\}\cdot|E|)$ 步内终止.

最大匹配算法与最大流

二部图的最大匹配问题可转化成最大流问题.

设 $G = \langle A, B, E \rangle$,

作容量网络

$N_G = \langle V, E', c, s, t \rangle$, 其中 $c \equiv 1$,

$V = \{s, t\} \cup A \cup B$

$E' = \{ \langle s, A_i \rangle \mid A_i \in A \} \cup \{ \langle B_j, t \rangle \mid B_j \in B \} \cup \{ \langle A_i, B_j \rangle \mid (A_i, B_j) \in E \}$

G 的匹配与 N 上的 0-1 可行流一一对应, 前向边是非饱和边, 后向边是饱和边, 增广链对应增广交错路径. 匈牙利算法实际上是FF算法的应用.

Dinic算法的应用

用 Dinic 算法解二部图的最大匹配问题

(1) 构造二部图 $G = \langle A, B, E \rangle$ 的对应容量网络 N_G .

(2) 对 N_G 应用 Dinic 算法, 求得 N_G 上的最大流 f .

(3) 把 f 转化为 G 的匹配 M .

(1)和(3)可以在 $O(n+m)$ 步内完成. 而 N_G 是一个简单容量网络, 容量均为 1, A_i 的入度为 1, B_j 的出度为 1. (2)可在 $O(n^{1/2}m)$ 步内完成 (根据定理7.4). 可设 G 中无孤立点, 于是有下述定理.

定理7.10 用 Dinic 算法解二部图的最大匹配问题的时间复杂度是 $O(n^{1/2}m)$.

赋权二部图的匹配

设赋权完全二部图 $G = \langle A, B, E, w \rangle$ ，其中
 $A = \{A_i | 1 \leq i \leq n\}$ ， $B = \{B_j | 1 \leq j \leq n\}$ ，
 $E = \{(A_i, B_j) | 1 \leq i \leq n, 1 \leq j \leq n\}$ ， $w: E \rightarrow R$ 。

并假设 $|A| = |B|$ ， $w(A_i, B_j) = w_{ij}$

求 G 的权

$$w(M) = \sum_{(A_i, B_j) \in M} w_{ij}$$

最小的完美匹配 M 。（也称指派问题）

例子：任务分配

设4项任务由4人去完成，每人完成每一项任务的时间如表所示，如何分配任务使总时间最少？

人员	任务			
	B_1	B_2	B_3	B_4
A_1	9	4	3	11
A_2	8	5	8	4
A_3	3	8	3	1
A_4	10	6	9	6

指派问题的线性规划表述 (P)

$$\text{令 } x_{ij} = \begin{cases} 1, & (A_i, B_j) \in M \\ 0, & \text{else} \end{cases} \quad 1 \leq i \leq n, 1 \leq j \leq n$$

$$\min \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_{ij}$$

$$\text{s.t. } \sum_{j=1}^n x_{ij} = 1, 1 \leq i \leq n$$

$$\sum_{i=1}^n x_{ij} = 1, 1 \leq j \leq n$$

$$x_{ij} \geq 0, 1 \leq i \leq n, 1 \leq j \leq n$$

虽未限制 $x_{ij} = 0$ 或 1 ，但实际的解却总是能保证这一点

原始规划

$$\max \sum_{j=1}^n c_j x_j$$

$$\sum_{j=1}^n a_{ij} x_j \leq b_i, \quad 1 \leq i \leq s$$

$$\sum_{j=1}^n a_{ij} x_j = b_i, \quad s+1 \leq i \leq m$$

$$x_j \geq 0, \quad 1 \leq j \leq t$$

$$x_j \text{ 任意}, \quad t+1 \leq j \leq n$$

对偶规划

$$\min \sum_{i=1}^m b_i y_i$$

$$y_i \geq 0, \quad 1 \leq i \leq s$$

$$y_i \text{ 任意}, \quad s+1 \leq i \leq m$$

$$\sum_{i=1}^m a_{ij} y_i \geq c_j, \quad 1 \leq j \leq t$$

$$\sum_{i=1}^m a_{ij} y_i = c_j, \quad t+1 \leq j \leq n$$

(P) 的对偶规划 (D)

$$\max \sum_{i=1}^n \alpha_i + \sum_{j=1}^n \beta_j$$

$$\text{s.t. } \alpha_i + \beta_j \leq w_{ij}, 1 \leq i \leq n, 1 \leq j \leq n$$

$$\alpha_i \text{ 任意}, 1 \leq i \leq n$$

$$\beta_j \text{ 任意}, 1 \leq j \leq n$$

$$\text{取 } \alpha_i^{(0)} = 0, 1 \leq i \leq n, \beta_j^{(0)} = \min_i \{w_{ij}\}, 1 \leq j \leq n$$

这是 (D) 的可行解, 记

$$IJ = \{(i, j) | \alpha_i^{(0)} + \beta_j^{(0)} = w_{ij}\}$$

互补松弛性

原始线性规划 (P)

$$\begin{aligned} \max \quad & c^T x \\ \text{s.t.} \quad & Ax \leq b \\ & x \geq 0 \end{aligned}$$

对偶线性规划 (D)

$$\begin{aligned} \min \quad & b^T y \\ \text{s.t.} \quad & A^T y \geq c \\ & y \geq 0 \end{aligned}$$

定理6.8 设 x 和 y 分别是原始规划 (P) 和对偶规划 (D) 的可行解, 则 x 和 y 分别是它们的最优解当且仅当

$$(b_i - \sum_{j=1}^n a_{ij} x_j) y_i = 0, \quad 1 \leq i \leq m \quad (*)$$

$$x_j (\sum_{i=1}^m a_{ij} y_i - c_j) = 0, \quad 1 \leq j \leq n \quad (**)$$

由互补松弛条件 (P) 的最优解应满足

$$\sum_{j=1}^n x_{ij} = 1, 1 \leq i \leq n$$

$$\sum_{i=1}^n x_{ij} = 1, 1 \leq j \leq n$$

$$x_{ij} \geq 0, (i, j) \in IJ$$

$$x_{ij} = 0, (i, j) \notin IJ$$

原始问题 (P) 的限制问题 (RP)

$$\begin{aligned} \min \xi &= \sum_{k=1}^{2n} y_k \\ \text{s.t. } \sum_{j=1}^n x_{ij} + y_i &= 1, 1 \leq i \leq n \\ \sum_{i=1}^n x_{ij} + y_{n+j} &= 1, 1 \leq j \leq n \\ y_k &\geq 0, 1 \leq k \leq 2n \\ x_{ij} &\geq 0, (i, j) \in IJ \\ x_{ij} &= 0, (i, j) \notin IJ \end{aligned}$$

其中 $y_k (1 \leq k \leq 2n)$ 是人工变量

限制的原始问题 (RP) 的含义

$$\begin{aligned}\text{由于 } \xi &= \sum_{i=1}^n (1 - \sum_{j=1}^n x_{ij}) + \sum_{j=1}^n (1 - \sum_{i=1}^n x_{ij}) \\ &= 2n - 2 \sum_{(i,j) \in IJ} x_{ij}\end{aligned}$$

(RP) 等同于 (RP')

$$\begin{aligned}& \max \sum_{(i,j) \in IJ} x_{ij} \\ \text{s.t. } & \sum_{j=1}^n x_{ij} \leq 1, 1 \leq i \leq n \\ & \sum_{i=1}^n x_{ij} \leq 1, 1 \leq j \leq n \\ & x_{ij} \geq 0, (i,j) \in IJ \\ & x_{ij} = 0, (i,j) \notin IJ\end{aligned}$$

(RP') 是二部图最大匹配问题

是二部图 $G(IJ) = \langle A, B, E(IJ) \rangle$ 的最大匹配问题

利用上一讲的算法求解之，求得其最大匹配为 M
设对应 (RP') 的最优解为 $\{x^*_{ij}\}$

- 如果 $|M| = n$ ，则 $\sum_{(i,j) \in IJ} x_{ij} = n$ ， $\xi=0$
从而 $\{x^*_{ij}\}$ 是 (P) 的最优解，最优值为

$$\sum_{(i,j) \in IJ} w_{ij} = \sum_{i=1}^n \alpha_i^{(0)} + \sum_{j=1}^n \beta_j^{(0)}$$

- M 即为所求的最小权完美匹配

(RP)的对偶规划问题的最优解

如果 $|M| < n$ ，根据在 $G(IJ)$ 求最大匹配结束时顶点的标号状态，令

$$\alpha_i^* = \begin{cases} 1 & A_i \text{已标号} \\ -1 & A_i \text{未标号} \end{cases}, 1 \leq i \leq n$$
$$\beta_j^* = \begin{cases} -1 & B_j \text{已标号} \\ 1 & B_j \text{未标号} \end{cases}, 1 \leq j \leq n$$

引理 7.15

$\alpha_i^*(1 \leq i \leq n), \beta_j^*(1 \leq j \leq n)$ 是式 (RP) 的对偶规划问题的最优解。

(RP)的对偶规划问题(DRP)

$$\max \sum_{i=1}^n \alpha_i + \sum_{j=1}^n \beta_j$$

$$\text{s.t. } \alpha_i + \beta_j \leq 0, (i, j) \in IJ$$

$$\alpha_i \leq 1, 1 \leq i \leq n$$

$$\beta_j \leq 1, 1 \leq j \leq n$$

$$\alpha_i \text{ 任意}, 1 \leq i \leq n$$

$$\beta_j \text{ 任意}, 1 \leq j \leq n$$

α_i^*, β_j^* 是(DRP)的可行解

证明:

$$\forall (i, j) \in IJ, (A_i, B_j) \in IJ$$

如果 A_i 已经标号, 则 B_j 也一定标号, 因此

$$\alpha_i^* + \beta_j^* \leq 0$$

得证 $\alpha_i^* (1 \leq i \leq n), \beta_j^* (1 \leq j \leq n)$ 是式 (DRP) 的可行解。

α_i^*, β_j^* 是(DRP)的最优解

证明:

对应 $\forall (A_i, B_j) \in M$, A_i 和 B_j 要么同时标号, 要么都未标号, 从而 $\alpha_i^* + \beta_j^* = 0$

其余的顶点都是非饱和点, 非饱和的 A_i 都已标号, 非饱和的 B_j 都未标号 (否则找到一条增广交错路径), 从而 $\alpha_i^* + \beta_j^* = 1$

于是

$$\sum_{i=1}^n \alpha_i^* + \sum_{j=1}^n \beta_j^* = 2(n - |M|) = 2n - 2 \sum_{(i,j) \in IJ} x_{ij}^* = \xi$$

用(DRP)的最优解改进(D)的解

通过让新的 (i, j) 进入 IJ 来改进(D)的可行解, 令

$$\alpha_i^{(1)} = \alpha_i^{(0)} + \theta \alpha_i^*, 1 \leq i \leq n$$

$$\beta_j^{(1)} = \beta_j^{(0)} + \theta \beta_j^*, 1 \leq j \leq n$$

由于

$$\begin{aligned} \sum_{i=1}^n \alpha_i^{(1)} + \sum_{j=1}^n \beta_j^{(1)} &= \left(\sum_{i=1}^n \alpha_i^{(0)} + \sum_{j=1}^n \beta_j^{(0)} \right) + \theta \left(\sum_{i=1}^n \alpha_i^* + \sum_{j=1}^n \beta_j^* \right) \\ &= \left(\sum_{i=1}^n \alpha_i^{(0)} + \sum_{j=1}^n \beta_j^{(0)} \right) + \theta \xi \end{aligned}$$

用(DRP)的最优解改进(D)的解

而 $\xi > 0$ ，应取 $\theta > 0$ 。

为了保证 $\alpha_i^{(1)}, \beta_j^{(1)}$ 是(D)的可行解，要求

$$w_{ij} \geq \alpha_i^{(1)} + \beta_j^{(1)} = \left(\alpha_i^{(0)} + \beta_j^{(0)} \right) + \theta(\alpha_i^* + \beta_j^*)$$

有4种可能性：

1. A_i 已标号， B_j 已标号，有 $\alpha_i^{(1)} + \beta_j^{(1)} = \alpha_i^{(0)} + \beta_j^{(0)}$
2. A_i 未标号， B_j 未标号，有 $\alpha_i^{(1)} + \beta_j^{(1)} = \alpha_i^{(0)} + \beta_j^{(0)}$
3. A_i 未标号， B_j 已标号，有 $\alpha_i^{(1)} + \beta_j^{(1)} = \alpha_i^{(0)} + \beta_j^{(0)} - 2\theta$
4. A_i 已标号， B_j 未标号，有 $\alpha_i^{(1)} + \beta_j^{(1)} = \alpha_i^{(0)} + \beta_j^{(0)} + 2\theta$

$$\theta = \min \left\{ \frac{1}{2} \left(w_{ij} - \alpha_i^{(0)} + \beta_j^{(0)} \right) \mid A_i \text{已标号}, B_j \text{未标号} \right\}$$

- (1)和(2)不改变 $G(IJ)$ 的边. M 的边的顶点必属于(1)或(2), 从而 M 仍是新二部图的匹配.
- (3)若 (A_i, B_j) 是原图的边, 则在新图被剔除, 不影响下面标号.
- (4) A_i 和 B_j 使得 $\alpha_i^{(0)} + \beta_j^{(0)} < w_{ij}$, $\alpha_i^{(1)} + \beta_j^{(1)} = w_{ij}$, 从而新图得到一条新边 (A_i, B_j) , 使得终止的标号过程得以继续进行.

算法中的部分说明与优化

因为只有情况4时，才可能在 $G(IJ)$ 加入新边使得标号得以继续，可以把修改 α_i, β_j 的过程嵌入标号过程，当标号过程停止而又没有找到增广交错路径时，转入对 α_i, β_j ，加入新边后（可能要多次修改）才能找到增广交错路径。

为优化 θ 的计算，通过变量 s 和 h ，维护

$$s(B_j) = \min\{w_{ij} - \alpha_i - \beta_j \mid A_i \text{ 已标号}\}$$

$$h(B_j) = \text{使 } s(B_j) \text{ 取到最小值的 } A_i$$

使得每次计算 θ 的时间从 $O(n^2)$ 降低到 $O(n)$

匈牙利算法（续7）

算法7.8 匈牙利算法

输入 赋权完全二部图 $G=\langle A, B, E, w \rangle$, 其中 $|A|=|B|=n$

输出 match

1. for $i=1$ to n do $\text{match}(A_i) \leftarrow 0, \alpha_i \leftarrow 0$ //初始化
- 1.1 for $j=1$ to n do $\text{match}(B_j) \leftarrow 0, \beta_j \leftarrow \min\{w_{ij} \mid 1 \leq i \leq n\}$
2. for $q=1$ to n do // n 个阶段, 每阶段增加一条匹配边
- 2.1 $IJ \leftarrow \emptyset$
- 2.2 for $j=1$ to n do
- 2.3 $s(B_j) \leftarrow +\infty$ // $s(B_j) = \min\{w_{ij} - \alpha_i - \beta_j \mid A_i \text{ 已标号}\}$
- 2.4 for $i=1$ to n do
- 2.5 if $\alpha_i + \beta_j = w_{ij}$ then $IJ \leftarrow IJ \cup \{(A_i, B_j)\}$ //构造 $G(IJ)$

```

3.       $X \leftarrow \emptyset, XX \leftarrow \emptyset$            //标号过程开始
3.1     for  $A_i \in A \wedge \text{match}(A_i)=0$  do
3.2          $l(A_i) \leftarrow 0, X \leftarrow X \cup \{A_i\}, XX \leftarrow XX \cup \{A_i\}$ 
4.       $Y \leftarrow B$            //X是A中已标号顶点, XX是A中已标号
4.1     while  $XX \neq \emptyset$  do       未检查顶点, Y是B中未标号顶点
4.2         任取  $A_i \in XX, XX \leftarrow XX - \{A_i\}$ 
4.3         for  $B_j \in Y$  do
4.4             if  $0 < w_{ij} - \alpha_i - \beta_j < s(B_j)$  then
4.5                  $s(B_j) \leftarrow w_{ij} - \alpha_i - \beta_j, h(B_j) \leftarrow A_i$ 
4.6             if  $(A_i, B_j) \in IJ$  then //h(B_j)是使s(B_j)取最小值的A_i
4.7                  $l(B_j) \leftarrow A_i, Y \leftarrow Y - \{B_j\}$ 
4.8                 if  $\text{match}(B_j)=0$  then goto 6           //存在P
4.9                  $A_k \leftarrow \text{match}(B_j), l(A_k) \leftarrow B_j,$ 
                      $X \leftarrow X \cup \{A_k\}, XX \leftarrow XX \cup \{A_k\}$ 

```

```

5.       $\theta \leftarrow 0.5 \times \min\{s(B_j) \mid B_j \in Y\}$                                 //修改  $\alpha_i, \beta_j$ 
5.1    for  $i=1$  to  $n$  do
5.2      if  $A_i \in X$  then  $\alpha_i \leftarrow \alpha_i + \theta$  else  $\alpha_i \leftarrow \alpha_i - \theta$ 
5.3    for  $j=1$  to  $n$  do
5.4      if  $B_j \in Y$  then  $\beta_j \leftarrow \beta_j + \theta$  else  $\beta_j \leftarrow \beta_j - \theta$ 
5.5    for  $B_j \in Y$  do
5.6       $s(B_j) \leftarrow s(B_j) - 2\theta$ 
5.6      if  $s(B_j) = 0$  then
5.7         $A_i \leftarrow h(B_j), XX \leftarrow XX \cup \{A_i\}, IJ \leftarrow IJ \cup \{(A_i, B_j)\}$ 
5.8    goto 4.1                                //返回继续标号
6.       $A_i \leftarrow l(B_j), \text{match}(B_j) \leftarrow A_i, \text{match}(A_i) \leftarrow B_j$     //  $M \leftarrow M \oplus P$ 
6.1    if  $l(A_i) \neq 0$  then  $B_j \leftarrow l(A_i), \text{goto } 6$  //否则  $q$  的这一阶段结束

```

匈牙利算法（续8）

定理7.11 算法7.8得到的 M 是最小权完美匹配, 并且在 $O(n^3)$ 步内终止计算.

证 每个阶段增加一条匹配边, n 个阶段得到的 M 是完美匹配. 结束时, $\alpha_i (1 \leq i \leq n)$, $\beta_j (1 \leq j \leq n)$ 是(D)的可行解, 目标函数值为 $\sum_{i=1}^n \alpha_i + \sum_{j=1}^n \beta_j$, 对应 M 的 $x_{ij} (1 \leq i \leq n, 1 \leq j \leq n)$ 是 (P)的可行解,

其目标函数值为

$$\sum_{(A_i, B_j) \in M} w_{ij} = \sum_{(A_i, B_j) \in M} (\alpha_i + \beta_j) = \sum_{i=1}^n \alpha_i + \sum_{j=1}^n \beta_j$$

故 $\alpha_i, \beta_j (1 \leq i, j \leq n)$ 和 $x_{ij} (1 \leq i, j \leq n)$ 分别是 (D) 和 (P) 的最优解, 得证 M 是最小权完美匹配.

匈牙利算法（续9）

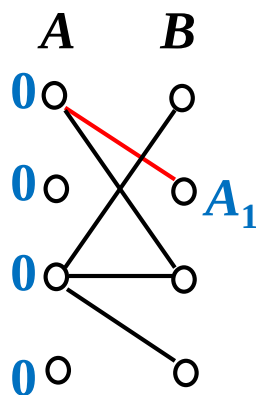
共有 n 个阶段. 每个阶段, 构造 $G(IJ)$ (2.1~2.5)的工作量为 $O(n^2)$. 至多有 n 个已标号未检查的 A_i , 检查一个 A_i (4.2~4.9)的工作量为 $O(n)$, 标号的总工作量为 $O(n^2)$.

注意, 对那些经过修改对偶变量重新返回的 A_i , 是已进行的标号过程的继续, 不是从头检查一个新的已标号未检查的 A_i , 所以不增加已标号未检查的 A_i 的数量.

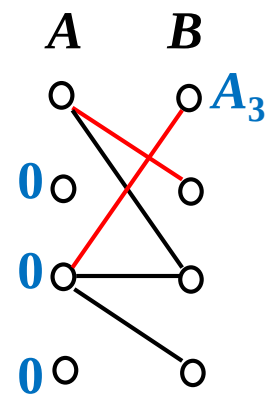
每次修改 α_i, β_j ($1 \leq i, j \leq n$), 或者找到一条增广交错路径, 或者至少使一个未标号的 B_j 获得标号, 因此在一个阶段内修改对偶变量的次数不超过 n . 每一次修改(5~5.7) 的工作量是 $O(n)$, 其总的工作量为 $O(n^2)$. 综上所述, 每一个阶段的工作量为 $O(n^2)$. 得证算法的时间复杂度为 $O(n^3)$.

例7.6 (续)

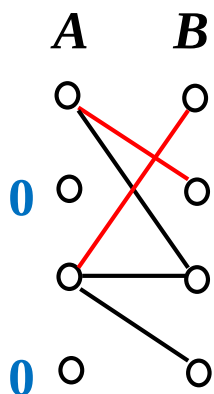
	α			
	9	4	3	9 0
	8	5	8	4 0
	3	8	3	1 0
	9	6	9	6 0
β	3	4	3	1
s				
h				



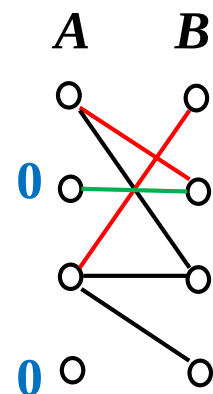
	α			
	9	4	3	9 0
	8	5	8	4 0
	3	8	3	1 0
	9	6	9	6 0
β	3	4	3	1
s				
h				



	α			
	9	4	3	9 0
*	8	5	8	4 0
	3	8	3	1 0
*	9	6	9	6 0
β	3	4	3	1
s	5	1	5	3
h	A_2	A_2	A_2	A_2

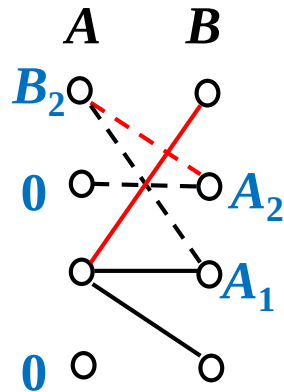


	α			
	9	4	3	9 -0.5
*	8	5	8	4 0.5
	3	8	3	1 -0.5
*	9	6	9	6 0.5
β	3.5	4.5	3.5	1.5
s	4	0	4	2
h	A_2	A_2	A_2	A_2

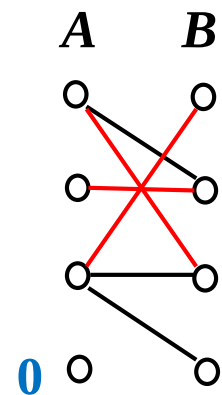


例7.6 (续)

				α	
	9	4	3	9	-0.5
	8	5	8	4	0.5
	3	8	3	1	-0.5
	9	6	9	6	0.5
β	3.5	4.5	3.5	1.5	
s					
h					

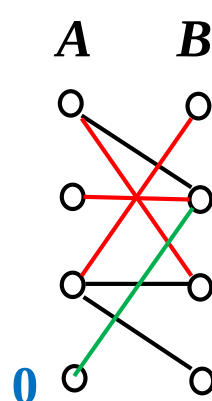


					α
	9	4	3	9	-0.5
	8	5	8	4	0.5
	3	8	3	1	-0.5
*	9	6	9	6	0.5
β	3.5	4.5	3.5	1.5	
s	5	1	5	4	
h	A_4	A_4	A_4	A_4	

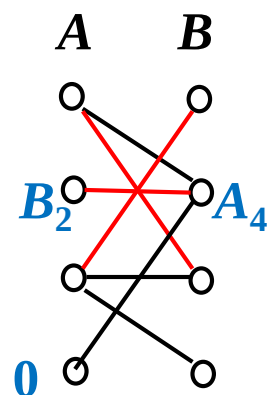


例7.6 (续)

				α	
	9	4	3	9	-1
	8	5	8	4	0
	3	8	3	1	-1
*	9	6	9	6	1
β	4	5	4	2	
s	4	0	4	3	
h	A_4	A_4	A_4	A_4	

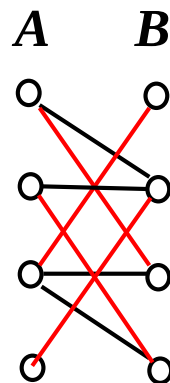


					α
	9	4	3	9	-1
*	8	5	8	4	0
	3	8	3	1	-1
*	9	6	9	6	1
β	4	5	4	2	
s	4		4	2	
h	A_2		A_2	A_2	



例7.6 (续)

				α	
	9	4	3	9	-2
	8	5	8	4	1
	3	8	3	1	-2
	9	6	9	6	2
β	5	4	5	3	
s					
h					



最优解 $M = \{ (A_1, B_3), (A_2, B_4), (A_3, B_1), (A_4, B_2) \}$,

$$w(M) = 3 + 4 + 3 + 6 = 16.$$

PageRank算法

- 在实际应用中许多数据都以图 (graph)的形式存在, 比如, 互联网、社交网络都可以看作是一个图
- 图数据上的机器学习具有理论与应用上的重要意义
- PageRank算法是图的链接分析 (link analysis) 的代表性算法, 属于图数据上的无监督学习方法。
- PageRank可以定义在任意有向图 上, 后来被应用到社会影响力分析、文本摘要等多个问题。

PageRank算法

- PageRank算法的基本想法是在有向图上定义一个随机游走模型，即一阶马尔可夫链，描述随机游走者沿着有向图随机访问各个结点的行为
- 在一定条件下，极限情况访问每个结点的概率收敛到平稳分布，这时各个结点的平稳概率值就是其PageRank值，表示结点的重要度。
- PageRank是递归定义的，PageRank的计算可以通过迭代算法进行。

PAGERANK的定义

基本想法

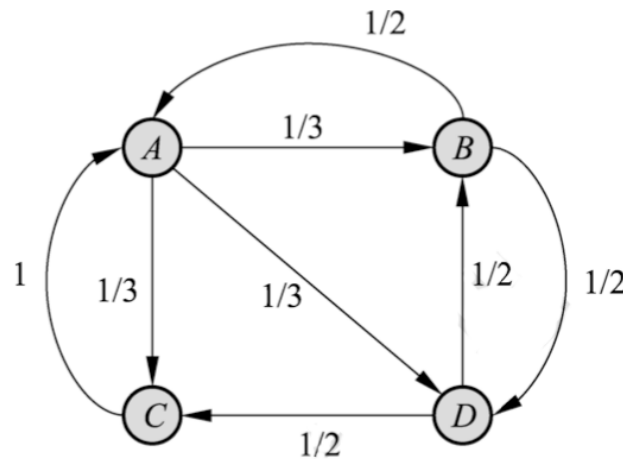
- 历史上，PageRank算法作为计算互联网网页重要度的算法被提出
- PageRank是定义在网页集合上的一个函数，它对每个网页给出一个正实数，表示网页的重要程度，整体构成一个向量
- PageRank值越高，网页就越重要，在互联网搜索的排序中可能就被排在前面

基本想法

- 假设互联网是一个有向图，在其基础上定义随机游走模型，即一阶马尔可夫链，表示网页浏览者在互联网上随机浏览网页的过程
- 假设浏览者在每个网页依照连接出去的超链接以等概率跳转到下一个网页，并在网上持续不断进行这样的随机跳转，这个过程形成一阶马尔可夫链
- PageRank表示这个马尔可夫链的平稳分布
- 每个网页的PageRank值就是平稳概率。

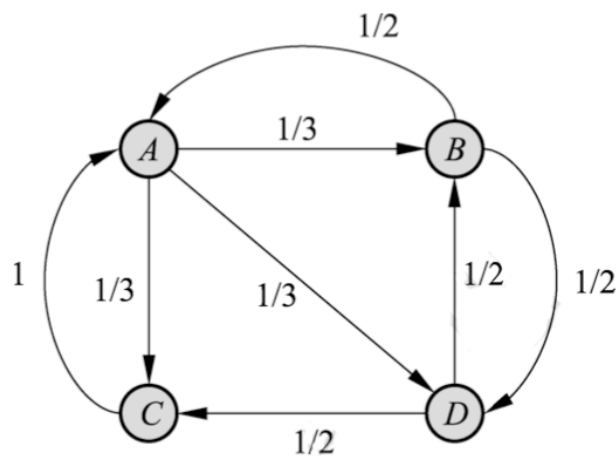
基本想法

- 下图表示一个有向图，假设是简化的互联网例，结点A,B,C和D表示网页，结点之间的有向连接，边上的权值表示概率



基本想法

- 假设有一个浏览者，在时间 t 浏览网页 i
- 如果浏览者在网页A，
 - 则下一步以 $1/3$ 的概率转移到网页B
 - 以 $1/3$ 的概率转移到网页C
 - 以 $1/3$ 的概率转移到网页D
- 如果浏览者在网页B，
 - 则下一步以 $1/2$ 的概率转移到网页A
 - 以 $1/2$ 的概率转移到网页D
- 如果浏览者在网页C，
 - 则下一步以概率1转移到网页A
- 如果浏览者在网页D，
 - 以 $1/2$ 的概率转移到网页B
 - 以 $1/2$ 的概率转移到网页C



基本想法

- 直观上，一个网页，如果指向该网页的超链接越多，随机跳转到该网页的概率也就越高，该网页的PageRank值就越高，这个网页也就越重要
- 一个网页，如果指向该网页的PageRank值越高，随机跳转到该网页的概率也就越高，该网页的PageRank值就越高，这个网页也就越重要
- PageRank值依赖于网络的拓扑结构，一旦网络的拓扑（连接关系）确定，PageRank值就确定

基本想法

- PageRank的计算可以在互联网的有向图上进行，通常是一个迭代过程
- 先假设一个初始分布，通过迭代，不断计算所有网页的PageRank值，直到收敛为止

有向图

定义 21.1 (有向图) 有向图 (directed graph) 记作 $G = (V, E)$, 其中 V 和 E 分别表示结点和有向边的集合。

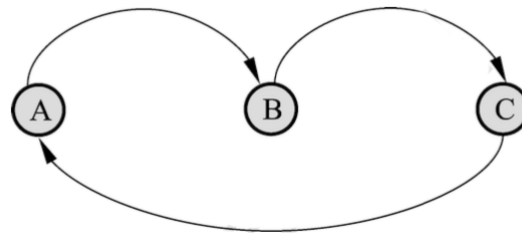
- 从一个结点出发到达另一个结点, 所经过的边的一个序列称为一条路径 (path), 路径上边的个数称为路径的长度
- 如果一个有向图从其中任何一个结点出发可以到达 其他任何一个结点, 就称这个

有向图

- 假设 k 是一个大于1的自然数
- 如果从有向图的一个结点出发返回到这个结点的路径的长度都是 k 的倍数，那么称这个结点为周期性结点
- 如果一个有向图不含有周期性结点，则称这个有向图为非周期性图 (aperiodic)

有向图

- 下图是一个周期性有向图的例子



- 从结点A出发返回到A，必须经过结点B—C—A，所有可能的路径的长度都是3的倍数，所以结点A是周期性结点。
这个有向图是周期性图

随机游走模型

定义 21.2 (随机游走模型) 给定一个含有 n 个结点的有向图, 在有向图上定义随机游走 (random walk) 模型, 即一阶马尔可夫链^①, 其中结点表示状态, 有向边表示状态之间的转移, 假设从一个结点到通过有向边相连的所有结点的转移概率相等。具体地, 转移矩阵是一个 n 阶矩阵 M

$$M = [m_{ij}]_{n \times n} \quad (21.1)$$

第 i 行第 j 列的元素 m_{ij} 取值规则如下: 如果结点 j 有 k 个有向边连出, 并且结点 i 是其连出的一个结点, 则 $m_{ij} = \frac{1}{k}$; 否则 $m_{ij} = 0$, $i, j = 1, 2, \dots, n$ 。

随机游走模型

- 注意转移矩阵具有 $m_{ij} \geq 0$

$$\sum_{i=1}^n m_{ij} = 1$$

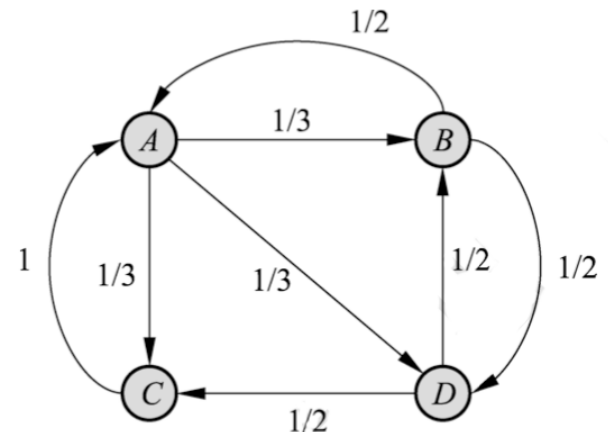
- 即每个元素非负，每列元素之和为1，即矩阵M为随机矩阵 (stochastic matrix)。

随机游走模型

- 在有向图上的随机游走形成马尔可夫链。也就是说，随机游走者每经一个单位时间转移一个状态
- 如果当前时刻在第 j 个结点（状态），那么下一个时刻在第 i 个结点（状态）的概率是 m_{ij}
- 这一概率只依赖于当前的状态，与过去无关，具有马尔可夫性。

随机游走模型

- 在下图的有向图上可以定义随机游走模型
- 结点A到结点B,C和D存在有向边, 可以以概率 $1/3$ 从A分别转移到B,C和D, 并以概率0转移到A, 于是可以写出转移矩阵的第1列
- 结点B到结点A和D存在有向边, 可率 $1/2$ 从B分别转移到A和D, 并以概率0移到B和C, 于是可以写出矩阵的第2列



随机游走模型

- 于是得到转移

$$M = \begin{bmatrix} 0 & 1/2 & 1 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

- 随机游走在某个时刻 $R_{t+1} = MR_t$ 一个结点的概率分布就是马尔可夫链在时刻 t 的状态分布, 可以用一个 n 维列向量 R_t 表示. 那么在时刻

PageRank的基本定义

- 给定一个包含 n 个结点的强连通且非周期性的有向图，在其基础上定义随机游走模型
- 假设转移概率分布为 $R_0, MR_0, M^2R_0, \dots, M^tR_0, \dots$ ，访问各结点的概率分布为 $\lim_{t \rightarrow \infty} M^tR_0 = R$

$$MR = R$$

- 则极限 R 存在

PageRank的基本定义

定义 21.3 (PageRank 的基本定义) 给定一个包含 n 个结点 $v_1, v_2, \dots, v_n$ 的强连通且非周期性的有向图，在有向图上定义随机游走模型，即一阶马尔可夫链。随机游走的特点是从一个结点到有有向边连出的所有结点的转移概率相等，转移矩阵为 M 。这个马尔可夫链具有平稳分布 R

$$MR = R \quad (21.6)$$

平稳分布 R 称为这个有向图的 PageRank。 R 的各个分量称为各个结点的 PageRank 值。

$$R = \begin{bmatrix} PR(v_1) \\ PR(v_2) \\ \vdots \\ PR(v_n) \end{bmatrix}$$

其中 $PR(v_i)$, $i = 1, 2, \dots, n$, 表示结点 v_i 的 PageRank 值。

PageRank的基本定义

- 显然有

$$PR(v_i) \geq 0, \quad i = 1, 2, \dots, n$$

$$\sum_{i=1}^n PR(v_i) = 1$$

$$PR(v_i) = \sum_{v_j \in M(v_i)} \frac{PR(v_j)}{L(v_j)}, \quad i = 1, 2, \dots, n$$

- $M(v_i)$ 表示指向结点 v_i 的结点集合
- $L(v_j)$ 表示结点 v_j 连出的有向边的个数
- PageRank的基本定义是理想化的，在这种情况下，PageRank存在，而且可以通过不断迭代求得PageRank值

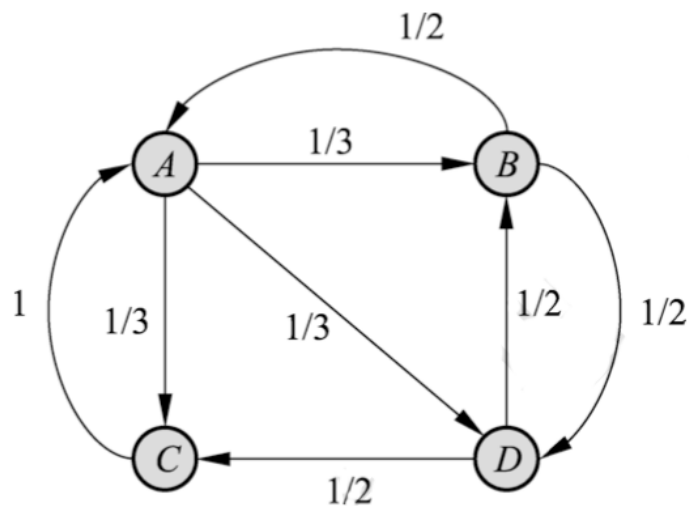
PageRank的基本定义

定理 21.1 不可约且非周期的有限状态马尔可夫链，有唯一平稳分布存在，并且当时间趋于无穷时状态分布收敛于唯一的平稳分布。

根据马尔可夫链平稳分布定理，强连通且非周期的有向图上定义的随机游走模型（马尔可夫链），在图上的随机游走当时间趋于无穷时状态分布收敛于唯一的平稳分布。

例

- 求下图的PageRank



例

- 转移矢 $M = \begin{bmatrix} 0 & 1/2 & 1 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$

- 取初始分布向: $R_0 = \begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}$

例

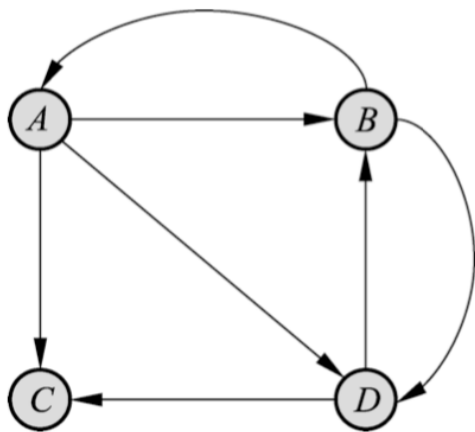
- 以转移矩阵M连乘初始向量 R_0 得到向量序列

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}, \begin{bmatrix} 9/24 \\ 5/24 \\ 5/24 \\ 5/24 \end{bmatrix}, \begin{bmatrix} 15/48 \\ 11/48 \\ 11/48 \\ 11/48 \end{bmatrix}, \begin{bmatrix} 11/32 \\ 7/32 \\ 7/32 \\ 7/32 \end{bmatrix}, \dots, \begin{bmatrix} 3/9 \\ 2/9 \\ 2/9 \\ 2/9 \end{bmatrix}$$

- 最后得到极限向量 $R = \begin{bmatrix} 3/9 \\ 2/9 \\ 2/9 \\ 2/9 \end{bmatrix}$ 即有向图的PageRank值
- 一般的有向图未必满足强连通且非周期性的条件。所以PageRank 的基本定义不适用。

例

- 下图的有向图的转移矩阵M是



$$M = \begin{bmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

例

- 这时M不是一个随机矩阵，因为随机矩阵要求每一列的元素之和是1，这里第3列的和是0，不是1
- 如果仍然计算在各个时刻的各个结点的概率分布，就会得到如下结果

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}, \begin{bmatrix} 3/24 \\ 5/24 \\ 5/24 \\ 5/24 \end{bmatrix}, \begin{bmatrix} 5/48 \\ 7/48 \\ 7/48 \\ 7/48 \end{bmatrix}, \begin{bmatrix} 21/288 \\ 31/288 \\ 31/288 \\ 31/288 \end{bmatrix}, \dots, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

- 可以看到，随着时间推移，访问各个结点的概率皆变为0

PageRank的一般定义

- PageRank一般定义的想法是在基本定义的基础上导入平滑项
- 给定一个含有 n 个结点 $v_i, i=1,2,\dots,n$, 的任意有向图
- 假设考虑一个在图上 随机游走模型, 即一阶马尔可夫链, 其转移矩阵是 M , 从一个结点到其连出的所有结点的转移概率相等。

PageRank的一般定义

- 这个马尔可夫链未必具有平稳分布
- 假设考虑另一个完全随机游走的模型，其转移矩阵的元素全部为 $1/n$ ，也就是说从任意一个结点到任意一个结点的转移概率都是 $1/n$
- 两个转移矩阵的线性组合又构成一个新的

PageRank的一般定义

- 容易证明这个马尔可夫链一定具有平稳分布

$$R = dMR + \frac{1-d}{n}\mathbf{1} \quad (21.10)$$

- 式中 $d(0 \leq d \leq 1)$ 是系数, $R = \begin{bmatrix} PR(v_1) \\ PR(v_2) \\ \vdots \\ PR(v_n) \end{bmatrix}$ 是 n 维向量, $PR(v_i), i = 1, 2, \dots, n$ 是阻尼因子 (damping factor)
- R 是 n 维向量
- 是所有分量为 1 的 n 维向量

PageRank的一般定义

- 式 (21.10) 中第一项表示 (状态分布是平稳分布时) 依照转移矩阵 M 访问各个结点的概率, 第二项表示完全随机访问各个结点的概率
- 阻尼因子 d 取值由经验决定
- 例如 $d=0.85$ 。当 d 接近1时, 随机游走主要依照转移矩阵 M 进行

PageRank的一般定义

- 可以由式 (21.10) 写出每个结点的

PageRank: $PR(v_i) = d \left(\sum_{v_j \in M(v_i)} \frac{PR(v_j)}{L(v_j)} \right) + \frac{1-d}{n}, i = 1, 2, \dots, n$ 意义

- 第二项称为平滑项, 所有结点的PageRank之和为1, 具有以下性质:

$$\sum_{i=1}^n PR(v_i) = 1$$

PageRank的一般定义

定义 21.4 (PageRank 的一般定义) 给定一个含有 n 个结点的任意有向图, 在有向图上定义一个一般的随机游走模型, 即一阶马尔可夫链。一般的随机游走模型的转移矩阵由两部分的线性组合组成, 一部分是有向图的基本转移矩阵 M , 表示从一个结点到你连出的所有结点的转移概率相等, 另一部分是完全随机的转移矩阵, 表示从任意一个结点到任意一个结点的转移概率都是 $1/n$, 线性组合系数为阻尼因子 $d(0 \leq d \leq 1)$ 。这个一般随机游走的马尔可夫链存在平稳分布, 记作 R 。定义平稳分布向量 R 为这个有向图的一般 PageRank。 R 由公式

$$R = dMR + \frac{1-d}{n}\mathbf{1} \quad (21.14)$$

决定, 其中 $\mathbf{1}$ 是所有分量为 1 的 n 维向量。

PageRank的一般定义

- 一般PageRank的定义意味着互联网浏览者，按照以下方法在网上随机游走：
- 在任意一个网页上，浏览者或者以概率 d 决定按照超链接随机跳转，这时以等概率从连接出去的超链接跳转到下一个网页
- 或者以概率 $(1-d)$ 决定完全随机跳转，这时以等概率 $1/n$ 跳转到任意一个网页
- 第二个机制保证从没有连接出去的超链接的网页也可以跳转出。这样可以保证平稳分布，即一般PageRank的存在，因而一般PageRank适用于任何结构的网络。

PAGERANK的计算

迭代算法

- 给定一个含有 n 个结点的有向图，转移矩阵为 M ，有向式
$$R_{t+1} = dMR_t + \frac{1-d}{n}\mathbf{1}$$
由迭代公式 (21.15)
- 的极限向量 R 确定
- PageRank的迭代算法，就是按照这个一般定义进行迭代，直至收敛

PageRank的迭代算法

算法 21.1 (PageRank 的迭代算法)

输入：含有 n 个结点的有向图，转移矩阵 M ，阻尼因子 d ，初始向量 R_0 ；

输出：有向图的 PageRank 向量 R 。

(1) 令 $t = 0$

(2) 计算

$$R_{t+1} = dMR_t + \frac{1-d}{n}\mathbf{1}$$

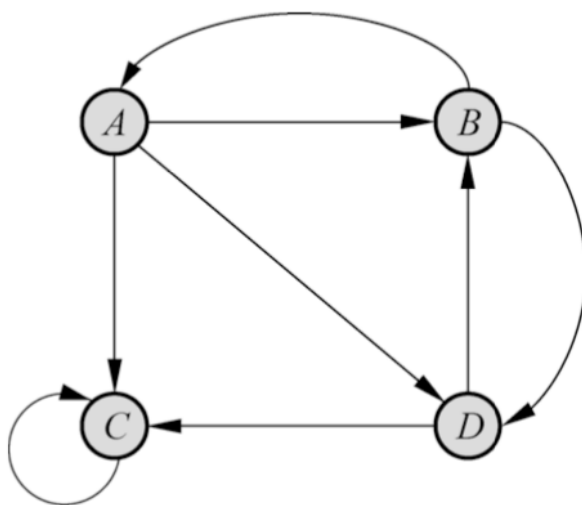
(3) 如果 R_{t+1} 与 R_t 充分接近，令 $R = R_{t+1}$ ，停止迭代。

(4) 否则，令 $t = t + 1$ ，执行步 (2)。



例

- 图中所示的有向图，取 $d = 0.8$ ，求图的PageRank



例

- 可得转移矩阵为

$$M = \begin{bmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 1 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

按照式 (21.15)

$$dM = \frac{4}{5} \times \begin{bmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 1 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 2/5 & 0 & 0 \\ 4/15 & 0 & 0 & 2/5 \\ 4/15 & 0 & 4/5 & 2/5 \\ 4/15 & 2/5 & 0 & 0 \end{bmatrix}$$

$$\frac{1-d}{n} \mathbf{1} = \begin{bmatrix} 1/20 \\ 1/20 \\ 1/20 \\ 1/20 \end{bmatrix}$$

例

- 迭代公式 $R_{t+1} = \begin{bmatrix} 0 & 2/5 & 0 & 0 \\ 4/15 & 0 & 0 & 2/5 \\ 4/15 & 0 & 4/5 & 2/5 \\ 4/15 & 2/5 & 0 & 0 \end{bmatrix} R_t + \begin{bmatrix} 1/20 \\ 1/20 \\ 1/20 \\ 1/20 \end{bmatrix}$

- 令初始 $R_0 = \begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}$

例

- 进行迭

$$R_1 = \begin{bmatrix} 0 & 2/5 & 0 & 0 \\ 4/15 & 0 & 0 & 2/5 \\ 4/15 & 0 & 4/5 & 2/5 \\ 4/15 & 2/5 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix} + \begin{bmatrix} 1/20 \\ 1/20 \\ 1/20 \\ 1/20 \end{bmatrix} = \begin{bmatrix} 9/60 \\ 13/60 \\ 25/60 \\ 13/60 \end{bmatrix}$$

$$R_2 = \begin{bmatrix} 0 & 2/5 & 0 & 0 \\ 4/15 & 0 & 0 & 2/5 \\ 4/15 & 0 & 4/5 & 2/5 \\ 4/15 & 2/5 & 0 & 0 \end{bmatrix} \begin{bmatrix} 9/60 \\ 13/60 \\ 25/60 \\ 13/60 \end{bmatrix} + \begin{bmatrix} 1/20 \\ 1/20 \\ 1/20 \\ 1/20 \end{bmatrix} = \begin{bmatrix} 41/300 \\ 53/300 \\ 153/300 \\ 53/300 \end{bmatrix}$$

例

- 最后得到

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}, \begin{bmatrix} 9/60 \\ 13/60 \\ 25/60 \\ 13/60 \end{bmatrix}, \begin{bmatrix} 41/300 \\ 53/300 \\ 153/300 \\ 53/300 \end{bmatrix}, \begin{bmatrix} 543/4500 \\ 707/4500 \\ 2543/4500 \\ 707/4500 \end{bmatrix}, \dots, \begin{bmatrix} 15/148 \\ 19/148 \\ 95/148 \\ 19/148 \end{bmatrix}$$

- 计算结果表明，结点C的PageRank值超过一半，其他结点也有相应的 PageRank值。

幂法

- 幂法 ((power method)是一个常用的PageRank计算方法，通过近似计算矩阵的主特征值和主特征向量求得有向图的一般PageRank
- 幂法主要用于近似计算矩阵的主特征值 (dominant eigenvalue) 和 主特征向量 (dominant eigenvector)
- 主特征值是指绝对值最大的特征值
- 主特征向量是其对应的特征向量
- 注意特征向量不是唯一的，只是其方向是确定的，乘上任意系数还是特征向量

幂法

- 假设要求 n 阶矩阵 A 的主特征值和主特征向量，采用下面的步骤。
- 首先，任取一个初始 n 维向量 x_0 ，构造如下的一个 n 维向量序列
- 然后， $x_0, x_1 = Ax_0, x_2 = Ax_1, \dots, x_k = Ax_{k-1}$
- 对应的 n 个线性无关 $|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_n|$
- 这 n 个特征向量构成 n 维空间的一组 $u_1, u_2, \dots, u_n$

幂法

- 于是, 可以将初始向量 $u_1, u_2, \dots, u_n$ 作为
的线性组合 $x_0 = a_1 u_1 + a_2 u_2 + \dots + a_n u_n$

- 得到

$$x_1 = Ax_0 = a_1 Au_1 + a_2 Au_2 + \dots + a_n Au_n$$

$$\vdots$$

$$x_k = A^k x_0 = a_1 A^k u_1 + a_2 A^k u_2 + \dots + a_n A^k u_n$$

$$= a_1 \lambda_1^k u_1 + a_2 \lambda_2^k u_2 + \dots + a_n \lambda_n^k u_n$$

幂法

- 接着 但这里左 λ_1 的特征估计 且特征方程的 $x_k = a_1 \lambda_1^k \left[u_1 + \frac{a_2}{a_1} \left(\frac{\lambda_2}{\lambda_1} \right)^k u_2 + \cdots + \frac{a_n}{a_1} \left(\frac{\lambda_n}{\lambda_1} \right)^k u_n \right]$

$$|\lambda_1| > |\lambda_j|, j = 2, \cdots, n$$

$$x_k = a_1 \lambda_1^k [u_1 + \varepsilon_k]$$

$$\varepsilon_k \quad k \rightarrow \infty$$

$$\varepsilon_k \rightarrow 0 \quad (k \rightarrow \infty)$$

- 由于 $x_k \rightarrow a_1 \lambda_1^k u_1 \quad (k \rightarrow \infty)$ 式(21.18)

- 这用 且当 时的无穷小量

幂法

- 说明当 k 充分大时向量 x_k 与特征向量 v_1 只相差一个系数 $x_k \approx a_1 \lambda_1^k u_1$ (18)知,
 $x_{k+1} \approx a_1 \lambda_1^{k+1} u_1$

$$\lambda_1$$

$$\lambda_1 \approx \frac{x_{k+1,j}}{x_{k,j}}$$

- 于是主特征值 可表示为

- 其中 $x_{k,j}$ 和 $x_{k+1,j}$ 分别是 x_k 和 x_{k+1} 的第 j 个分量

幂法

- 在实际计算时，为了避免出现绝对值过大或过小的情况， $y_{t+1} = Ax_t$ 一步迭代后即进行规范化，将向量 $x_{t+1} = \frac{y_{t+1}}{\|y_{t+1}\|}$ 范数，即

$$\|x\|_{\infty} = \max\{|x_1|, |x_2|, \dots, |x_n|\}$$

- 这里的范数是向量的无穷范数，即向量各分量的绝对值的最大值

幂法

- 现在回到计算一般PageRank。转移矩阵可以写作 $R = \left(dM + \frac{1-d}{n} \mathbf{E} \right) R = AR$

- 其中d是阻尼因子
- E是所有元素为1的n阶方阵
- 根据Perron-Frobenius定理，一般PageRank的向量R是矩阵A的主特征向量

计算一般PageRank的幂法

输入：含有 n 个结点的有向图，有向图的转移矩阵 M ，系数 d ，初始向量 x_0 ，计算精度 ε ；

输出：有向图的 PageRank R 。

- (1) 令 $t = 0$ ，选择初始向量 x_0
- (2) 计算有向图的一般转移矩阵 A

$$A = dM + \frac{1-d}{n}\mathbf{E}$$

- (3) 迭代并规范化结果向量

$$y_{t+1} = Ax_t$$

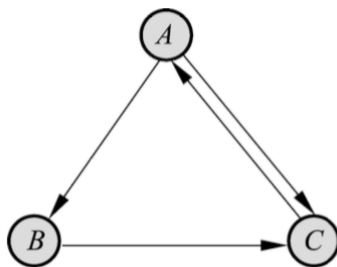
$$x_{t+1} = \frac{y_{t+1}}{\|y_{t+1}\|}$$

- (4) 当 $\|x_{t+1} - x_t\| < \varepsilon$ 时，令 $R = x_t$ ，停止迭代。
- (5) 否则，令 $t = t + 1$ ，执行步 (3)。
- (6) 对 R 进行规范化处理，使其表示概率分布。



例

- 给定一个如图所示的有向图，取 $d = 0.85$ ，求有向图的一般
- PageRank。



例

- 利用幂法，按照算法 21.2，计算有向图的一般 PageRank

$$M = \begin{bmatrix} 0 & 0 & 1 \\ 1/2 & 0 & 0 \\ 1/2 & 1 & 0 \end{bmatrix}$$

- 由图可知转移

$$x_0 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

- (1) 令 $t = 0$

例

- (2) 计算有向图的一般转移矩阵 A

$$A = dM + \frac{1-d}{n}\mathbf{E}$$

$$= 0.85 \times \begin{bmatrix} 0 & 0 & 1 \\ 1/2 & 0 & 0 \\ 1/2 & 1 & 0 \end{bmatrix} + \frac{0.15}{3} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.05 & 0.05 & 0.9 \\ 0.475 & 0.05 & 0.05 \\ 0.475 & 0.9 & 0.05 \end{bmatrix}$$

例

(3) 迭代并规范化

- (3)
$$y_1 = Ax_0 = \begin{bmatrix} 1 \\ 0.575 \\ 1.425 \end{bmatrix}$$
$$x_1 = \frac{1}{1.425} \begin{bmatrix} 1 \\ 0.575 \\ 1.425 \end{bmatrix} = \begin{bmatrix} 0.7018 \\ 0.4035 \\ 1 \end{bmatrix}$$
$$y_2 = Ax_1 = \begin{bmatrix} 0.05 & 0.05 & 0.9 \\ 0.475 & 0.05 & 0.05 \\ 0.475 & 0.9 & 0.05 \end{bmatrix} \begin{bmatrix} 0.7018 \\ 0.4035 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.9553 \\ 0.4035 \\ 0.7465 \end{bmatrix}$$
$$x_2 = \frac{1}{0.9553} \begin{bmatrix} 0.9553 \\ 0.4035 \\ 0.7465 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.4224 \\ 0.7814 \end{bmatrix}$$

例

如此继续迭代规范化, 得到 x_t , $t = 0, 1, 2, \dots, 21, 22$, 的向量序列

$$\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0.7018 \\ 0.4035 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0.4224 \\ 0.7814 \end{bmatrix}, \begin{bmatrix} 0.8659 \\ 0.5985 \\ 1 \end{bmatrix}, \begin{bmatrix} 0.9732 \\ 0.4912 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0.5516 \\ 0.9807 \end{bmatrix},$$
$$\begin{bmatrix} 0.9409 \\ 0.5405 \\ 1 \end{bmatrix}, \dots, \begin{bmatrix} 0.9760 \\ 0.5408 \\ 1 \end{bmatrix}, \begin{bmatrix} 0.9755 \\ 0.5404 \\ 1 \end{bmatrix}, \begin{bmatrix} 0.9761 \\ 0.5406 \\ 1 \end{bmatrix}, \begin{bmatrix} 0.9756 \\ 0.5406 \\ 1 \end{bmatrix}, \begin{bmatrix} 0.9758 \\ 0.5404 \\ 1 \end{bmatrix}$$

例

假设后面得到的两个向量已满足计算精度要求，那么取

$$R = \begin{bmatrix} 0.9756 \\ 0.5406 \\ 1 \end{bmatrix}$$

即得所求的一般 PageRank。如果将一般 PageRank 作为一个概率分布，进行规范化，使各分量之和为 1，那么相应的一般 PageRank 可以写作

$$R = \begin{bmatrix} 0.3877 \\ 0.2149 \\ 0.3974 \end{bmatrix}$$



代数算法

- 代数算法通过一般转移矩阵的逆矩阵计算求有向图的一般PageRank
- 按照一般PageRank的定义式 (21.14)

$$R = dMR + \frac{1-d}{n}\mathbf{1}$$

- 于是

$$(I - dM)R = \frac{1-d}{n}\mathbf{1} \quad (21.23)$$

- 这里I是单位矩阵。当 $0 < d < 1$ 时, (21.23) 的解存在且唯一
- 这样, 可以通过求逆矩阵 $(I - dM)^{-1}$ 得到有向图的一般PageRank